SE notes

1.2 Defining the Discipline of Software Engineering

The Official Definition

- **IEEE Definition [IEE17]:** Software engineering is the application of a systematic, disciplined, and quantifiable approach to developing, operating, and maintaining software—essentially, applying engineering principles to software.
- **Balancing Discipline and Agility:** A strict, systematic approach that works well for one software team might feel like a heavy burden to another. Therefore, effective software engineering requires a balance of **discipline**, **adaptability**, and **agility**.

FIGURE 1.3

Software engineering layers



Software Engineering as a Layered Technology (Figure 1.3) Software engineering is structured like a stack of four interconnected layers, built from the bottom up:

Layer (Bottom to Top)	Role in Software Engineering	Key Details & Examples
1. A Quality Focus *(The Bedrock)*	Serves as the fundamental base supporting all of software engineering.	Rests on an organization's commitment to quality. Philosophies like **Total Quality Management (TQM)** and **Six Sigma** create a culture of continuous process improvement, which leads to more effective software engineering.
2. Process *(The Foundation)*	Acts as the "glue" holding the technology layers together and enables rational, on-time software development.	Defines the framework for delivering software technology and provides the basis for management control. It sets the context for applying technical methods, producing work products (models, documents, data, reports, forms, etc.), setting milestones, ensuring quality, and managing change.
3. Methods *(Technical "How-To's")*	Provides the practical technical instructions for actually building the software.	Covers a wide range of tasks: communication, requirements analysis, design modeling, program construction, testing, and support. Methods rely on basic governing principles and include modeling activities and descriptive techniques.

4. Tools *(Automated Support)*	Provides automated or semi-automated support for the **process** and **methods** layers.	When tools are integrated so that information created by one tool can be used by another, it creates a system called **Computer-Aided Software Engineering (CASE)**.

1.3 The Software Process

What is a Software Process?

- **Core Definition:** A process is a collection of **activities**, **actions**, and **tasks** performed to create a work product.
- **Flexible, Not Rigid:** A software process is not a strict set of rules. Instead, it is an adaptable approach that lets the software team pick and choose the right actions and tasks for their specific project.
- **Ultimate Goal:** To always deliver software on time and with enough quality to satisfy both the project sponsors and the end users.

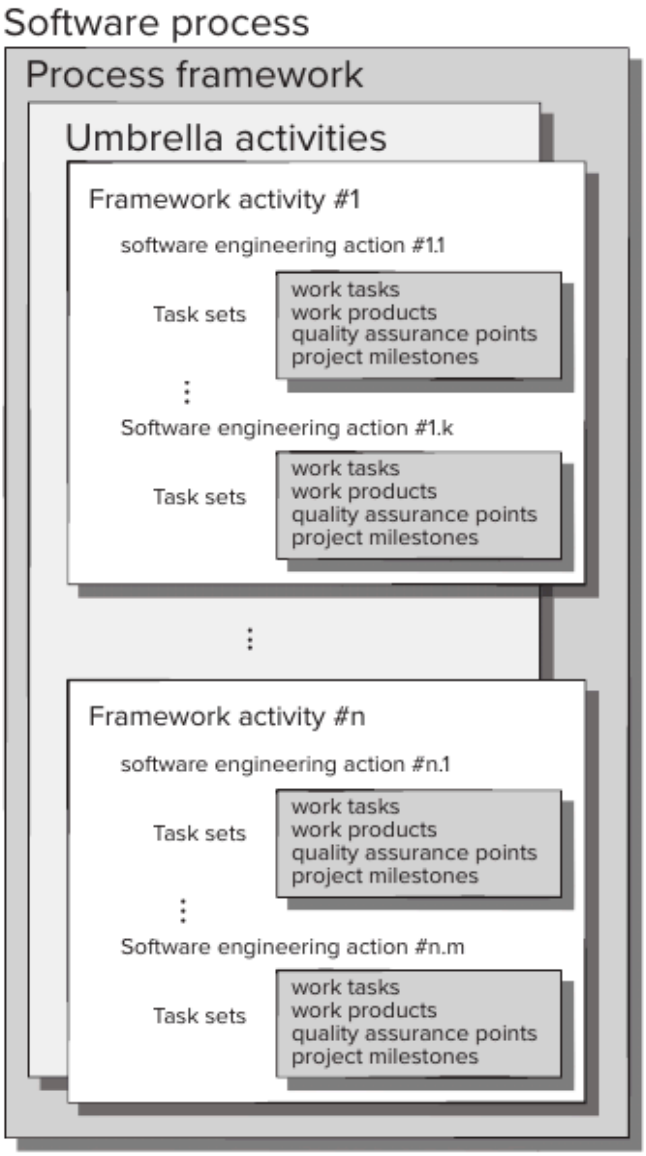
The Process Hierarchy (From Broadest to Smallest)

1. **Activity:** Aims to achieve a broad objective and is applied to every project regardless of the application domain, project size, complexity, or level of strictness (rigor). Example: Communicating with stakeholders.
2. **Action:** A set of tasks grouped together to produce a major work product. Example: Architectural design (which produces an architectural model).
3. **Task:** Focuses on a small, clearly defined goal that produces a concrete, tangible result. Example: Conducting a unit test.

Who is a Stakeholder? A stakeholder is anyone who has an interest ("a stake") in the successful outcome of the project, including business managers, end users, software engineers, and support staff. *(As consultant Rob Thomsett jokes: "A stakeholder is a person holding a large and sharp stake... If you don't look after your stakeholders, you know where the stake will end up.")*

FIGURE 2.1

A software process framework



1.3.1 The Process Framework

A **process framework** forms the foundation for a complete software engineering process. It consists of two main parts: a small set of **framework activities** (used on all projects regardless of size or complexity) and a set of **umbrella activities** (applied across the entire process).

The 5 Generic Framework Activities

1. **Communication:** Before any technical work starts, the team must communicate and collaborate with the customer and other stakeholders. The goal is to understand their project objectives and gather requirements to define the software's features and functions.

2. **Planning:** Because a software project is a complex journey, planning creates a "map"—called a **software project plan**—to guide the team. This plan outlines the technical tasks to be done, likely risks, required resources, work products to be created, and a work schedule.

3. **Modeling:** Just like landscapers, bridge builders, aeronautical engineers, carpenters, and architects create sketches to see the "big picture" (what something looks like architecturally and how the pieces fit

together) and refine those sketches into greater detail to solve problems, software engineers create **models**. These models help the team better understand the software requirements and the design needed to achieve them.

4. **Construction:** Turning the design into actual software. This combines **code generation** (writing code manually or automatically) and **testing** (finding errors in the code).

5. **Deployment:** Delivering the software (either as a finished product or a partially completed increment) to the customer, who then evaluates it and gives feedback.

How Framework Activities Work in Practice

- **Universal Application:** These same five activities are used whether you are building small, simple programs, Web applications, or huge, complex computer-based systems—even though the specific details of the process will differ widely for each.

- **Iterative Development:** In many projects, these five activities are repeated in cycles (**iterations**). Each pass through the activities produces a **software increment**—a working piece of software that gives stakeholders a subset of the overall features and functionality. With each new increment, the software becomes more and more complete.

1.3.2 Umbrella Activities

Umbrella activities run continuously in the background throughout the entire software project. They complement the five framework activities by helping the team manage and control progress, quality, change, and risk.

- **Software Project Tracking and Control:** Checks the team's actual progress against the project plan and takes action if needed to stay on schedule.

- **Risk Management:** Identifies and evaluates potential risks that could hurt the project's outcome or the product's quality.

- **Software Quality Assurance (SQA):** Defines and carries out the activities necessary to guarantee the quality of the software.

- **Technical Reviews:** Inspects software work products to find and fix errors early, before they are passed on (propagated) to the next activity.

- **Measurement:** Defines and collects data (measures) about the process, project, and product to help the team meet stakeholder needs; it can be used alongside all other framework and umbrella activities.

- **Software Configuration Management:** Manages and controls the effects of changes made throughout the software process.

- **Reusability Management:** Sets the rules for reusing existing work products (including software components) and creates ways to build reusable components.

- **Work Product Preparation and Production:** Covers all the steps required to create project deliverables, such as models, documents, logs, forms, and lists.

2.1 A Generic Process Model

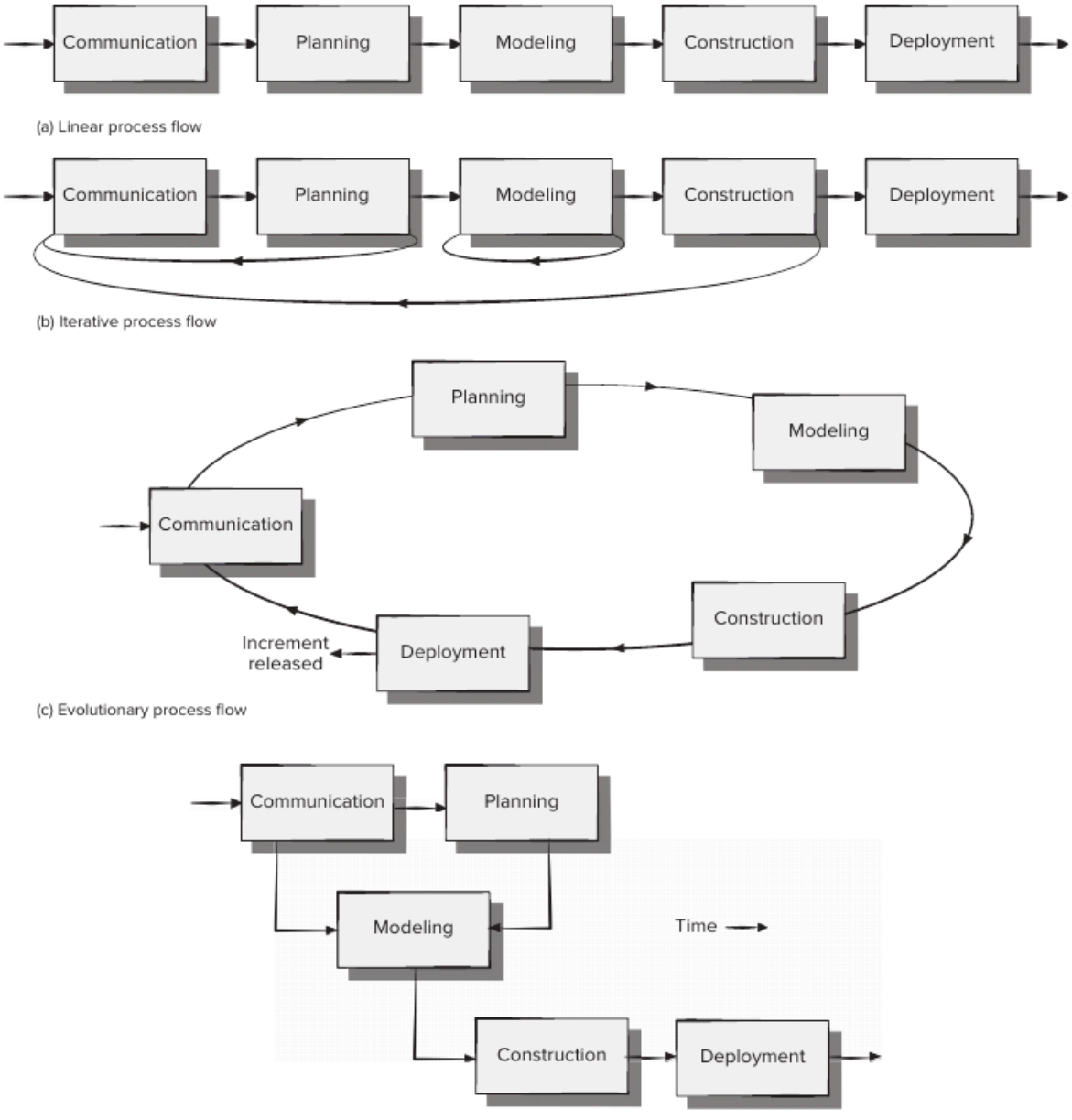
How a Software Process is Structured (Figure 2.1)

- **Core Concept:** As defined in Chapter 1, a process is a collection of activities, actions, and tasks performed to build a work product. All of these live inside a **framework** or **model** that defines how they relate to the overall process and to each other.

- **The 5 Framework Activities:** Communication, planning, modeling, construction, and deployment.

- **Umbrella Activities:** Applied continuously throughout the entire process (includes project tracking and control, risk management, quality assurance, configuration management, technical reviews, and others).
- **Inside Each Framework Activity:**
    - Every framework activity (#1 through #n) contains a set of **software engineering actions** (e.g., action #1.1 to #1.k).
    - Every software engineering action is defined by a **Task Set**, which specifies four things:
        1. **Work tasks** to be completed.
        2. **Work products** that will be produced.
        3. **Quality assurance points** that are required.
        4. **Project milestones** used to measure and show progress.

FIGURE 2.2 Process flow



Process Flow (Figure 2.2) Process flow describes how the five framework activities—along with the actions and tasks inside them—are organized in terms of **sequence** and **time**. There are four types of process flow:

Process Flow Type	How It Works	Visual Pattern (Figure 2.2)
1. Linear Process Flow	Executes each of the five framework activities one after another in a straight sequence, starting with **Communication** and ending with **Deployment**.	$Communication \rightarrow Planning \rightarrow Modeling \rightarrow Construction \rightarrow$
2. Iterative Process Flow	Repeats one or more of the activities before moving on to the next activity.	Loops back to repeat earlier steps (e.g., repeating Modeling, or looping from Planning back to Communication, or Construction back to Communication) before Deployment.
3. Evolutionary Process Flow	Executes the five activities in a continuous "circular" loop. Each complete circuit around the five activities releases an increment and produces a more complete version of the software.	A circle moving clockwise through Communication → Planning → Modeling → Construction → Deployment (Increment released) → back to Communication.
4. Parallel Process Flow	Executes one or more activities at the exact same time as other activities.	*Example:* **Modeling** for one part of the software runs in parallel with the **Construction** of another part as time moves forward.

2.5 Prescriptive Process Models

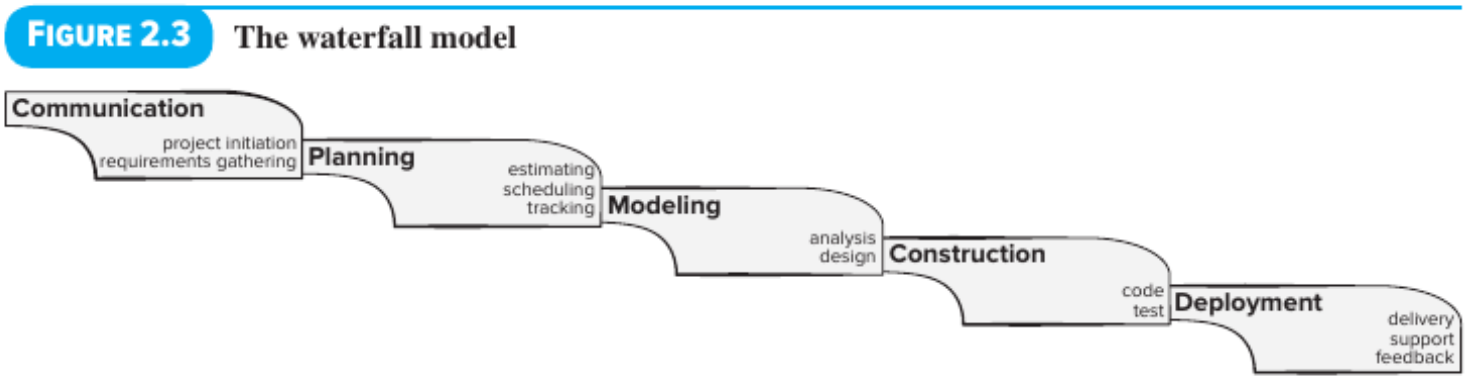
What Are Prescriptive Process Models?

- **Definition:** Also known as **"traditional" process models**, prescriptive process models lay out a predefined set of process elements and a predictable workflow (**process flow**).
- **Main Goal:** They aim to bring structure, order, and project consistency to software development. Activities and tasks happen in sequence with clear guidelines for measuring progress.
- **Why They Are Called "Prescriptive":** Because they *prescribe* (specify upfront) two things for every project:
    1. **Process Elements:** Framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanisms.
    2. **Process Flow (Work Flow):** How all of those process elements connect and interact with one another.

- **Common Ground, Different Emphasis:** Every software process model can fit the five generic framework activities from Chapter 1, but each model places a different emphasis on those activities and organizes the workflow in its own distinct way.
- **The Central Dilemma:** Software engineers face a tough question: *Are structured, prescriptive models suitable for a modern software world driven by constant change? Conversely, if we abandon traditional models for something less structured, do we make it impossible to keep our work coordinated and coherent?* (There are no easy answers, which is why different models exist for different project needs.)

2.5.1 The Waterfall Model

- **What It Is:** Also called the **linear sequential model**, the waterfall model is the oldest paradigm in software engineering. It takes a systematic, step-by-step approach where work flows in a straight line from customer requirements all the way to ongoing support of the finished software.
- **Historical Note on Winston Royce [Roy70]:** When Winston Royce originally proposed the waterfall model in 1970, he actually included provisions for "feedback loops." However, the vast majority of organizations that use it treat it as strictly linear.
- **When It Works Best:** When the requirements for a problem are well understood and reasonably stable.
    - *Situation 1 (Adaptations/Enhancements):* Making well-defined updates to an existing system (for example, updating accounting software to comply with new government regulations).
    - *Situation 2 (New Development):* A small number of new projects, **only** if the requirements are clearly defined upfront and unlikely to change.



The 5 Sequential Phases of the Waterfall Model (Figure 2.3)

1. **Communication:** Project initiation and requirements gathering.
2. **Planning:** Estimating, scheduling, and tracking.
3. **Modeling:** Analysis and design.
4. **Construction:** Code and test.
5. **Deployment:** Delivery, support, and feedback.

Why the Waterfall Model is Criticized (4 Major Problems)

Over the last 50 years, even strong supporters have questioned its effectiveness due to four common problems:

1. **Unrealistic Flow:** Real-world projects rarely follow the strict step-by-step sequence the model proposes.

2. **Upfront Uncertainty:** It is usually very hard for customers to state every single requirement clearly at the very beginning of a project.
3. **Delayed Working Software:** The customer must be patient because a working version of the program will not be ready until very late in the project timeline.
4. **Late Error Detection:** Major mistakes ("blunders") might not be caught until the working program is finally reviewed at the end.

- **Modern Verdict:** Because today's software development is fast-paced and faces a non-stop stream of changes (to features, functions, and information), the waterfall model is often inappropriate.

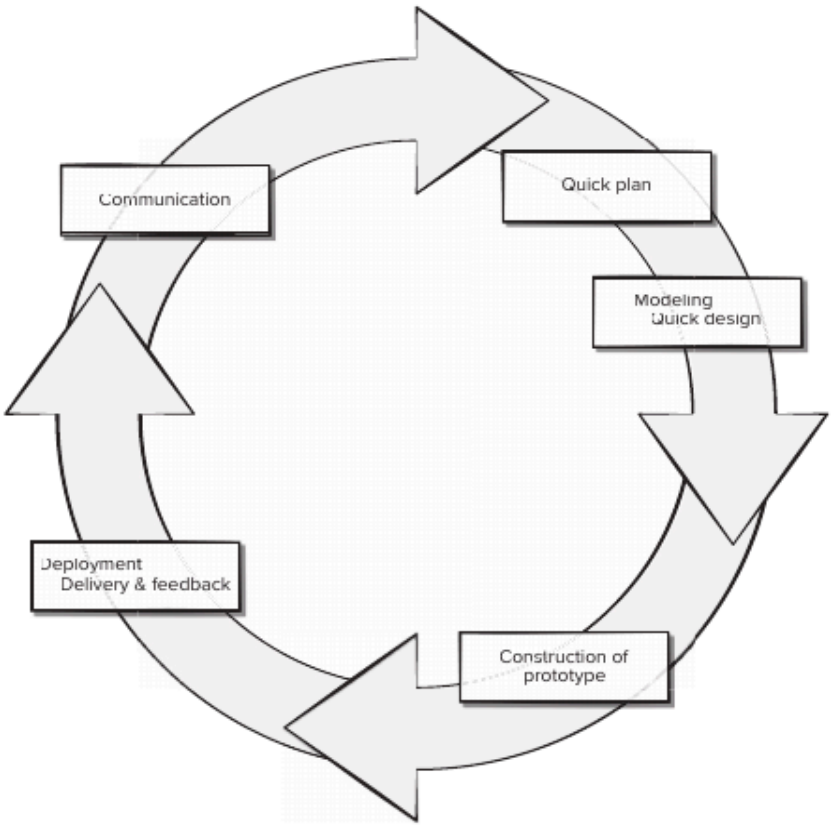
2.5.2 Prototyping Process Model

Overview & When to Use It

- **What It Is:** A process paradigm used to help software engineers and stakeholders better understand what needs to be built when requirements are unclear ("fuzzy").
- **How It Is Used:** It can be used as a **stand-alone process model**, but it is more commonly used as a **technique** inside any of the other process models.
- **When It Works Best:**
    - When a customer knows the general goals for the software but cannot specify detailed requirements for functions and features.
    - When the developer is unsure about how efficient an algorithm will be, how well an operating system will adapt, or what the user interface (human-machine interaction) should look like.

FIGURE 2.4

The prototyping paradigm



Real-World Example: Building a Fitness App with Incremental Prototypes

- **1st Prototype:** Delivers the basic user interface (UI) screens needed to sync a smartphone with a fitness device and display current data.
- **2nd Prototype:** Adds the ability to set goals and store fitness data in the cloud, while creating and updating UI screens based on customer feedback from the first prototype.
- **3rd Prototype:** Adds social media integration so users can set fitness goals and share their progress with friends.

How the Prototyping Cycle Works (Figure 2.4)

1. **Communication:** Meet with stakeholders to define overall software goals, list whatever requirements are already known, and identify areas that need more definition.
2. **Quick Plan:** Plan the prototyping iteration rapidly.
3. **Modeling (Quick Design):** Create a rapid design focusing on the parts of the software that end users will actually see (such as user interface layouts or output display formats).
4. **Construction of Prototype:** Build the prototype based on the quick design. *(Tip: To build a working prototype fast, developers can reuse existing code fragments or use automated tools that generate working programs quickly.)*
5. **Deployment, Delivery & Feedback:** Deliver the prototype to stakeholders, who test it and give feedback to help refine the requirements.

- **Continuous Iteration:** The loop repeats to fine-tune the prototype for stakeholders while giving developers a clearer picture of what needs to be built.

Pros, Cons, and "Rules of the Game"

- **Why People Like It:** Users get a hands-on feel for the actual system early on, and developers get to start building something immediately.
- **2 Major Risks/Problems:**
    1. *Misleading Appearance of Quality:* Stakeholders see what looks like a working program and don't realize the underlying architecture (program structure) is still evolving—meaning developers may not have built in overall quality or long-term maintainability yet.
    2. *Permanent Shortcuts:* Developers might take shortcuts ("implementation compromises") just to get the prototype running fast. If they aren't careful, these poor technical choices stay in the code and become a permanent part of the final system.
- **The Key to Success ("Rules of the Game"):** All stakeholders must agree at the very beginning that the prototype is built partly as a tool to discover requirements. While it is nice to evolve a prototype into the final product, developers must be prepared to **throw away (discard)** at least part of the prototype to properly meet the customer's evolving needs.

2.5.3 Evolutionary Process Model (The Spiral Model)

Why Evolutionary Models Are Needed

- Software evolves over time. Business and product requirements frequently change mid-project, making a straight-line path to the final product unrealistic.
- Tight market deadlines can make it impossible to build a complete software product all at once. If a company misses its market window, the entire project can become meaningless.

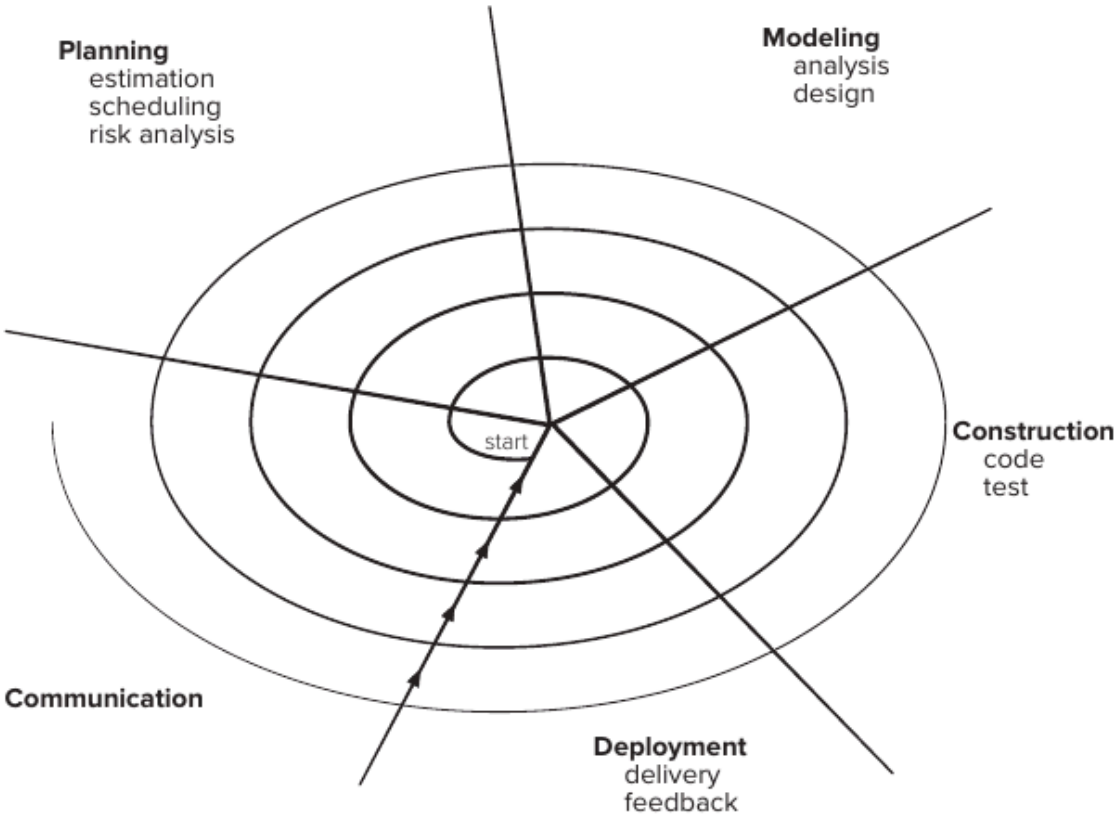
- **The Solution:** Release a limited version of the product quickly to meet business or competitive pressure, and then release refined versions once all system features are better understood. Evolutionary models are designed to build high-quality software iteratively/incrementally while prioritizing flexibility, extensibility, speed, and **customer satisfaction** (the ultimate judge of software quality).

The Spiral Model (Figure 2.5)

- **Origin & Core Concept:** Originally proposed by **Barry Boehm [Boe88]**, the spiral model combines the **iterative nature of prototyping** with the **controlled, systematic aspects of the waterfall model**.
- **Evolutionary Releases:** Software is built in a series of evolutionary releases:
    - *Early iterations:* The release might just be a paper model or a prototype.
    - *Later iterations:* Increasingly complete and sophisticated versions of the engineered system are produced.

FIGURE 2.5

A typical spiral model



How the Spiral Model Works Step-by-Step (Figure 2.5)

- **Structure:** The spiral is divided into segments representing the framework activities defined by the team:
    - **Communication**
    - **Planning** (estimation, scheduling, risk analysis)
    - **Modeling** (analysis, design)
    - **Construction** (code, test)
    - **Deployment** (delivery, feedback)

- **Clockwise Movement from the Center:** The team starts at the center of the spiral and moves outward in a **clockwise direction**:
    - *1st Circuit (innermost loop):* Might result in a **product specification**.
    - *2nd Circuit:* Might be used to build a **prototype**.
    - *Subsequent Circuits:* Produce progressively more complete versions of the software.
- **Continuous Risk Analysis:** Risk is actively evaluated during **every single pass** around the spiral.
- **Anchor Point Milestones:** Represented by the arrows pointing outward along the spiral's axis in Figure 2.5, these are specific combinations of work products and conditions achieved during each pass around the spiral to track progress.
- **Dynamic Planning:** Every time the team passes through the **Planning** region, they adjust the project plan:
    - Cost and schedule are updated based on customer feedback after each delivery.
    - The project manager adjusts the planned number of iterations needed to finish the software.
- **Full Lifecycle Coverage:** Unlike other models that end once the software is delivered, the spiral model can be adapted to continue throughout the entire life of the software.

Strengths and Weaknesses of the Spiral Model

- **Strengths:**
    - A realistic approach for building **large-scale systems and software**.
    - Uses prototyping specifically as a **risk-reduction mechanism**.
    - Forces the team to directly confront technical risks at every stage, solving them before they turn into major problems.
- **Weaknesses:**
    - It can be hard to convince customers (especially in strict contract situations) that an evolutionary approach with a changing plan is controllable.
    - It relies heavily on **risk assessment expertise**—if a major risk goes unnoticed and unmanaged, serious problems will occur.

Case Study Summary: SafeHome – Selecting a Process Model (Parts 1 & 2)

- **The Scenario:** The software engineering team at CPI Corporation—Lee Warren (engineering manager), Doug Miller (software engineering manager), and team members Jamie, Vinod, and Ed—are choosing a process model for *SafeHome*, a complex new product line with a tight deadline.
- **Why They Reject Ad Hoc Work:** Their disorganized ("ad hoc") approach on past projects caused a lot of grief and won't work for a project as big and fast-paced as *SafeHome*.
- **Why They Reject the Waterfall Model:** They don't have all requirements upfront, and Vinod notes that a linear approach feels too rigid and "IT-oriented" (better suited for a standard inventory control system than *SafeHome*).
- **Why They Hesitate on Pure Prototyping:** While Ed likes prototyping because it's similar to how they already work, Vinod worries it doesn't provide enough structure for a large project.

- **Why They Favor the Spiral / Evolutionary Approach:** Jamie and Vinod love the spiral model ("keepin' it real") because they can release a working increment to the market fast, learn from customer feedback, re-plan, and add features in each new increment.
- **The Management Conflict:** Lee (the engineering manager) objects to updating the plan on every loop of the spiral because senior management wants a single fixed plan and schedule. Doug pushes back, arguing that a fixed plan is "old-school thinking"—tweaking the plan as requirements change reflects reality, and senior management needs to be reeducated.

SAFEHOME



Selecting a Process Model, Part 2

The scene: Meeting room for the software engineering group at CPI Corporation, a company that makes consumer products for home and commercial use.

The players: Lee Warren, engineering manager; Doug Miller, software engineering manager; Vinod and Jamie, members of the software engineering team.

The conversation: (Doug describes evolutionary process options.)

Jamie: Now I see something I like. An incremental approach makes sense, and I really like the flow of that spiral model thing. That's keepin' it real.

Vinod: I agree. We deliver an increment, learn from customer feedback, re-plan, and then deliver another increment. It also fits into the nature of the product. We can have something on the market fast and then add functionality with each version, er, increment.

Lee: Wait a minute. Did you say that we regenerate the plan with each tour around the spiral, Doug? That's not so great; we need one plan, one schedule, and we've got to stick to it.

Doug: That's old-school thinking, Lee. Like the guys said, we've got to keep it real. I submit that it's better to tweak the plan as we learn more and as changes are requested. It's way more realistic. What's the point of a plan if it doesn't reflect reality?

Lee (frowning): I suppose so, but . . . senior management's not going to like this . . . they want a fixed plan.

Doug (smiling): Then you'll have to reeducate them, buddy.

SAFEHOME



Selecting a Process Model, Part 1

The scene: Meeting room for the software engineering group at CPI Corporation, a (fictional) company that makes consumer products for home and commercial use.

The players: Lee Warren, engineering manager; Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member; and Ed Robbins, software team member.

The conversation:

Lee: So let's recapitulate. I've spent some time discussing the *SafeHome* product line as we see it at the moment. No doubt, we've got a lot of work to do to simply define the thing, but I'd like you guys to begin thinking about how you're going to approach the software part of this project.

Doug: Seems like we've been pretty disorganized in our approach to software in the past.

Ed: I don't know, Doug, we always got product out the door.

Doug: True, but not without a lot of grief, and this project looks like it's bigger and more complex than anything we've done in the past.

Jamie: Doesn't look that hard, but I agree . . . our ad hoc approach to past projects won't work here, particularly if we have a very tight time line.

Doug (smiling): I want to be a bit more professional in our approach. I went to a short course last week and learned a lot about software engineering . . . good stuff. We need a process here.

Jamie (with a frown): My job is to build computer programs, not push paper around.

Doug: Give it a chance before you go negative on me. Here's what I mean. (Doug proceeds to describe the process framework described in Chapter 1 and the prescriptive process models presented to this point.)

Doug: So anyway, it seems to me that a linear model is not for us . . . assumes we have all requirements up front and, knowing this place, that's not likely.

Vinod: Yeah, and it sounds way too IT-oriented . . . probably good for building an inventory control system or something, but it's just not right for *SafeHome*.

Doug: I agree.

Ed: That prototyping approach seems okay. A lot like what we do here anyway.

Vinod: That's a problem. I'm worried that it doesn't provide us with enough structure.

Doug: Not to worry. We've got plenty of other options, and I want you guys to pick what's best for the team and best for the project.

Quick Comparison Summary of Prescriptive Models

Feature	Waterfall Model	Prototyping Model	Spiral Model (Evolutionary)
Process Flow	Strictly Linear / Sequential	Iterative Loop	Evolutionary (Clockwise Spiral)
Best Used When...	Requirements are clear, fixed, and well-understood upfront (e.g., regulatory updates).	Requirements are "fuzzy" or UI/algorithm feasibility is uncertain.	Building large-scale systems where requirements evolve, deadlines are tight, and risks are high.

Main Strength	Simple, structured, and easy to understand when requirements don't change.	Users see a working model fast; great for clarifying fuzzy requirements.	Combines prototyping's flexibility with waterfall's control; actively reduces technical risk at every stage.
Main Weakness	Working software arrives very late; cannot handle changing requirements well.	Shortcuts taken to build the prototype fast may hurt long-term quality/architecture.	Hard to sell to clients wanting a fixed contract/schedule; requires strong risk-assessment skills.

3.1 What Is Agility?

The Core Driver of Agility

- **Why Agility Exists:** According to Ivar Jacobson [Jac02a], the primary driver for agility is the **pervasiveness of change** (the fact that change happens everywhere, all the time). Software engineers must be "quick on their feet" to handle rapid, ongoing changes.
- **Alternative Names:** Agile methods are also sometimes called **light methods** or **lean methods**.

The Agile Philosophy

Agility is more than just reacting quickly to change; it also follows the philosophy of the Agile Manifesto:

- **Easier Communication:** Encourages team structures and attitudes that make communication much easier among team members, between technical staff and business people, and between software engineers and their managers.
- **Rapid Delivery Over Intermediate Documents:** Focuses on delivering working ("operational") software quickly while placing less importance on intermediate work products like heavy documentation *(note: the text points out that deemphasizing intermediate work products is not always a good thing)*.
- **Customer as a Team Member:** Brings the customer directly into the development team to eliminate the "us versus them" mindset that harms many software projects.
- **Flexible Planning:** Accepts that planning in an uncertain world has limits, meaning the project plan must stay flexible.

4 Requirements to Apply Agility to Any Software Process

Agility can be applied to *any* software process, provided the process is designed to do four things:

1. **Adapt and Streamline Tasks:** Allow the project team to adjust tasks and make them simpler and faster.
2. **Fluid Planning:** Plan in a way that accounts for the fluid, changing nature of agile development.
3. **Lean Work Products:** Cut out all non-essential work products and keep the necessary ones as lean (simple and lightweight) as possible.

4. **Incremental Delivery Strategy:** Focus on delivering working software increments to the customer as fast as realistically possible for that specific product type and operating environment.

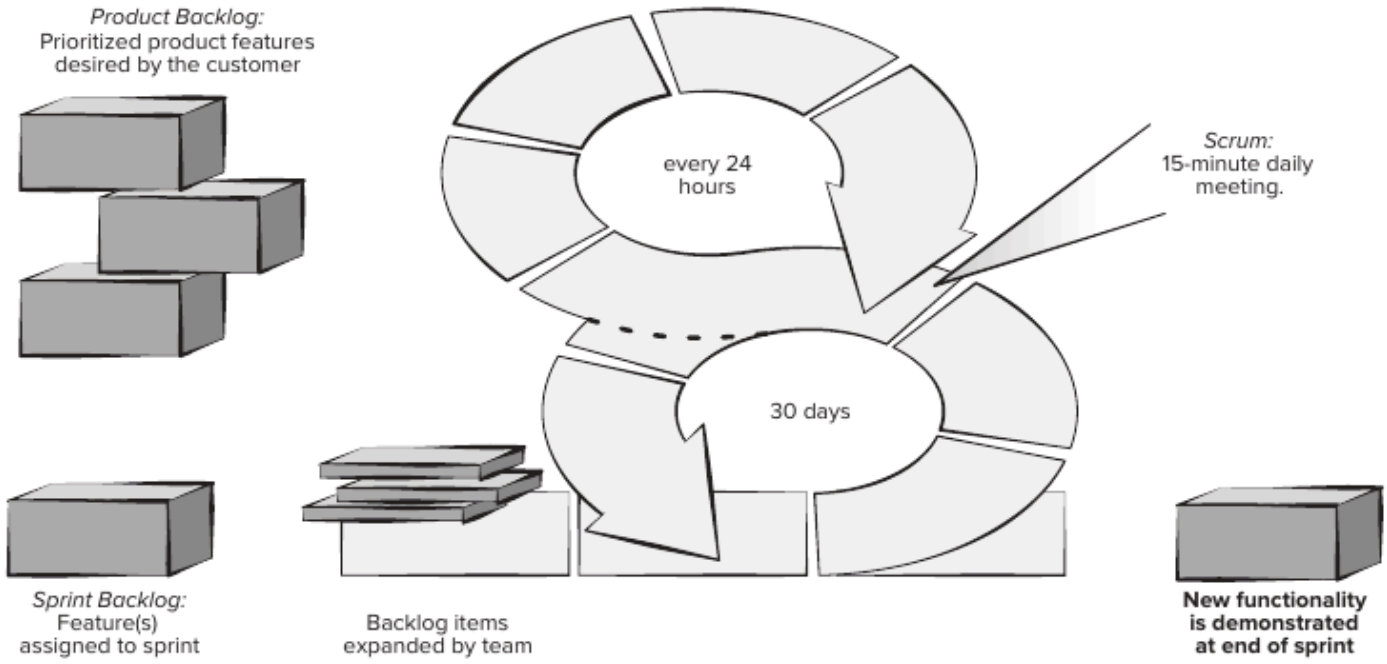
3.4 Scrum

- **Origin of the Name:** Derived from an activity in a rugby match (where players lock together to restart play).
- **Creators:** Conceived by **Jeff Sutherland** and his development team in the early 1990s, with further development by **Schwaber and Beedle [Sch01b]** (and additional framework descriptions by **Fowler and Sutherland [Fow16]**).
- **Scrum Framework Activities:** Scrum principles align with the Agile Manifesto and guide work across five framework activities:
    1. **Requirements**
    2. **Analysis**
    3. **Design**
    4. **Evolution**
    5. **Delivery**

How Sprints Work (Figure 3.2)

- **What is a Sprint?** Inside each framework activity, work tasks happen in short, **time-boxed** periods called **sprints** (allocated a fixed maximum time, shown as a **30-day** cycle in Figure 3.2, and typically lasting **2 to 4 weeks** or **3 to 4 weeks** in practice).
- **Adaptability:** The number of sprints needed for each framework activity depends on the product's size and complexity.
- **Real-Time Modification:** Work done inside a sprint is tailored to the specific problem and is defined—and often modified in real time—by the Scrum team.
- **The Visual Flow (Figure 3.2):**
    1. **Product Backlog:** A stack of prioritized product features desired by the customer.
    2. **Sprint Backlog:** Specific feature(s) assigned to the current sprint, where backlog items are expanded by the team.
    3. **The Sprint Cycle (30 Days) & Daily Scrum (Every 24 Hours):** The team works through a 30-day sprint loop while holding a 15-minute daily Scrum meeting every 24 hours.
    4. **Sprint Output:** New functionality is demonstrated at the end of the sprint.

FIGURE 3.2 Scrum process flow



3.4.1 Scrum Teams and Artifacts

The Scrum Team Structure

A Scrum team is a **self-organizing, interdisciplinary team** made up of three roles:

Role	Size	Key Responsibilities & Authority
Product Owner	1 person	• Orders/prioritizes items in the product backlog to meet stakeholders' most important goals. • Approves adding new items to the product backlog (with the development team's consent). • **Exclusive Authority:** The *only* person who can decide to end a sprint early (prematurely) or extend a sprint if the increment is not accepted.
Scrum Master	1 person	• Acts as a **facilitator** for the entire Scrum team. • Runs the Daily Scrum meeting and removes obstacles reported by the team. • Coaches development team members to help each other finish sprint tasks when they have free time. • Helps the product owner find ways to manage the product backlog and ensures backlog items are written clearly and concisely.
Development Team	3 to 6 people	• Plans how to deliver the selected features for the current sprint's goal (as negotiated with the product owner).

		• **Full Autonomy:** Decides *how* to build the increment, which roles are needed, and when the increment is **"done"** and ready to demo to the product owner.

The 3 Principal Scrum Artifacts

1. **Product Backlog**
    - **What It Is:** A prioritized list of product requirements or features that deliver business value to the customer.
    - **Adding Items:** Items can be added at any time, provided they have the **approval of the product owner** and the **consent of the development team**.
    - **Never Finished:** Because the product continually evolves to meet stakeholder needs, the product backlog is never completely finished.
2. **Sprint Backlog**
    - **What It Is:** The specific subset of product backlog items chosen by the team to be built as the code increment during the current active sprint.
    - **Strict Rule on Changes: No new features** can be added to the sprint backlog once the sprint starts—unless the entire sprint is cancelled and restarted.
3. **The Code Increment (or simply "The Increment")**
    - **What It Is:** The combination ("union") of all product backlog items finished in previous sprints **plus** all backlog items completed in the current sprint.
    - **Sprint Length:** Text notes that projects are broken into incremental prototype periods of **2 to 4 weeks**, with most sprints time-boxed at **3 to 4 weeks**.

3.4.2 to 3.4.5 Scrum Meetings & Ceremonies

1. Sprint Planning Meeting (Section 3.4.2)

- **Pre-Meeting Preparation (Building & Ranking the Backlog):**
    1. Before starting, the development team works with the product owner and all other stakeholders to create the items in the product backlog.
    2. The product owner and development team rank backlog items based on two factors:
        1. **Importance** of the product owner's business needs.
        2. **Complexity** of the software engineering tasks (programming and testing) required to finish each item.
    3. *Benefit:* This ranking process often reveals missing features that are necessary to deliver the required functionality to end users.
- **What Happens During Sprint Planning:**
    1. **Goal Setting:** Before the sprint begins, the product owner states the development goal for the upcoming sprint's increment.
    2. **Selecting Items:** The Scrum master and development team choose which items to move from the product backlog into the sprint backlog.

3. **Estimating Work & Capacity:** The development team figures out how much can realistically be delivered within the sprint's time-box and works with the Scrum master to define the work needed to deliver it.

4. **Assigning Roles:** The development team decides what roles are needed for the sprint and how team members will fill them.

2. Daily Scrum Meeting (Section 3.4.3)

- **Duration & Timing:** A **15-minute** time-boxed meeting held at the **start of every workday** to help the team sync their work and plan for the next 24 hours.
- **Who Attends:** The **Scrum master** and **development team** *always* attend. Some teams let the **product owner** attend occasionally.
- **The 3 Key Questions Every Member Must Answer:**
    - *What did you do since the last team meeting?*
    - *What obstacles are you encountering?*
    - *What do you plan to accomplish by the next team meeting?*
- **Key Rules & Benefits:**
    - **Early Problem Detection:** Led by the Scrum master (who evaluates everyone's answers), the meeting spots potential problems as early as possible. The Scrum master works to clear any reported obstacles before the next day's meeting.
    - **No Live Problem-Solving:** This is *not* a problem-solving meeting—actual problem-solving discussions happen offline and only include the specific people affected.
    - **Knowledge Socialization [Bee99]:** Sharing daily updates spreads knowledge across the group, which helps maintain a **self-organizing team structure**.
    - **Tracking "Done" Items:** Some teams use this meeting to mark sprint backlog items as complete ("done"). Once all sprint backlog items are complete, the team can schedule a demo and review with the product owner.

3. Sprint Review Meeting (Section 3.4.4)

- **When & Duration:** Held at the **end of the sprint** once the development team judges the increment complete. Typically time-boxed at **4 hours** for a 4-week sprint.
- **Who Attends:** Scrum master, development team, product owner, and selected stakeholders.
- **Primary Activity (The Demo):** The team gives a live demonstration of the software increment built during the sprint.
    - *Important Reality Check:* The demo might not include every single feature originally planned, but rather the functions that could actually be finished within the sprint's fixed time-box.
- **Outcomes:**
    - The product owner decides whether to **accept** the increment as complete or **reject** it.
    - If **not accepted**, the product owner and stakeholders give feedback so a new round of sprint planning can happen.
    - **Updating the Product Backlog:** This is the point where new features can be added to or removed from the product backlog, which may change what gets built in the next sprint.

4. Sprint Retrospective (Section 3.4.5)

- **When & Duration:** Scheduled by the Scrum master at the end of a sprint, **ideally right before the next Sprint Planning Meeting**. Time-boxed at **3 hours** for a 4-week sprint.
- **Who Attends:** Scrum master and the development team.
- **The 3 Core Discussion Topics:**
    - *What went well in the sprint*
    - *What could be improved*
    - *What the team will commit to improving in the next sprint*
- **Outcomes:**
    - The Scrum master leads the session and encourages the team to sharpen its development practices so they are more effective in the next sprint.
    - The team plans ways to boost product quality by updating and adapting its **definition of "done."**
    - The team leaves the meeting knowing exactly what improvements are needed and ready to plan the next increment.

Summary Table: Scrum Meetings at a Glance

Meeting	**When It Happens**	**Time-Box (for a 4-week sprint)**	**Attendees**	**Main Purpose**
Sprint Planning	Before each sprint starts	Not specified	Product Owner, Scrum Master, Dev Team	Set sprint goal, move items to Sprint Backlog, plan work and roles.
Daily Scrum	Start of each workday (every 24 hrs)	**15 minutes**	Scrum Master, Dev Team *(Product Owner occasionally)*	Answer 3 questions, sync next 24 hours, identify obstacles for Scrum Master to clear.
Sprint Review	End of the sprint	**4 hours**	Scrum Master, Dev Team, Product Owner, selected Stakeholders	Demo the completed increment; Product Owner accepts/rejects; give feedback & update Product Backlog.
Sprint Retrospective	After Sprint Review / Before next Sprint Planning	**3 hours**	Scrum Master, Dev Team	Reflect on the process (what went well, what to improve), refine definition of "done."

Case Study Summary: SafeHome – Considering Agile Software Development

- **The Scene:** Team members Jamie and Vinod pitch **Scrum** to their manager, Doug Miller, for the *SafeHome* project.
- **What They Like About Scrum ("The Good"):**
    - Jamie likes that Scrum lets the team build software fast using **sprints** and deliver increments once the team decides the product is "done".
    - Doug likes the sprint concept, the focus on **early test case creation**, and having a **product owner** embedded in the team.
- **Stakeholder vs. Product Owner Distinction:** When Jamie worries that the marketing department will sit on the team and demand changes every 5 minutes, Doug clarifies that marketing is a *stakeholder*, whereas a single person (Marg) will act as the **Product Owner** to filter and prioritize those requests.
- **Doug's Concern ("The Bad"):** Doug dislikes how Scrum can downplay upfront **analysis and design**, acting as if "writing code is where the action is". He wants the team to spend less time coding-and-recoding and more time analyzing and designing a solid solution first.
- **The Takeaway:** Vinod suggests combining **"agility with a little discipline"**—adapting Scrum's best features into an incremental approach while keeping proper analysis and design.

SAFEHOME



Considering Agile Software Development

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door, Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure Jamie, what's up?

Jamie: We've been thinking about our process discussion yesterday . . . you know, what process we're going to choose for this new *SafeHome* project.

Doug: And?

Vinod: I was talking to a friend at another company, and he was telling me about Scrum. It's an agile process model . . . heard of it?

Doug: Yeah, some good, some bad.

Jamie: Well, it sounds pretty good to us. Lets you develop software really fast, uses something called sprints to deliver software increments when the team decides the product is done . . . it's pretty cool, I think.

Doug: It does have a lot of really good ideas. I like the sprint concept, the emphasis on early test case creation, and the idea that the process owner should be part of the team.

Jamie: Huh? You mean that marketing will work on the project team with us?

Doug (nodding): They're stakeholders but not really the product owner. That would be Marg.

Jamie: Good. She will filter the changes marketing will want to send every 5 minutes.

Vinod: Even so, my friend said that there are ways to "embrace" changes during an agile project.

Doug: So you guys think we should use Scrum?

Jamie: It's definitely worth considering.

Doug: I agree. And even if we choose an incremental model as our approach, there's no reason why we can't incorporate much of what Scrum has to offer.

Vinod: Doug, before you said "some good, some bad." What was the bad?

Doug: The thing I don't like is the way Scrum downplays analysis and design . . . sort of says that writing code is where the action is . . .

(The team members look at one another and smile.)

Doug: So you agree with the Scrum approach?

Jamie (speaking for both): It can be adapted to fit our needs. Besides, writing code is what we do, Boss!

Doug (laughing): True, but I'd like to see you spend a little less time coding and then recoding and a little more time analyzing what needs to be done and designing a solution that works.

Vinod: Maybe we can have it both ways, agility with a little discipline.

Doug: I think we can, Vinod. In fact, I'm sure of it.

The Incremental Process Model

- **What It Is:** The Incremental Process Model is an approach to software development where the software is broken down, designed, built, and delivered to the customer in smaller, manageable pieces called **software increments** rather than all at once.
- **How It Combines Process Flows:** It combines elements of the **linear process flow** and the **parallel process flow** (and shares characteristics with **evolutionary process flow**, where each circuit through the five activities releases an increment).
- **How Each Increment Works:**
    - In a project using incremental development, the five generic framework activities—**Communication, Planning, Modeling, Construction, and Deployment**—are applied repeatedly through a series of iterations.
    - Each pass through these five activities produces a deliverable **software increment** that provides stakeholders with a working subset of the overall software features and functionality.
    - As each new increment is built and delivered, the software becomes progressively more complete until the full product is finished.

When to Use the Incremental Model

The Incremental Model works best in the following situations:

- **Clear Core Requirements, Evolving Details:** When the initial software requirements are reasonably well defined, but the overall scope of the project prevents a purely linear (Waterfall) process.
- **Need to Reach the Market Fast:** When there is strong business pressure to get a working product onto the market quickly to provide core functionality to users, and then add more features in later versions (increments).
- **Limited Initial Staffing or Deadlines:** When a complete system cannot be delivered by the business deadline due to a shortage of personnel—early increments can be built by a smaller team, and more staff can be added for later increments if needed.
- **Managing Technical Risk:** When early increments can be planned to avoid unproven or risky hardware/technology that is still under development, deferring those features to later increments.

How the Incremental Model Works Step-by-Step

In the Incremental Model, the team applies a staggered sequence of linear workflows over time. Each linear sequence produces a deliverable increment of the software:

Stage	What Happens	Deliverable Outcome
Increment 1 (The Core Product)	The first increment goes through **Communication → Planning → Modeling → Construction → Deployment**. It addresses the basic, most essential requirements while leaving supplementary features (many known, some still undetailed) for later.	**The Core Product:** A working version containing only the fundamental features needed by the customer.
Customer Evaluation & Re-Planning	The customer uses and evaluates the Core Product in real operating conditions. Based on their feedback,	Refined requirements, modification requests, and an updated plan for Increment 2.

	the team develops or updates the plan for the next increment.	
Increment 2	Goes through the five framework activities again to modify the core product (if needed) and add the next priority set of features and functionality.	A more complete working version of the software (Increment 1 + Increment 2 features).
Increment n (Subsequent Increments)	The process of delivering an increment, learning from customer feedback, re-planning, and delivering the next increment repeats until the complete product is finished.	The final, fully featured software system.

(Note: Multiple increments can also be worked on using a **parallel process flow**—for example, while Increment 1 is in the **Construction** and **Deployment** phases, the team can simultaneously begin **Modeling** for Increment 2.)

Real-World Examples of Incremental Development

Example 1: Fitness App (From Your Textbook)

- **1st Increment:** Delivers the basic user interface (UI) screens needed to sync a smartphone with the fitness device and display current data.
- **2nd Increment:** Adds the ability to set fitness goals and store device data on the cloud, while modifying and creating UI screens based on customer feedback from the first increment.
- **3rd Increment:** Adds social media integration so users can set fitness goals and share their progress with friends.

Example 2: Word-Processing Software (Classic Software Engineering Example)

- **1st Increment (Core Product):** Delivers basic file management, text editing, and document production functions.
- **2nd Increment:** Adds more sophisticated editing and document formatting capabilities.
- **3rd Increment:** Adds automated spelling and grammar checking tools.
- **4th Increment:** Adds advanced page layout and desktop publishing capabilities.

Strengths (Pros)

1. **Early Working Software:** Unlike the Waterfall model, customers do not have to wait until the very end of the project to see and use a working program; the first increment gets something onto the market fast.
2. **Built-In Customer Feedback:** After each increment is deployed, the team learns from real customer feedback and re-plans before delivering the next increment.
3. **Easier Staffing & Resource Management:** Projects can start with fewer developers focusing on the core product, making it ideal when full staffing is unavailable at the start.
4. **Lower Risk of Total Project Failure:** Because the highest-priority features (the core product) are built first and tested repeatedly across increments, critical problems are discovered early.

5. **Compatability with Agile Practices:** As noted in the *SafeHome* case study, even if a team chooses an incremental model as its primary structure, it can easily incorporate agile practices like **Scrum** (where each sprint delivers a code increment).

Weaknesses (Cons)

1. **Requires Strong Upfront Architecture:** For new increments to plug in smoothly, the overall software architecture must be well-planned early on; otherwise, adding later increments may require rewriting earlier code.
2. **Management Resistance to Re-Planning:** Traditional senior management often prefers a single fixed plan and schedule upfront and may resist an incremental approach that updates the plan after each increment.
3. **Higher Integration & Testing Overhead:** Because each new increment must be integrated with all previously completed increments and regression-tested to ensure nothing broke, repeated testing takes extra effort.

Case Study Connection: SafeHome on the Incremental Approach

- **Why the Team Likes It:** In *SafeHome*, Jamie and Vinod favor an incremental approach because it allows them to deliver an increment, learn from customer feedback, re-plan, and deliver another increment.
- **Business Advantage:** Vinod highlights that it fits the nature of consumer products because they can release a working product to the market fast and add functionality with each subsequent increment.
- **Blending with Scrum:** Later, Doug points out that choosing an incremental model still allows the team to adopt the best parts of **Scrum**—combining agile sprints and early test creation with disciplined analysis and design.

Quick Comparison: Incremental Model vs. Other Process Models

Feature	Waterfall Model	Incremental Model	Prototyping Model	Spiral Model
How Software is Delivered	All at once at the very end of the project.	In working pieces (**increments**), starting with a core product and adding functionality each time.	As a "quick design" working model used primarily to clarify fuzzy requirements.	In evolutionary passes around a clockwise spiral, driven by risk analysis.
Customer Feedback	Mainly at the very beginning and after final delivery.	Gathered after every delivered increment to re-plan the next increment.	Gathered continuously to tune the prototype until requirements are clear.	Gathered at the end of every loop around the spiral to adjust cost, schedule, and iterations.
Code Quality of Early Releases	N/A (no early release).	Production-quality working code (each increment is a real	Often low quality / uses shortcuts; may need to be	Early loops may be paper specs or prototypes; later loops are production systems.

		part of the final system).	discarded (thrown away) later.	
Best Used When...	All requirements are stable, clear, and known upfront.	Core requirements are clear, but the product needs to get to market fast and grow in stages.	Requirements are unclear ("fuzzy") or user interface needs testing.	The project is a large-scale, high-risk system with evolving requirements.

3.4 Functional Specification with Use Cases

Overview & Purpose

- **Traditional vs. Use Case Approach:** Functional requirements form the heart of a software requirements document. While the traditional method simply lists every individual function the software must perform, **use cases** define functionality by describing how the system *behaves* during interactions with its users.
- **Levels of Application:** Use cases can model high-level business processes across an entire organization, or they can focus specifically on the behavior of a software system being developed.
- **Role in Requirements Engineering:** Beyond specifying system behavior, use cases are highly effective during requirements analysis and help teams uncover (elicit) new requirements.
- **Origins & Common Misconception:** Ivar Jacobson popularized use cases as part of his object-oriented (OO) modeling methodology. Because of this history, many people assume use cases are strictly an object-oriented tool. In reality, they are a universal technique for describing interactions with any kind of system—including non-IT systems.

3.4.1 Basics of Use Cases

Core Terminology (From Table 3.2)

Term	Meaning in Simple Terms	Key Details & Examples
Actor	Any person or external system that interacts with the software being built to achieve a specific goal.	• **Not just humans:** An actor can be another piece of software or hardware. • **Logical roles, not individuals:** An actor represents a *category* of users or systems that behave the same way and share the same goal. • *Example:* In a messaging app, instead of creating one generic "User" actor, you should create separate **"Sender"** and **"Receiver"** actors because they have different goals.

Primary Actor	The main actor for whom a use case is started and whose goal the use case is primarily trying to satisfy.	• **Consumer of the result:** The primary actor is the logical entity that actually uses the outcome of the use case, even if someone else physically operates the system on their behalf. • *Example 1 (Human Agent):* For a *"Get regional sales growth report"* use case, the **Vice President (VP)** is the primary actor because the VP uses the report, even if an assistant actually runs the program. • *Example 2 (Time Trigger):* If that report is generated automatically on a timer, the VP is still the primary actor.
Scenario	A sequence of actions carried out under specific conditions to reach a goal.	• **Written sequentially:** For convenience in text, steps are listed in order, even though in real life some steps might run in parallel or in a different sequence. • **What counts as a step:** Each step is a complete logical action taken by either the actor or the system: 1. *Actor action:* e.g., entering data. 2. *System progress step:* e.g., validating data or displaying results. 3. *Internal system state change:* e.g., logging a transaction or updating a database record.
Main Success Scenario	The step-by-step interaction when everything goes smoothly and every step succeeds without errors.	• Every use case has a main success scenario, and some may have multiple success paths.
Extension Scenario *(Exception Scenario)*	How the system behaves when a step in the main scenario fails or hits an unexpected condition.	• Captures alternative paths when unexpected events prevent the system from completing the goal normally. • **Complete Use Case:** A single use case is the combined collection of its main success scenario(s) plus all of its extension/exception scenarios.

Key Principles of Use Cases

- **Satisfying All Stakeholders (Not Just the Primary Actor):**
    - While the primary actor's goal drives the use case, the system's behavior must also protect the interests of **every other stakeholder** involved.
    - *Example (ATM Cash Withdrawal):* A bank customer is the primary actor whose goal is to get cash. However, the **bank** is also a stakeholder. Therefore, the use case must include steps that satisfy the bank's goals—such as logging every step, checking that the account has enough funds, and enforcing daily withdrawal limits.
- **Hierarchical Organization (Sub-Goals & Supporting Actors):**
    - To fulfill a main goal, a system can break it down into smaller **sub-goals**.

    - Some sub-goals are handled internally by the system, while others are treated as separate, lower-level use cases carried out with **supporting actors** (such as an external system).
    - *Example:* In the ATM withdrawal use case, verifying the user's identity might rely on an external authentication service. That verification process can be written as its own separate use case called by the main use case.
- **Focus on External Behavior (Avoiding Premature Design):**
    - Use cases view a software system as an entity that responds to requests from actors.
    - **Major Advantage:** Because use cases only describe *external* interactions between actors and the system, they naturally stop engineers from jumping into *internal* software design during the requirements phase—a common mistake in many other modeling methods.
- **Role in the SRS & UML Use Case Diagrams:**
    - **Textual Nature & SRS Scope:** Use cases are primarily written as text descriptions and capture the *behavioral (functional)* requirements of a system. Because a complete **Software Requirements Specification (SRS)** also requires non-functional details (such as performance targets and design constraints), use cases make up an important part of the SRS, but not the entire document.
    - **UML Use Case Diagrams:** Visual diagrams in UML can provide a high-level summary of actors and use cases:
        - **Ellipses (Ovals):** Represent individual use cases.
        - **Stick Figures:** Represent primary actors, connected to their use cases with lines.
        - **Arcs/Arrows:** Show dependencies and relationships between different use cases.
        - *Limitation:* Because the actual value of use cases lies in their detailed text stories, diagrams play only a minor supporting role.

3.4.2 Examples: Online Auction System

Imagine building an online auction platform where people buy and sell items, supported by a separate external **financial subsystem** where users hold accounts to make payments.

- **Logical Actors & Stakeholders:** Even if the exact same person buys one day and sells the next, we do not use a generic "User" actor. Instead, we define four distinct actors/stakeholders based on their goals:
    1. **Seller** (wants to list and sell items)
    2. **Buyer** (wants to bid on and purchase items)
    3. **Auction System** (stakeholder and actor that manages auction closing and site records)
    4. **Financial System** (external actor that processes fund holds, debits, and credits)
- **Why Stakeholders Love Use Cases:** Because use cases read like natural, step-by-step stories, both technical analysts and non-technical clients can easily understand them, greatly reducing communication gaps.

Walkthrough of the 3 Main Auction Use Cases (From Figure 3.15)

Use Case 1: Listing an Item for Auction

- **Primary Actor:** Seller

- **Precondition:** The seller is already logged into the system.
- **Main Success Scenario:**
    - The seller submits the item's details (category, description, photo, etc.) to put it up for auction.
    - The system displays historical selling prices of similar items to guide the seller.
    - The seller enters the starting bid price and the closing date for the auction.
    - The system accepts the item and publishes the listing live on the site.
- **Exception Scenario:**
    - **Step 2a (No past items exist in this category):** The system informs the seller that no historical pricing data is available.

Use Case 2: Placing a Bid

- **Primary Actor:** Buyer
- **Precondition:** The buyer is already logged into the system.
- **Main Success Scenario:**
    - The buyer *searches* or *browses* (sub-use cases) and picks an item.
    - The system displays the seller's rating, starting price, current bids, and highest bid, then prompts the buyer to enter a bid.
    - The buyer enters their bid amount, maximum bid limit, and bidding increment.
    - The system accepts the bid and places a hold (blocks) the required funds in the bidder's financial account.
    - The system updates the bid prices of competing bidders as needed and updates the item's internal records.
- **Exception Scenarios:**
    - **Step 3a (Bid is lower than the current highest bid):** The system notifies the bidder and asks them to submit a higher bid.
    - **Step 4a (Insufficient funds in the bidder's account):** The system cancels the bid and prompts the user to add more funds to their account.

Use Case 3: Completing the Auction of an Item

- **Primary Actor:** Auction System
- **Precondition:** The closing date for bidding has been reached.
- **Main Success Scenario:**
    1. The system identifies the highest bidder, emails the final winning price to both the winning bidder and the seller, and emails the remaining bidders.
    2. The system *debits the winning bidder's account and credits the seller's account* (sub-use case).
    3. The system *transfers the commission fee from the seller's account to the auction organization's account* (sub-use case).
    4. The system removes the auctioned item from the active website and updates internal records.

- **Exception Scenarios:** None listed.

Important Lessons from the Auction Use Cases

1. **Numbering and Goal-Based Naming:**
    - Use cases are numbered (e.g., UC1, UC2, UC3) so they are easy to reference across documents.
    - The title of each use case directly states the primary actor's goal (e.g., *"Make a bid"*), eliminating the need for a separate "Goal:" line.
2. **Systems Can Be Primary Actors:**
    - While UC1 and UC2 are triggered by humans (Seller and Buyer), UC3 is triggered by the **Auction System** itself once the deadline arrives. Another software application requesting a service can also act as a primary actor.
3. **Role of Preconditions:**
    - A **precondition** states what the system guarantees is true *before* the use case is allowed to begin.
    - Examples include: *"User is logged in"* (UC1, UC2), *"Bidding deadline is reached"* (UC3), *"Input data exists," "Item exists"* (for a delete operation), or *"Tracking number is valid"* (for a package tracking operation).
4. **Steps That Protect Other Stakeholders:**
    - A use case often includes actions that the primary actor doesn't personally care about, but that are required by other stakeholders.
    - For instance, in UC2 (Step 5), the current buyer doesn't care about updating other bidders' prices or system records—but the **Auction System** and **other bidders** are stakeholders whose needs must be met. The same applies to Step 4 of UC1.
5. **Handling Unlisted Exceptions:**
    - Figure 3.15 only lists the most obvious failures, but real-world projects include general failure conditions that can happen at any step—such as *"User abandons the transaction before finishing,"* along with cleanup rules (e.g., wiping temporary records).
6. **Underlined Steps, Hierarchy, and SRS Scope:**
    - In UC2 and UC3, actions like *"searches," "browses," "blocks funds," "debit bidder's account and credit seller's,"* and *"transfer commission"* are separate lower-level use cases:
        - **Inside System Scope (Must be in the SRS):** Features like *"search"* and *"browse"* belong to the auction software itself. They must be written out as full, standalone use cases elsewhere in the SRS, usually keeping the same primary actor (e.g., the **Buyer** is the primary actor for *"Find an item"*).
        - **Outside System Scope (Excluded from the SRS):** Financial actions (blocking funds, debiting/crediting accounts) belong to the external financial system. They are not written as use cases in the auction SRS, though engineers must obtain interface specifications for how to connect to them.
    - **Step-by-Step Refinement:** Writing high-level scenarios first and breaking underlined steps into smaller use cases lets analysts organize requirements in a neat hierarchy and naturally discover new use cases and actors along the way.

3.1 Software Requirements

Official IEEE Definition of a Requirement [91]

IEEE defines a software requirement in two ways:

1. **User-Focused:** A condition or capability needed by a user to solve a problem or achieve a specific goal.
2. **System/Contract-Focused:** A condition or capability that a system must possess or meet to satisfy a contract, standard, specification, or other formally imposed document.

Core Goal and Challenge of the Requirements Phase

- **Specifying a Non-Existent System:** Software requirements deal with the capabilities of a *proposed* system that has not been built yet. Describing something that does not yet exist is what makes gathering requirements so difficult.
- **The Goal (The SRS):** The ultimate goal of the requirements activity is to produce the **Software Requirements Specification (SRS)**—a document that describes **what** the proposed software should do *without* describing **how** the software will do it.
- **The Fundamental Limitation:** User needs constantly change over time because the business or operational environment in which the system functions keeps changing. Even though some requirement changes are unavoidable, producing a thorough, high-quality, and relatively stable SRS upfront is essential for project success.

3.1.1 Need for SRS

Bridging the Communication Gap Between the 3 Major Parties

Every new software system involves three main groups of people:

1. **The Client (Customer):** The party whose needs initiate the project. *Problem:* Clients usually do not understand software or the software development process.
2. **The Developer (Supplier):** The team that builds the software. *Problem:* Developers often do not understand the client's specific business problem or application domain.
3. **The End Users:** The people who will actually operate and use the finished system.

Because the client does not understand software and the developer does not understand the client's domain, a **communication gap** exists. The primary purpose of the SRS is to bridge this gap by accurately translating client and user needs into clear specifications for the developer.

The 4 Major Advantages of a High-Quality SRS

1. An SRS establishes the basis for agreement between the client and the supplier on what the software product will do.

- **Legal Contract:** This agreement is frequently turned into a formal legal contract between the client (customer) and the developer (supplier).
- **Clear Expectations:** Through the SRS, the client clearly states what they expect, and the developer clearly understands what capabilities to build.
- **Avoiding the "Bug vs. Feature" Dispute:** Without an SRS agreement, the client is almost guaranteed to be unhappy at the end of the project, leading to unhappy developers and classic disputes where the client says, *"Hey! There is a bug,"* and the developer replies, *"No, it is a software feature." (Note: Even with a formal agreement, keeping clients completely satisfied remains a challenge!)*

2. An SRS provides a reference for validation of the final product.

- **Proving Completion:** Without a proper SRS, the client has no objective way to check if the delivered software is actually what they ordered, and the developer has no way to prove to the client that all requirements have been fulfilled.

3. A high-quality SRS is a prerequisite to high-quality software.

- **Impact on Project Drivers (Cost, Schedule, and Quality):** Many software errors originate in the requirements phase. If the SRS specifies the *wrong* system (one that doesn't meet the client's true goals), even a 100% bug-free coding implementation of that SRS will produce a system that fails the client.
- **Real-World Research & Error Statistics:**
    - **Boehm's Findings [45]:** In several projects, **54%** of all detected errors were found *after* coding and unit testing were finished—and **45%** of those late errors actually originated during the requirements and early design stages. This means roughly **25% of all software errors** happen during requirements and early design (54% × 45% ≈ 25%).
    - **The A-7 Project [8]:** About **80 errors** were discovered in the requirements document over a few months, each resulting in formal change requests.
    - **Approved SRS Study [45]:** More than **500 errors** were uncovered in an SRS that had *already been approved*.
    - **Structured Analysis Study [47]:** More than **250 errors** were found in a previously reviewed SRS simply by rewriting the requirements in a structured format and analyzing the document with automated tools.

4. A high-quality SRS reduces the development cost (and helps stay on schedule).

- **Exponential Rise in Error-Fixing Costs:** The cost of fixing an error grows almost exponentially the later it is found in the project lifecycle. Fixing a requirement error *after* the system is deployed can cost up to **100 times more** than fixing it during the requirements phase itself.
- **Average Cost to Fix a Requirement Error by Phase:**
    - Research [8] showed that fixing a requirement error during the requirements phase took an average of **2.4 person-hours** (excluding outliers) or **5.0 person-hours** (including outliers). Using a baseline of **2 person-hours** in the requirements phase alongside multicompany study data [17, 20], Table 3.1 shows how fixing costs escalate:

Phase	Cost (person-hours)
Requirements	2
Design	5
Coding	15
Acceptance test	50
Operation and maint.	150

Table 3.1: Cost of fixing requirement errors.

3 Economic Proof Examples: How Investing in the SRS Saves Money

- **Example 1: Saving Person-Hours by Catching 50 Errors Early**

- **The Investment:** Suppose a team invests an extra **100 person-hours** during the requirements phase. At 2 person-hours per error, they find and remove **50 requirement errors**. *(Note: This linear rate applies to errors caught early; as fewer errors remain, each new error takes more effort to find.)*
- **What Happens if Left Unfixed (A-7 Project Distribution [8]):** Uncaught requirement errors are typically discovered in later phases at the following rates:
  - **65%** caught during **Design** (65% of 50 = 32.5 errors)
  - **2%** caught during **Coding** (2% of 50 = 1 error)
  - **30%** caught during **Testing / Acceptance Test** (30% of 50 = 15 errors)
  - **3%** caught during **Operation and Maintenance** (3% of 50 = 1.5 errors)
- **Total Cost to Fix Later:** Using the phase costs from Table 3.1:

$$\text{Cost} = (32.5 \times 5) + (1 \times 15) + (15 \times 50) + (1.5 \times 150) = 162.5 + 15 + 750 + 225 = 1152.5 \text{ person-hours}$$

- **Net Savings:** By spending 100 extra person-hours upfront, the project avoids 1152 person-hours of later fixes—a **net savings of 1052 person-hours**!

- **Example 2: Expected Cost of a Single Unremoved Requirement Error**
  - Suppose an unremoved requirement error has the following probabilities of being caught in later phases:
    - **0.4 (40%)** in Design (cost = 5 hrs)
    - **0.1 (10%)** in Coding and Unit Testing (cost = 15 hrs)
    - **0.4 (40%)** in Acceptance Testing (cost = 50 hrs)
    - **0.1 (10%)** in Operation and Maintenance (cost = 150 hrs)
  - **Expected Cost Formula:**

$$\text{Expected Cost} = (0.4 \times 5) + (0.1 \times 15) + (0.4 \times 50) + (0.1 \times 150) = 2 + 1.5 + 20 + 15 = 38.5 \text{ person-hours}$$

  - **Economic Rule:** As long as finding and fixing a requirement error during the requirements phase takes **less than 38.5 person-hours**, it is financially worthwhile to spend the extra effort during requirements.

- **Example 3: Cutting Rework Costs Caused by Requirement Changes**
  - **Why Rework Happens:** While some requirement changes come from genuinely evolving business needs, many happen simply because the initial requirements were poorly analyzed and validated. A high-quality SRS drastically cuts down this avoidable "requirement change traffic."
  - **Industry Benchmarks:**
    - **Rework Cost [22]: 20% to 40%** of total project development effort is spent on **rework**, mostly driven by requirement changes.
    - **COCOMO Model [20]:** The requirements phase typically takes about **6%** of the total project cost.

- **Project Calculation (50 Person-Month Project):**
  - **Total Project Effort:** 50 person-months.
  - **Baseline Requirements Effort (6%):** 3 person-months.
  - **Baseline Rework Effort (20% to 40%):** 10 to 20 person-months.
  - **The Investment & Payoff:** If the team spends **33% more effort** in the requirements phase (adding **1 extra person-month**) and reduces requirement change requests by **33%**, total rework effort drops from 10–20 person-months down to **6 to 12 person-months**.
  - **Net Savings:** Even after accounting for the extra effort in requirements, the project saves **5 to 11 person-months** overall—a **10% to 22% reduction in total project cost**!

3.1.2 Requirement Process

The 3 Basic Tasks of the Requirement Process (Figure 3.1) The requirement process is the sequence of steps performed to transform initial **Client/User Needs** into a **Validated SRS**. It consists of three main activities:

Activity	**Main Focus & Purpose**	**Key Tasks & Details**
1. Problem (or Requirement) Analysis	Thoroughly understanding **what** the software needs to provide.	• Starts with a high-level "problem statement" based on client/user needs. • Models the problem domain and environment to understand system behavior, constraints, inputs, and outputs. • Uses **"divide-and-conquer" (decomposition)** to break complex problems into smaller, understandable parts. • Builds abstraction structures like **Data Flow Diagrams (DFDs)** and **Object Diagrams** to manage large volumes of information.
2. Requirement Specification *(labeled "Product Description" in Fig. 3.1)*	Clearly writing and organizing the requirements in a formal document.	• Translates the understanding gained during analysis into a clear specification document. • Addresses representation formats, specification languages, and tools. • Organizes massive amounts of analysis information and eliminates redundancies.
3. Requirements Validation *(or Verification)*	Ensuring the SRS is complete, accurate, and high quality.	• Verifies that everything written in the SRS represents all true software requirements. • Ends with the final deliverable: the **Validated SRS**.

How the Process Flows in Reality: Overlap and Feedback Loops (Figure 3.1)

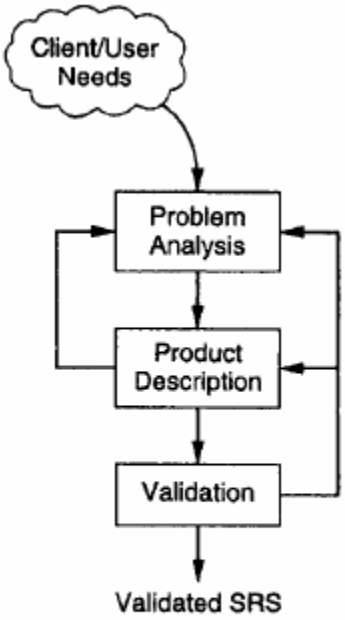


Figure 3.1: The requirement process.

- While the three tasks look linear on paper (Analysis → Specification → Validation), only trivial systems move in a straight line.
- In real-world systems, different parts of the system are analyzed and specified at different times, with constant **feedback loops**:
    1. **From Specification (Product Description) back to Problem Analysis:** Writing down the formal specifications frequently exposes gaps or shortcomings in the team's understanding of the problem, forcing them to return to Problem Analysis.
    2. **From Validation back to Specification (Product Description):** If validation uncovers errors or ambiguities in how the document was written, the team loops back to fix the specification.
    3. **From Validation back to Problem Analysis:** If validation reveals a fundamental misunderstanding of the client's problem, the team must loop all the way back to Problem Analysis.

Key Conceptual Distinctions in the Requirements Process

1. Why Moving from Analysis to Specification is Difficult

- At first glance, turning completed analysis models into an SRS document seems easy—but in practice, it is quite hard because the two steps have different goals:
    - **Problem Analysis** studies the **internal structure** and components of the problem domain (along with inputs and outputs) so the analyst can understand how the business/problem works.
    - **Requirement Specification** focuses *exclusively* on the **external behavior** of the proposed software ("what" it does), ignoring internal problem structures.
- **Indirect Value of Analysis Models:** Because the SRS only describes external behavior, many diagrams and structures built during analysis are **not copied directly into the SRS**. Instead, their

value is *indirect*: they build the deep understanding required before an analyst can accurately write the specification.

2. Problem Analysis vs. Software Design (Davis [45])

- **How They Are Similar:** Both activities struggle with the same core challenge—**managing complexity**—and both use the exact same techniques to solve it: **decomposition** (divide-and-conquer) and building **abstraction structures** (such as Data Flow Diagrams and Object Diagrams).
- **How They Are Completely Different:**
    - **Problem Analysis** operates in the **Problem Domain** → its goal is **understanding the problem**.
    - **Software Design** operates in the **Solution Domain** → its goal is **optimizing the design**.
- **Avoiding a Common Mistake:** Because both phases use tools like DFDs and Object Diagrams, people often mistakenly assume that the diagrams created during analysis should automatically become the software design. Because their objectives differ, they yield different structures. Analysis structures should *only* be carried into design if they happen to align with design optimization goals.

3. Choosing the Right Level of Detail: Abstract vs. Detailed Requirements

The level of detail uncovered during the requirements process depends on what the document will be used for:

Feature	**Abstract Requirements (High-Level)**	**Detailed Requirements (Lower-Level / SRS)**
What It Describes	Broad, overall system needs stated at a high level of abstraction.	Every external behavior and external interface specified in full detail.
Primary Purpose	• Feasibility analysis • Cost estimation & cost-benefit analysis • Competitive bidding (inviting tenders from vendors) • Initial starting point for analyzing complex systems	• Actual **software development** by the contracted engineering team (this is the level required for a proper **SRS**).
Example: Car Manufacturer Inventory Control System	• Total number of parts to track • Level of concurrency required • Online vs. batch processing • General types of reports needed (e.g., on-demand item status, low-stock purchase orders, consumption patterns)	• Exact layout/format of every report • Every allowed query and its exact structure • Total number of terminals to support • Structure of the major databases

- **Perspective Shift ("What" vs. "How" Across Levels):** Viewed from a high level of abstraction, detailed requirements can look like they are specifying *"how"* the abstract requirements are met (for

example, specifying the exact layout of a report defines *how* the broad goal of "providing inventory information" is satisfied). However, for the software developer, those details are still strictly defining **external software behavior**—and are essential for building the system properly.

3.3.1 Characteristics of an SRS

To fulfill its purpose effectively, a good Software Requirements Specification (SRS) must possess **8 key characteristics**:

Characteristic	Definition in Simple Terms	Key Details, Challenges & Examples
1. Correct	Every requirement listed in the SRS accurately represents something that is genuinely needed in the final software.	• **Easier to verify than completeness:** You only need to examine each written requirement to confirm it matches actual user needs.
2. Complete	Everything the software must do—including how it responds to **all classes of input data**—is included in the SRS.	• **Correctness vs. Completeness:** Correctness ensures that *what is written* is accurate; completeness ensures that *everything needed* is written. • **The Most Important & Hardest Property:** Completeness is the hardest quality to achieve because you have to detect the *absence* of missing requirements rather than checking existing ones. • **Cost of Incompleteness:** Missing client requirements are a major source of client–supplier disputes and force expensive additions and changes later in development.
3. Unambiguous	Every stated requirement has **one and only one** possible interpretation.	• **The Natural Language Problem:** Requirements are usually written in everyday human (natural) language, which is naturally prone to multiple interpretations. • **Using Formal Languages:** Formal specification languages eliminate ambiguity, but they have three big drawbacks: **(1)** they take massive effort to write, **(2)** they are expensive, and **(3)** they are very hard for clients and users to read and understand.
4. Verifiable	There is a cost-effective process to check whether the finished software actually meets every stated requirement.	• **Avoid Subjectivity:** Subjective statements cannot be objectively tested, so requirements must be concrete. • **Requires Unambiguity & Understandability:** A requirement cannot be verified if it is ambiguous. Furthermore, because requirements are often verified through human **reviews**, the SRS must be

		easy to understand by developers, clients, and users so they can all agree on it.
5. Consistent	No requirement contradicts or conflicts with any other requirement in the document.	• **Terminology Conflicts:** Using different words in different places to describe the exact same object. • **Logical or Temporal (Timing) Conflicts:** Two requirements that cannot possibly be satisfied together. *Example:* One requirement states that event e must happen before event f, while another states (directly or indirectly) that event f must happen before event e.
6. Ranked for Importance and/or Stability	Every requirement clearly states both how important it is and how likely it is to change in the future.	• **Levels of Importance:** Requirements are not equal—some are **critical**, some are **important (but not critical)**, and others are merely **desirable (not very important)**. • **Stability:** Measures the likelihood that a requirement will change in the future (expressed as expected change volume). **"Core" requirements** remain stable over time, while others are time-dependent and likely to change.
7. Modifiable	The structure and writing style of the SRS allow changes to be made easily without breaking completeness or consistency.	• **Why It Matters:** Writing an SRS is iterative, and client needs inevitably evolve. • **The Enemy of Modifiability (Redundancy):** Repeating the same requirement in multiple places makes an SRS hard to modify. If a requirement is written in two places and later updated in only one place, the SRS immediately becomes **inconsistent**.
8. Traceable	The origin of every requirement is clear, and every requirement can be easily referenced in later stages of development.	• **Backward Traceability:** Tracing design and code elements *backward* to the specific requirements they fulfill (and knowing where each requirement came from). • **Forward Traceability:** Tracing each requirement in the SRS *forward* to the specific design and code elements that implement it. • **Benefit:** Directly supports verification and validation.

3.3.2 Components of an SRS

Because completeness is so difficult to achieve and verify, following a standard checklist of required components helps ensure nothing is left out. Conceptually, every SRS must address **4 fundamental areas**:

1. **Functionality** *(Note: In traditional analysis, functionality has its own dedicated section; in object-oriented or use-case approaches, it may be specified indirectly via object services or use cases.)*
2. **Performance**
3. **Design constraints imposed on an implementation**
4. **External interfaces**

1. Functional Requirements

Functional requirements define the relationship between the system's **inputs** and **outputs**—specifying exactly what outputs must be produced from given inputs.

- **Detailed Input Specifications:** For every functional requirement, you must specify:
    - All data inputs and where they come from (their source).
    - The units of measurement.
    - The exact range of valid inputs.
- **Operations & Transformations:** You must specify all operations performed on input data to generate the output, including:
    - Validity checks on both input and output data.
    - System parameters affected by the operation.
    - Exact mathematical formulas, equations, or logical operations required to transform inputs into outputs.
    - *Important Boundary (SRS vs. Design):* Only specify formulas/rules that are part of the business problem itself. **Do not** specify internal programming algorithms needed to implement the system—leave those choices to the software designer.
- **System Behavior in Abnormal Situations (Critical for Robustness):**
    - Many systems fail to be robust because writers forget to specify special error conditions. An SRS must state how the system behaves for **all foreseen inputs and all foreseen system states**, including:
        1. **Invalid inputs** (which can happen in many ways).
        2. **Invalid outputs** or **errors during computation**.
        3. **Valid inputs when normal operation is impossible:** For example, in an airline reservation system, if a valid passenger tries to book a seat on a flight that is already 100% full, the SRS must specify exactly how the system responds.

2. Performance Requirements

Performance requirements define the speed and capacity constraints placed on the software. They fall into two categories:

Type	What It Constrains	Examples
Static Performance Requirements *(Capacity Requirements)*	Constraints that **do not** depend on live execution behavior; they define the size and scale the system must support.	• Number of terminals to support. • Number of simultaneous users to support. • Number of files the system must process and their sizes.
Dynamic Performance Requirements	Constraints on the **live execution behavior** of the running system under both **normal** and **peak** workloads.	• **Response Time:** The expected time to finish a single operation under specific conditions. • **Throughput:** The expected number of operations/transactions processed per unit of time (e.g., per second).

- **Rule of Measurability:** Performance requirements must always be written in concrete, measurable, and verifiable terms (including acceptable ranges):
    - *Bad (Imprecise/Unverifiable):* "Response time should be good" or "The system must process all transactions quickly."
    - *Good (Measurable/Verifiable):* "The response time of command x should be less than 1 second 90% of the time" or "A transaction should be processed in less than 1 second 98% of the time."

3. Design Constraints

Design constraints are real-world factors in the client's environment that limit the software designer's technical choices:

- **Standards Compliance:** Rules and standards the software is required to follow, such as standard report layouts, accounting procedures, or **audit tracing** (recording specific types of changes or operations in an audit file).
- **Hardware Limitations:** If the software must run on existing or pre-selected hardware, the SRS must list constraints such as machine types, available operating systems, supported programming languages, and limits on primary (RAM) and secondary (disk) storage.
- **Reliability and Fault Tolerance:** Essential for critical applications (though it makes software more complex and expensive). Specifies how the software must behave when specific faults happen, along with **recovery requirements** (what actions the system must take after a failure to protect data and system properties).
- **Security:** Crucial for defense systems, databases, and modern everyday software. Includes:
    - Restricting access to specific commands and data.
    - Providing different access levels for different user roles.
    - Requiring passwords, encryption (cryptography), and system activity logs.
    - Assessing security threats, using secure coding practices, and running tools to catch vulnerabilities like **buffer overflows**.

4. External Interface Requirements

Specifies every point where the software interacts with **people**, **hardware**, and **other software**:

- **User Interfaces (UI):**
    - Must describe the characteristics of every user interface.
    - Should include a **preliminary user manual** covering all user commands, screen layouts/formats, how the system looks to the user, and feedback/error messages.
    - *Must be verifiable:* Avoid vague claims like *"The system should be user-friendly."* Instead, write testable rules like *"Commands should be no longer than six characters"* or *"Command names should reflect the function they perform."*
- **Hardware Interfaces:**
    - Defines the logical interface between the software and hardware components.
    - If using existing/predetermined hardware, specify all hardware traits, memory limits, and current usage/load characteristics.
- **Software Interfaces:**
    - Defines how the system connects with other software (such as the **operating system** and external applications) that either uses this system or is used by it.
    - Must specify the exact **message content and data format** for every software interface.

3.2.2 Data Flow Modeling

Overview of Structured Analysis

- **What It Is:** Data-flow based modeling—commonly called the **structured analysis technique**—models a problem using **function-based decomposition** in a **top-down refinement approach**.
- **Focus:** It focuses on the functions (processes) performed in the problem domain and the data consumed and produced by those functions.
- **2 Core Tools:** Relies heavily on **Data Flow Diagrams (DFDs)** and an accompanying **Data Dictionary**.

Data Flow Diagrams (DFDs)

A **Data Flow Diagram (DFD)** (or *data flow graph*) visualizes how data moves through a system. It views any system as a series of transformations that convert input data into desired output data.

Standard DFD Symbols & Conventions (Illustrated in Figure 3.2)

- **Process (Bubble):** Drawn as a **named circle**. Represents an agent/function that transforms data from one state to another.
- **Data Flow:** Drawn as a **named arrow** entering or leaving a bubble to show the path of data movement.
- **Source or Sink (External Entity):** Drawn as a **rectangle**. Represents an external originator of data (**source**) or final consumer of data (**sink**) outside the main system.
- **External File (Data Store):** Drawn as a **labeled straight horizontal line** (e.g., *Employee Record*, *Company Records*, *Tax Rates*).
- *** (AND Relationship):** Placed between multiple data flow arrows entering or leaving a process to show that **all** of those data flows are required together.

- **+ (OR Relationship):** Placed between data flow arrows to show that **either one or the other** data flow is used.

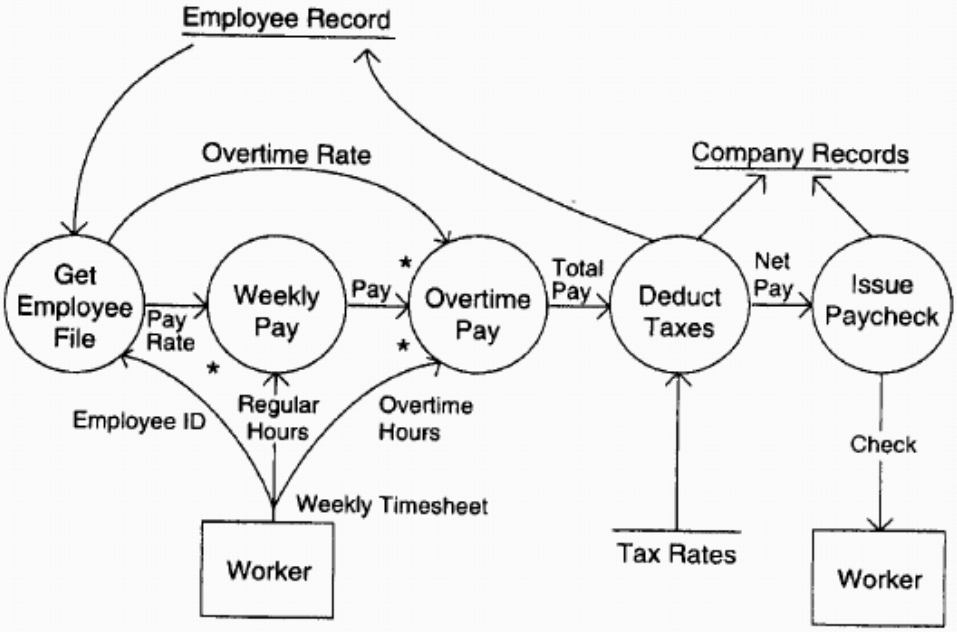


Figure 3.2: DFD of a system that pays workers.

Walkthrough Example: Worker Payment System DFD (Figure 3.2)

- **Source & Input:** The rectangle Worker acts as the source, submitting a Weekly Timesheet (which contains Employee ID, Regular Hours, and Overtime Hours).
- **Step 1 (Get Employee File):** Takes Employee ID and reads the Employee Record file to output the worker's Pay Rate and Overtime Rate.
- **Step 2 (Weekly Pay):** Uses Pay Rate **AND (*)** Regular Hours to calculate regular Pay.
- **Step 3 (Overtime Pay):** Combines Overtime Rate **AND (*)** regular Pay **AND (*)** Overtime Hours to calculate Total Pay.
- **Step 4 (Deduct Taxes):** Uses Total Pay and information from the Tax Rates file to calculate taxes. It records the deducted tax in both the Employee Record and Company Records files, and outputs Net Pay.
- **Step 5 (Issue Paycheck):** Takes Net Pay, updates Company Records with the amount paid, and outputs the final Check to the sink Worker.

3 Crucial Rules to Remember About DFDs

1. **Manual or Automated:** A DFD is an abstract description of data transformation—it applies equally whether the system is 100% manual (using hand calculators and paper ledgers) or fully computerized.
2. **Omit Minor Error Paths Initially:** High-level DFDs deliberately leave out minor paths (like what happens if a timesheet has a typo) so the analyst doesn't get bogged down in small details too early.
3. **A DFD is NOT a Flowchart:**
    - A **DFD** shows the **flow of data** (what major transformations happen to data).

- A **Flowchart** shows the **flow of control** (procedural step-by-step execution).
- **Strict Rule:** Never include procedural thinking, loops, or decision branches (if/else) in a DFD!

5 Practical Guidelines for Drawing a DFD

1. **Work Consistently in One Direction:** Start by identifying major inputs and outputs (ignoring minor ones like error messages). Trace transformations from inputs to outputs (or backward from outputs to inputs—if you get stuck, reverse direction). Begin with a high-level graph of a few major bubbles, then refine them.
2. **Ban Control Logic:** If you catch yourself thinking about loops or if/then decisions, stop immediately and start over.
3. **Label Everything Carefully:** Give every arrow a clear data label and carefully verify the inputs and outputs of each bubble.
4. **Use Logical Operators:** Use * (AND) and + (OR) symbols and include sufficient detail.
5. **Draft Alternatives:** Sketch multiple versions of the DFD before choosing the best one.

Leveled DFD Sets (Handling Large, Complex Systems)

When a system is too big to fit on a single diagram, analysts organize DFDs into a hierarchy called a **leveled DFD set**:

- **Top-Down Refinement:** You start with a single, high-level abstract DFD showing only the main inputs, outputs, and major processes. Next, each process bubble is expanded ("zoomed in") into its own separate, more detailed DFD.
- **When to Stop Refining:** Refinement stops when every bubble is **"atomic"**—meaning its function is simple enough to be easily understood and specified on its own.
- **The Balancing / Consistency Rule:** Whenever you expand a parent bubble into a child DFD, the **net inputs and net outputs** of the child DFD must match the exact inputs and outputs connected to that bubble in the parent DFD.
- **Data Decomposition:** As processes are broken down into smaller sub-processes across levels, broad data packets are simultaneously broken down into their smaller individual data fields.

The Data Dictionary (Figure 3.3)

While a DFD labels each arrow with a meaningful name, it does not show the exact internal data structure of those arrows or files. The **Data Dictionary** is a companion catalog that defines the exact structure and elements of every data flow and file in the DFD.

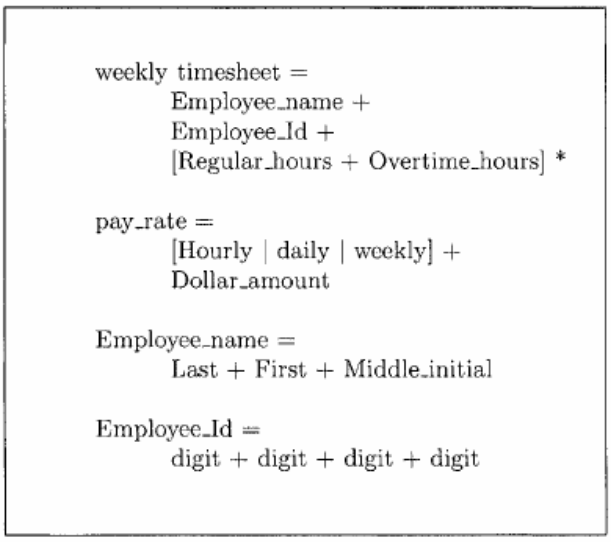


Figure 3.3: Data dictionary.

Data Dictionary Notations (Regular-Expression Style)

- **+ (Sequence / Composition):** Means one data item combined with / followed by another.
- **| (Selection / OR):** Means choosing one option **OR** another.
- *** (Repetition / Iteration):** Means one or more occurrences of the bracketed item.

Example Data Dictionary for the Worker Payment System (Figure 3.3)

- weekly timesheet = Employee_name + Employee_Id + [Regular_hours + Overtime_hours] * *(Composed of employee name, ID, and multiple daily pairs of regular and overtime hours)*.
- pay_rate = [Hourly | daily | weekly] + Dollar_amount *(Pay type selection followed by dollar amount)*.
- Employee_name = Last + First + Middle_initial.
- Employee_Id = digit + digit + digit + digit *(A 4-digit number)*.

How to Verify a DFD and Data Dictionary (6 Common Errors)

Because there is no formal mathematical blueprint of the real world to test a DFD against, analysts verify DFDs through **client walkthroughs** and by checking for **6 common errors**:

Common DFD Error	What Goes Wrong	How to Test & Catch It
1. Unlabeled Data Flows *(Most Common Error)*	An arrow has no name, usually because the analyst doesn't fully understand what data is moving there.	Verify that every single arrow has a label and a precise, complete definition in the **Data Dictionary**.
2. Missing Data Flows	A process bubble does not receive all the input data it needs to create its outputs.	Ask for each bubble: *"Can this process build all of its shown outputs using ONLY the given inputs?"*

3. Extraneous (Redundant) Data Flows	A process receives input data arrows that it never actually uses.	Ask for each bubble: *"Are ALL of the input data flows actually needed to compute the outputs?"*
4. Consistency Lost During Refinement	A new data flow is added to a child DFD without adding it to the parent DFD (or vice versa).	Cross-check parent and child DFDs whenever modifications are made to ensure net inputs/outputs match.
5. Missing Processes	Important activities from the real-world environment are accidentally left out of the diagram.	Carefully scrutinize the DFD against the physical environment with the client.
6. Contains Control Information	Arrows are used to pass control triggers ("start now", "loop") instead of actual data.	Check the Data Dictionary: any data flow that has **no internal structure or composition** is likely a disguised control signal.

Complete Case Study: Automating a Restaurant (Figures 3.4 – 3.7)

1. Project Background & Stakeholders

A restaurant owner wants to automate operations to boost efficiency and attract customers.

- **Client:** The restaurant owner.
- **Potential End Users:** Waiters and the cash register operator.

2. Step-by-Step Analysis Progression

- **Stage 1: The Context Diagram (Figure 3.4)**
    1. Shows the entire Restaurant as a single black-box bubble interacting with external entities:
        - **Supplier:** Receives Order for Supplies from the restaurant and sends Supplies in return.
        - **Customer:** Sends Orders and Payment to the restaurant; receives Served Meals, Final Bill, and Receipt.
        - **Other Flows:** Menu enters the restaurant; Supplies Information and Sale Information exit the restaurant.

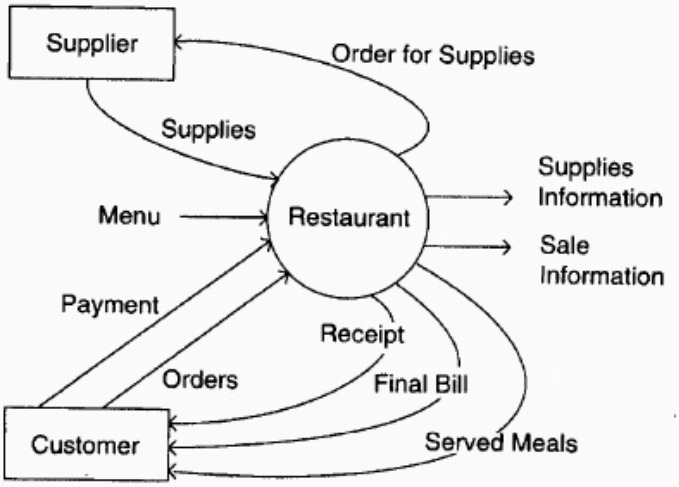


Figure 3.4: Context diagram for the restaurant.

- **Stage 2: Logical DFD of the Existing Manual Restaurant (Figure 3.5)**

    1. Created after observing restaurant operations and interviewing the owner. *(Note: A separate "physical DFD" was skipped because the existing manual processes and data had no special physical names, making the logical and physical diagrams identical.)*

    2. Maps existing manual bubbles (Receive Supplies, Make Payment, Make Dishes, Order Supplies, Take Order, Process Order, Produce Bill, Record Sale, Payment) and physical ledger files (Supplies Register, Sale Register).

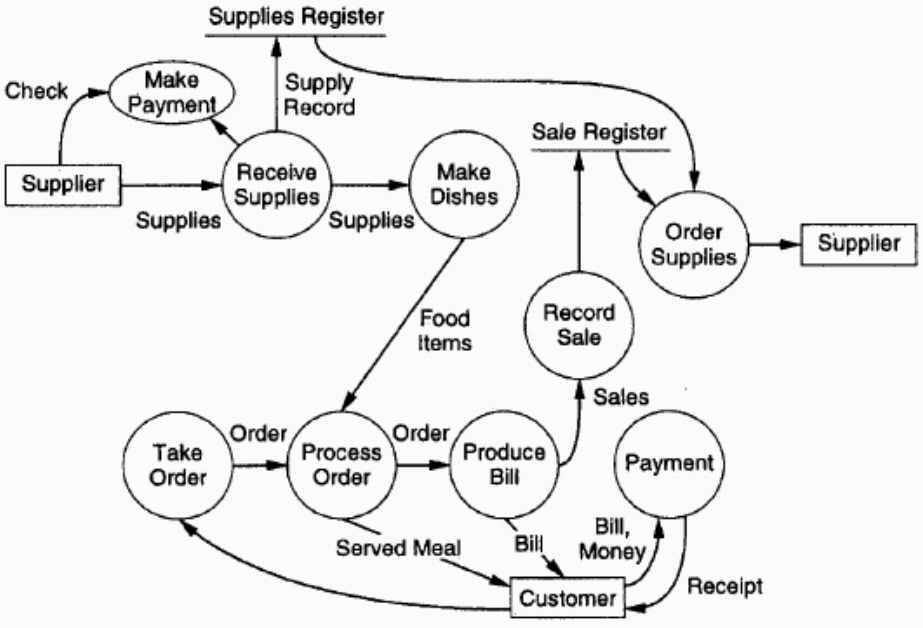


Figure 3.5: DFD for the existing restaurant system.

- **Stage 3: Setting 5 Goals for the New Automated System**
  After meetings with the owner, five specific goals were established:

  1. Automate most of order processing and customer billing.
  2. Automate accounting.
  3. Make supply ordering much more accurate (based on actual sales analysis) to minimize both end-of-day food waste (leftovers) and out-of-stock menu items.
  4. Detect and reduce staff theft or unauthorized eating of food/supplies.
  5. Provide sales statistics for different menu items.

- **Stage 4: Defining Boundaries & Drawing the New System DFD (Figure 3.6)**

  1. **Boundary of Change:** Includes almost every part of the old system **except Make Dishes** (cooking food remains unchanged). Even though waiters still take orders manually, the order-taking process falls *inside* the boundary of change because waiters now have to enter item codes for menu dishes.
  2. **Man-Machine Boundary:** Shown as a dashed line running down Figure 3.6, separating manual human tasks on the left (Process Supplies, Make Dishes, Process Order, and Supply Manager) from the automated computer processes and files on the right.
  3. **4 Major Files (Data Stores) in the New System:** Supplies File, Account File (Accounting file), Orders File, and Menu.
  4. **3 Brand-New Processes Added to Meet the Owner's Goals:**
     - **Check for Discrepancy (Anti-Theft/Waste Goal):** Compares total supplies received (from Supplies File), Supplies Left at the end of the day (reported by the Supply Manager), total customer orders placed (from Orders File), and ingredient consumption rates per dish (from Menu) to generate a discrepancy Report for the Owner.
     - **Account Reports (Automated Accounting Goal):** Reads the Account File and delivers accounting reports to the Owner.
     - **Statistics (Sales Tracking Goal):** Analyzes the Orders File and delivers item sales statistics to the Owner.
  5. **Smarter Supply Ordering (Place Supply Order):** Calculates next-day supply needs using both satisfied orders and *unsatisfied orders* (orders that couldn't be fulfilled due to missing ingredients), both stored in the Orders File, alongside recipe rates from the Menu.

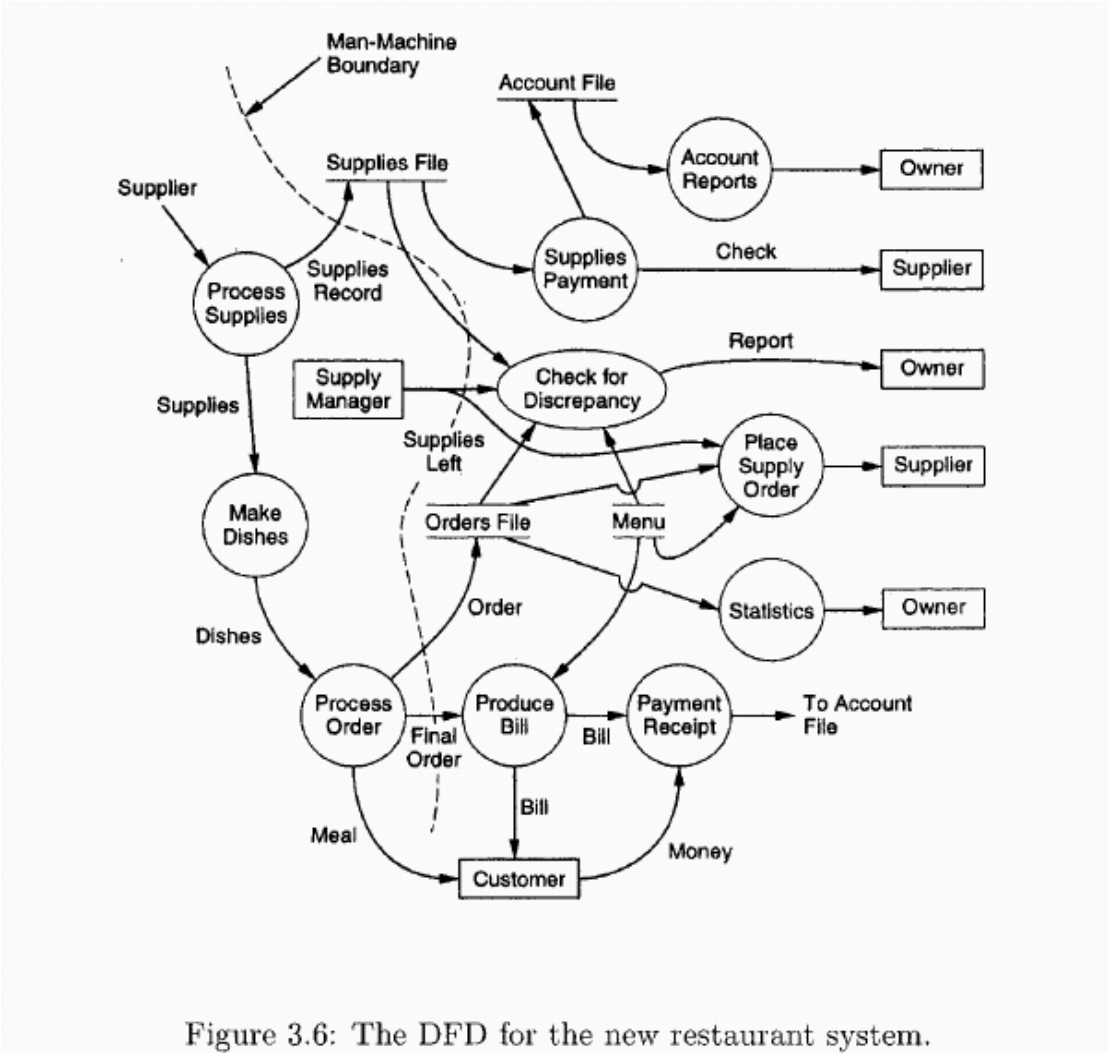


Figure 3.6: The DFD for the new restaurant system.

- **Stage 5: Restaurant Data Dictionary (Figure 3.7)**
    1. Precisely defines the data structures used in the new restaurant DFD:
        - Supplies_file = [date + [item_no + quantity + cost]*]*
        - Orders_file = [date + [menu_item_no + quantity + status]*]*
        - status = satisfied | unsatisfied
        - order = [menu_item_no + quantity]*
        - menu = [menu_item_no + name + price + supplies_used]*
        - supplies_used = [supply_item_no + quantity]*
        - bill = [name + quantity + price]* + total_price + sales_tax + service_charge + grand_total
        - discrepancy_report = [supply_item_no + amt_ordered + amt_left + amt_consumed + descr]*

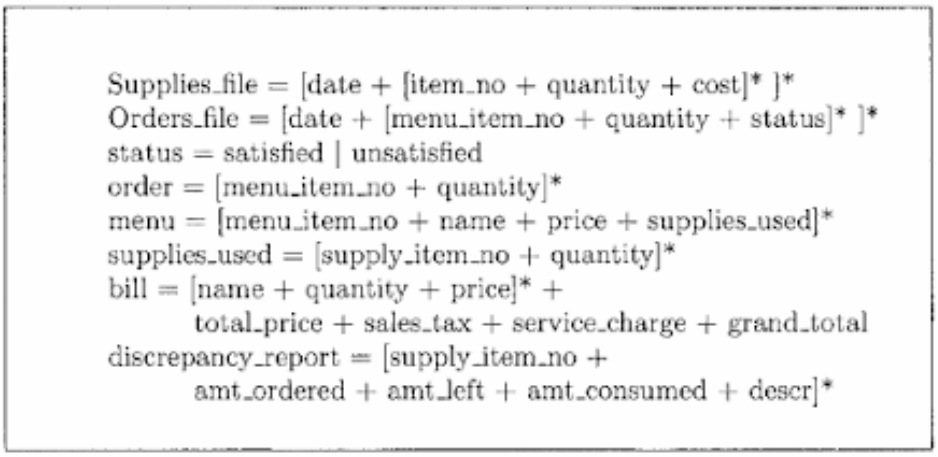


Figure 3.7: Data dictionary for the restaurant.

- **Stage 6: Final Steps Before Writing the SRS**
  Once the restaurant owner approves the new DFD (Figure 3.6) and Data Dictionary (Figure 3.7), problem understanding is complete. To finish the analysis phase and compile the formal SRS document, the analyst must:

  1. Specify the detailed functional requirements for every automated bubble inside the man-machine boundary.
  2. Define all non-functional (performance, security, constraint, and interface) requirements.
  3. Finalize the numbering/coding scheme for menu items and supply items.
  4. Refine complex bubbles further (such as defining the exact types and visual layouts of the accounting reports and sales statistics).

Chapter 5: Software Design (Unit 3: Design Modeling — Sections 5.1 to 5.4.2 & Abstraction)

Introduction to Software Design

- **Creativity vs. Analysis:** Software design is a more creative process than analysis because it deals with building the actual mechanics for a new, workable system. While analysis can produce a single "correct" model of an existing system, there is no such thing as a single "correct" design.
- **System Dependency:** Good design is always dependent on the specific system—what is a good design for one system may be bad for another.
- **Trial-and-Error Problem Solving:** Design must be done in great detail because it serves as the foundation for future computer programming and system implementation. It is a problem-solving activity based on trial and error, where the designer and users search for a solution using professional wisdom until they agree that a satisfactory solution has been found.
- **The Bridge Between Specifications and Coding:** For small projects (such as student projects), you can sit down with the specifications and directly write a program. For larger projects, you need a concrete bridge between specifications and coding—this bridge is **software design**.

5.1 What is Design?

- **Core Definition ("What" vs. "How"):** Design is the highly significant phase in software development where the designer plans **"how"** a software system should be produced so that it is functional, reliable, and reasonably easy to understand, modify, and maintain.
- **From SRS to SDD:** A **Software Requirements Specification (SRS)** document tells us **"what"** a system does and serves as the input to the design process. The design process determines **"how"** the software system works (how requirements are realized) and produces a **Software Design Document (SDD)**. Thus, the purpose of the design phase is to produce a solution to a problem given in the SRS document.

The Design Framework (Figure 5.1) The design framework begins with initial requirements and ends with a completed design through an iterative cycle:

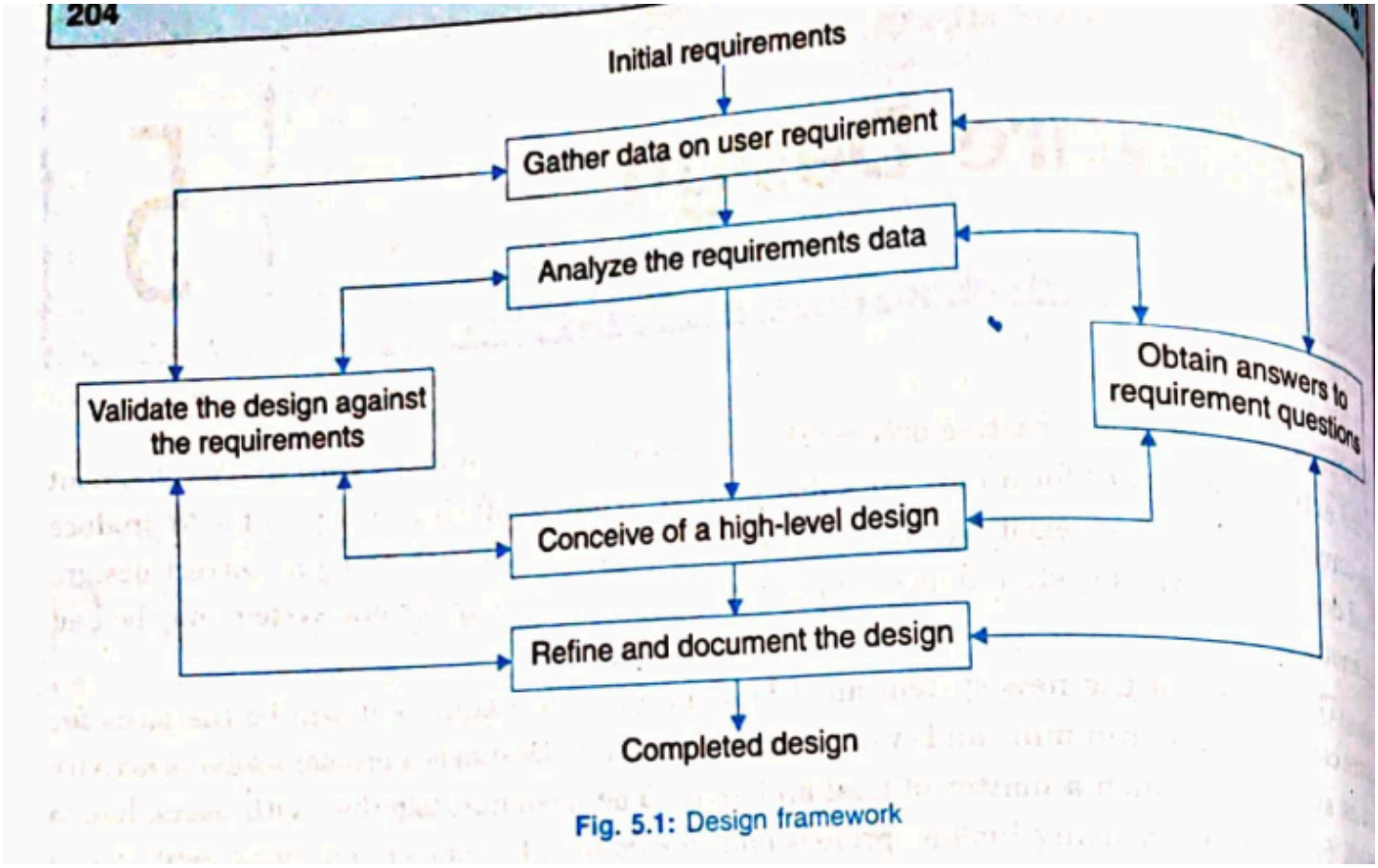


Fig. 5.1: Design framework

1. **Initial Requirements → Gather Data on User Requirement:** Data is collected on user requirements.
2. **Analyze the Requirements Data:** The gathered data is analyzed, and any gaps are resolved via **Obtain answers to requirement questions**.
3. **Conceive of a High-Level Design:** A high-level design is prepared after answering requirement questions.
4. **Validate the Design Against the Requirements:** The design is regularly validated against requirements, looping back to earlier steps as needed.
5. **Refine and Document the Design → Completed Design:** The design is refined in every cycle and finally documented to produce the **Software Design Document (SDD)**.

5.1.1 Conceptual and Technical Designs

To turn requirements into a working system, designers must satisfy two different audiences—**customers** (who need to know *what* the system will do) and **system builders / coders** (who need to know *how* the system is to work). Because of this, software design is an iterative, **two-part design process (Figure 5.2)**:

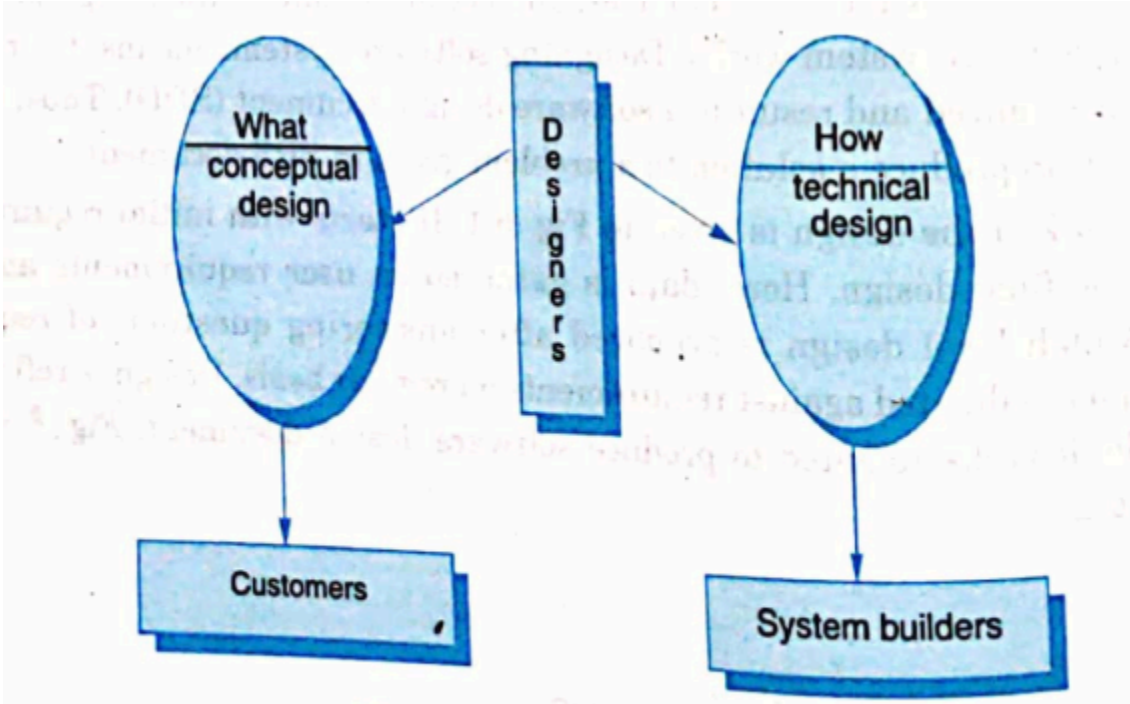


Fig. 5.2: A two-part design process

Feature	Part 1: Conceptual Design	Part 2: Technical Design
Core Focus	**"What"** the system will do.	**"How"** the system will work.
Target Audience	**Customers**.	**System builders** (coders/implementers).
Language & Style	Written in plain language understandable to the customer, completely free of technical jargon, and independent of implementation details.	A highly detailed technical document created *after* the customer approves the conceptual design.
What It Describes / Answers	Answers 6 key questions [PFLE98]: 1. Where will the data come from? 2. What will happen to the data in the system? 3. How will the system look to users?	Describes: • Hardware configuration • Software needs • Communications interfaces • Input and output of the system

	4. What choices will be offered to users? 5. What is the timing of events? 6. How will the reports and screens look like?	• Network architecture • Anything else that translates requirements into a technical solution

- **Exception for Sophisticated Customers:** When customers are software developers themselves and can understand both the "what" and "how" together, a separate conceptual design may not be necessary; instead, a single **comprehensive design document** may be produced.

5.1.2 Objectives of Design

- **Bridging the "Outside" and "Inside" Views:** A specification represents the "outside" view of a program and should be free of how the program works internally (the "inside" view). Because you can rarely write code directly from a specification, design bridges this gap by deciding how the program will be organized and what methods it will use in enough detail to be directly codeable.
- **Stepping-Stones for Complex Systems:** When a specification calls for a large or complex program, design works down through multiple levels, breaking the implementation problem into smaller and simpler problems at each level. Filling a large gap requires multiple "stepping-stones"—the wider the gap, the greater the number of stepping-stones required.
- **4 Essential Qualities of a Good Design:**
    - **Correct and complete**.
    - **Understandable**.
    - **At the right level**.
    - **Maintainable**, and structured to facilitate maintenance of the produced code.
- **Iterative Refinement from Informal to Finished Design (Figure 5.3 [SOMM2K]):**

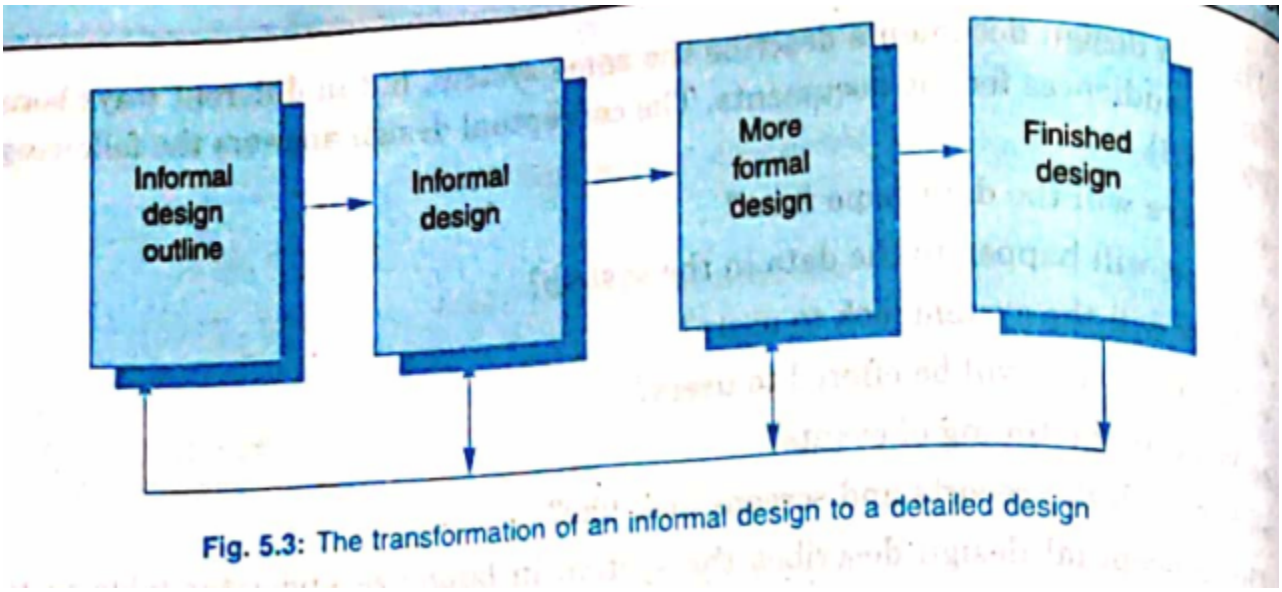


Fig. 5.3: The transformation of an informal design to a detailed design

    - Designers do not produce a finished design document all at once; instead, they add details iteratively across phases while constantly **backtracking** to correct earlier, less formal designs to make them consistent and complete:

- $Informaldesignoutline \rightarrow Informaldesign \rightarrow Moreformaldesign \rightarrow Finisheddesign$ *(with backtracking arrows returning from later stages to earlier stages)*.

5.1.3 Why Design is Important?

- **Vitruvius's 3 Classic Attributes of Good Design:** Almost 2,000 years ago, the Roman architect Vitruvius recorded three attributes of a good design: **Durability**, **Utility**, and **Charm**.
- **Risks of Skipping Design:** A well-designed system is easy to implement, understandable, reliable, and allows for smooth evolution. Without proper design, we risk building an **unstable system**:
    1. One that will fail when small changes are made.
    2. One that will be difficult to maintain.
    3. One whose quality cannot be assessed until late in the software process.
- **3 Guide Characteristics for Evaluating a Good Design:**
    1. **Complete Requirements Coverage:** The design must implement all **explicit requirements** contained in the analysis model and accommodate all **implicit requirements** desired by the customer.
    2. **Readable Guide:** The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
    3. **Complete Implementation Picture:** The design should provide a complete picture of the software, addressing the **data, functional, and behavioral domains** from an implementation perspective.

5.2 Design Concepts: Abstraction, Modularity, and Functional Independence

1. Abstraction (From Section 5.6.1)

- **Definition:** In software design, complexity is managed using **abstraction**, which is defined as **the elimination of the irrelevant and the amplification of the essentials**.
- **Levels of Abstraction (The Car Analogy [FIEL01]):**
    - **High Level of Abstraction (Driving a Car):** You can teach someone to drive any car by amplifying the essentials (the ignition and steering wheel) and eliminating irrelevant internal details (how fuel is pumped into the engine, ignited by a spark, pushing down a piston, and driving a crankshaft).
    - **Intermediate Level of Abstraction (The Mechanic):** A mechanic works at a lower level of abstraction and cares about the engine and battery, but still uses abstraction—charging a battery without caring about the complex chemical reaction inside it.
    - **Low Level of Abstraction (The Battery Designer):** A battery designer cares about the internal chemical reactions, but ignores the electronics inside the car's stereo.
- **Why Abstraction Matters in Design:** By separating details into manageable chunks and ignoring irrelevant lower-level details when viewing the whole system, designers can comprehend both the entire system and its individual parts easily. Working at the right level of abstraction lets you fix design flaws in 30 seconds with a pencil and eraser rather than knocking down and rebuilding a wall after a house is already built.

2. Modularity (Section 5.2)

- **What is a Module? [FAIR2K]:** Valid definitions of a "module" range from a FORTRAN subroutine, an Ada package, PASCAL and C procedures and functions, C++/Java classes, and Java packages, to a work assignment for an individual programmer.
- **Definition of a Modular System:** A system consisting of well-defined, manageable, discrete units (components) with well-defined interfaces, such that each component can be implemented separately and a change to one component has minimal impact on other components.
- **Why Modularity is Crucial [MYER78]:** Modularity is the single attribute of software that allows a program to be **intellectually manageable**. It enhances design clarity, which eases implementation, debugging, testing, documenting, and maintenance.
- **6 Desirable Properties of a Modular System:**
    1. Each module is a well-defined subsystem that is potentially useful in other applications.
    2. Each module has a single, well-defined purpose.
    3. Modules can be separately compiled and stored in a library.
    4. Modules can use other modules.
    5. Modules should be easier to use than to build.
    6. Modules should be simpler from the outside than from the inside.

Modularity and Software Cost (Figure 5.4)

- **The Tradeoff:** A software system cannot be made modular simply by chopping it into random pieces; each module must support a well-defined abstraction and have a clear interface for interacting with other modules.
- **Understanding the Graph (Figure 5.4):**

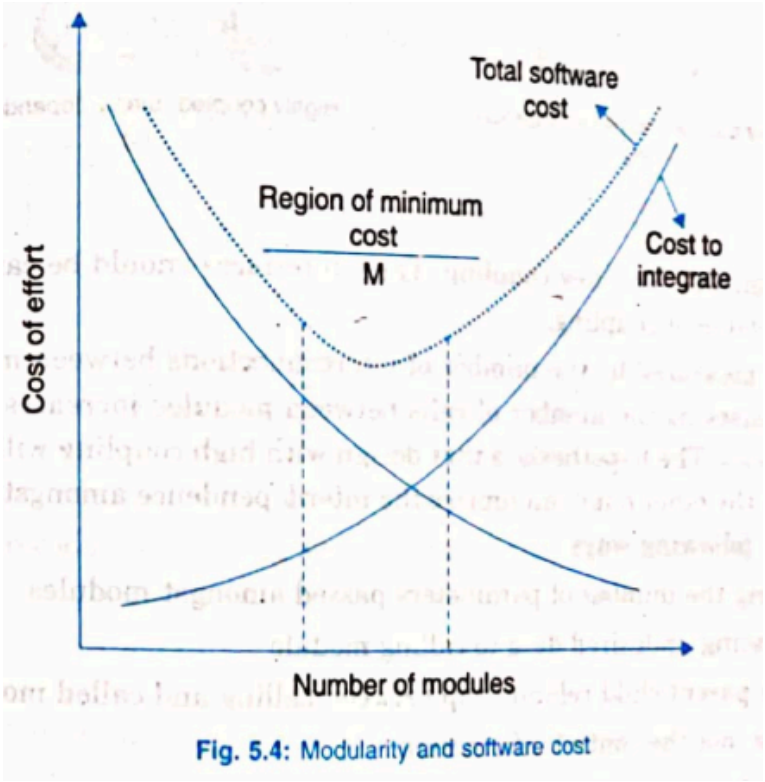


Fig. 5.4: Modularity and software cost

- As the **number of modules** increases, the effort/cost to code individual modules decreases (downward curve), because smaller modules are easier to build.
- However, as the number of modules grows, the **cost to integrate** those modules also grows (upward curve).
- Combining both curves produces a U-shaped **Total software cost** curve with a sweet spot in the middle labeled **Region of minimum cost (M)**.
- **Conclusion:** Both **under-modularity** (too few modules) and **over-modularity** (too many modules) increase total cost and should be avoided.

3. Functional Independence: Coupling and Cohesion (Sections 5.2.1 & 5.2.2)

Functional independence is achieved by designing modules with **low (loose) coupling** and **high (strong) cohesion**.

A. Module Coupling (Section 5.2.1)

- **Definition:** Coupling is the measure of the **degree of interdependence between modules**.
- **3 Degrees of Coupling (Figure 5.5):**

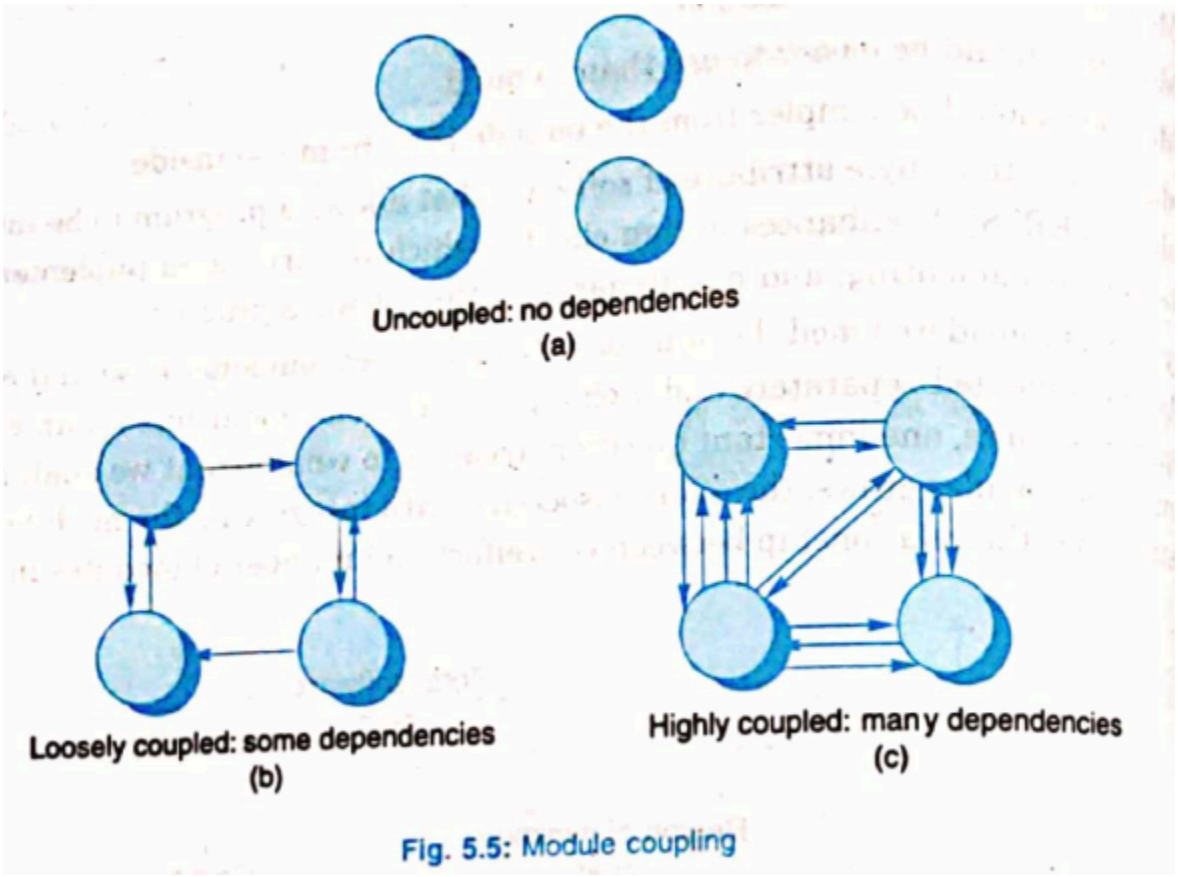


Fig. 5.5: Module coupling

- **Uncoupled (Fig. 5.5a):** Modules have no interconnections or dependencies at all; they are completely independent.
- **Loosely Coupled (Fig. 5.5b):** Modules have some dependencies but are relatively independent.
- **Highly Coupled (Fig. 5.5c):** Modules have many dependencies and are strongly interconnected (for example, when modules share global variables).

- **How Coupling is Measured & Why Low Coupling Matters:** Coupling is measured by the number of interconnections between modules—it increases as the number of calls between modules or the amount of shared data increases. High coupling leads to more software errors, so a good design always aims for **low (loose) coupling**.
- **4 Ways to Achieve Loose Coupling:**
    - Control the number of parameters passed amongst modules.
    - Avoid passing undesired/superfluous data to a calling or called module.
    - Maintain a clear parent/child relationship between calling and called modules.
    - Pass data, not control information.
- **Example of Tight vs. Loose Coupling & "Tramp Data" (Figure 5.6):**

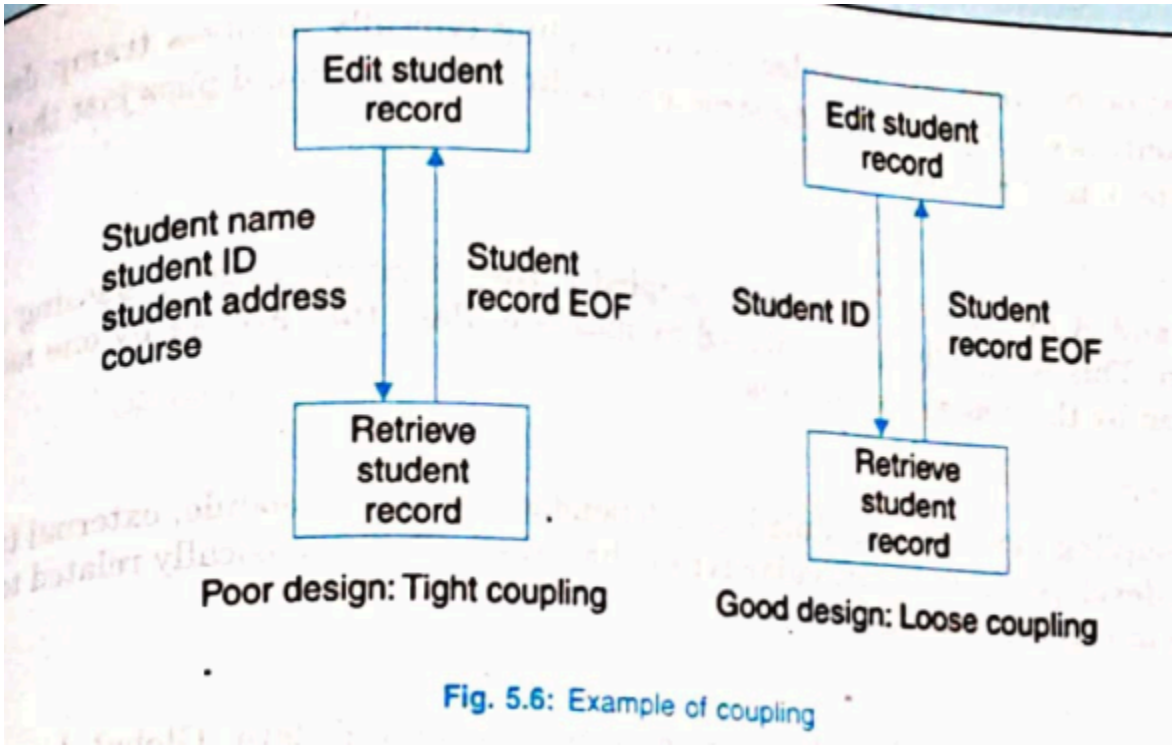


Fig. 5.6: Example of coupling

    - In a Student Information System where Edit student record calls Retrieve student record:
        - **Poor Design (Tight Coupling):** Passes Student name, student ID, student address, and course to Retrieve student record. Here, student name, address, and course are **"tramp data"** (unnecessary/superfluous data passed between modules that the receiving module does not need), which increases overhead and reduces system performance and efficiency.
        - **Good Design (Loose Coupling):** Passes *only* the necessary Student ID and returns Student record EOF, minimizing module dependency.

The 6 Types of Module Coupling — Ranked from Best (Lowest) to Worst (Highest) (Figure 5.7)

Rank	Coupling Type	How It Works & Example

1 (Best / Lowest)	**Data Coupling**	Modules A and B communicate **only by passing necessary elementary data** (parameters) and are otherwise completely independent. Contains zero "tramp data".
2	**Stamp Coupling**	Modules A and B pass a **complete data structure** from one module to another, even though the receiving procedure only needs a small part of that structure. Because unneeded fields are sent along, stamp coupling involves **tramp data**. *(Fix: Pass only the specific fields needed)*.
3	**Control Coupling**	Modules A and B communicate by passing **control information**—typically done using **flags** set by one module that tell the dependent module how to behave or execute.
4	**External Coupling**	A module has a dependency on something **external** to the software being developed, such as external tools, devices, or a specific type of hardware.
5	**Common Coupling** *(Figure 5.8)*	Modules A and B share a **common global data area** (global variables). In Fig. 5.8 [PFLE98], global variables V1, V2 and A1, A2, A3 are directly modified by Module X (*Change V1 to Zero*), Module Y (*Increment V1*), and Module Z (*V1 = V2 + A1*). If you change a global variable, you must trace back every single module that accesses it, making it very hard to know which module set a variable to a given value.
6 (Worst / Highest)	**Content Coupling** *(Figure 5.9)*	Occurs when Module A directly modifies the internal data of Module B, or when control branches (Go To) from one module directly into the **middle of another module**. In Fig. 5.9, Module B executes Go To temp to jump directly into the middle of Module D, violating hierarchy even though D is supposed to be controlled by C.

B. Module Cohesion (Section 5.2.2)

- **Definition (Figure 5.10):** Cohesion is the measure of the degree to which the elements *inside* a single module are functionally related. It is the "glue" (Module strength) or mutual affinity that keeps the components of a module together.

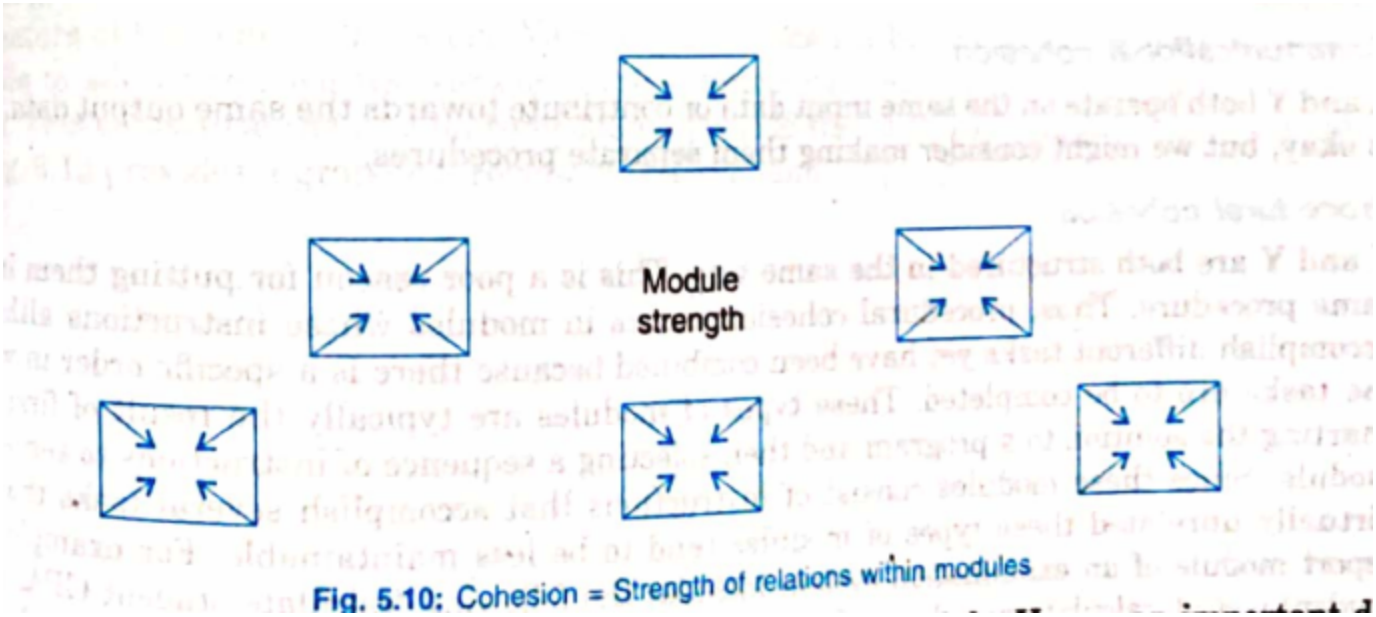


Fig. 5.10: Cohesion = Strength of relations within modules

- **Core Goal:** A strongly cohesive module implements functionality tied to **one single feature** of the solution and requires little or no interaction with other modules. Therefore, our primary design objective is to **maximize module cohesion and minimize module coupling**.

The 7 Types/Levels of Module Cohesion — Ranked from Best (Highest) to Worst (Lowest) (Figure 5.11)

Assuming a procedure carries out internal operations X and Y, cohesion is ranked from highest (best) to lowest (worst) as follows:

Rank	Cohesion Type	Relationship Between XXX and YYY & Textbook Example
1 (Best / Highest)	**Functional Cohesion**	X and Y are part of a **single functional task**, often transforming a single input datum into a single output datum. *Example:* Mathematical subroutines like 'calculate current GPA' or 'calculate cumulative GPA'.
2	**Sequential Cohesion**	Operation X outputs data that directly forms the **input to operation** Y. *Example:* Adding marks of individual subjects into a specific format (X), which is immediately used as input to calculate the GPA (Y) for preparing student results.
3	**Communicational Cohesion**	X and Y both operate on the **same input data** or contribute toward the **same output data** for communicational convenience. *(This is acceptable, though making them separate procedures can be considered)*. *Example:* Calculating both current GPA and cumulative GPA inside one component because both use the "student grade record" as input.
4	**Procedural Cohesion**	X and Y perform different, virtually unrelated tasks, but are combined because they must execute in a **specific order**. Typically happens when a developer draws a flowchart first and slices a sequence of steps into a module; these modules are less maintainable. *Example:* An exam report module that runs "calculate

		student GPA, print student record, calculate cumulative GPA, print cumulative GPA".
5	**Temporal Cohesion**	X and Y are grouped only because both must be executed around the **same time-span** (a poor reason to combine them). *Example:* Initialization or startup functions that set program counters and control flags at program startup.
6	**Logical Cohesion**	X and Y perform **logically similar operations** that fall into the same general category of functions, often causing code duplication. *Example:* Writing separate inline code to validate multiple date fields in an input transaction (better solved by creating a single DATECHECK module and calling it whenever needed).
7 (Worst / Lowest)	**Coincidental Cohesion**	X and Y have **little or no conceptual relationship** to one another other than shared code. Instead of making two separate components, two unrelated tasks are lumped into one; this must be avoided as much as possible. *Example:* A single component that performs check validity and print.

Relationship Between Cohesion and Coupling (Figure 5.12)

- **Surviving Requirement Changes:** If software is not properly modularized, even trivial enhancements or requirement changes can result in the death of the project. Clean decomposition into modules with **high cohesion** (strong internal bonds) and **low coupling** (minimal links between boxes, as shown on the right side of Fig. 5.12) localizes changes.
- **Real-World Analogy ("Plug and Play"):** A computer motherboard's 'plug and play' slots allow you to add or remove hardware cards/services without disrupting the rest of the computer because each add-on component provides its service in a highly cohesive, loosely coupled manner.
- **Black-Box Advantage:** High cohesion and low coupling allow each module to be treated as a **black box** when describing the overall system structure, and handled independently when describing its internal functionality.

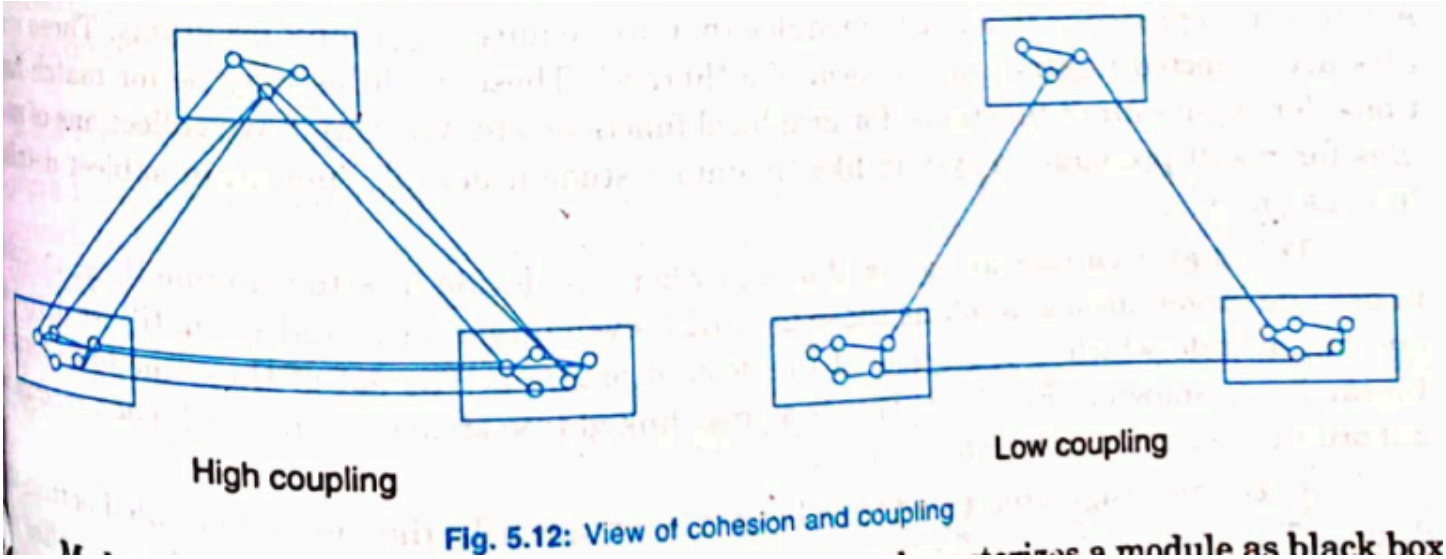


Fig. 5.12: View of cohesion and coupling

5.3 Strategy of Design

Why Structured Design Strategies Are Needed

- A good system design strategy organizes program modules so they are easy to develop and easy to change later. Analysts create instructions for developers on how code should be written and pieced together for two reasons:
    - Even pre-existing code (if any) must be understood, organized, and pieced together.
    - Project teams still commonly have to write original code to support the application logic of the system.
- **Why Flowcharts Failed for Large Software Design:**
    - In the early days, design meant "writing down the flowchart in words". Because flowcharts are at a very low level, they force detailed decisions too early and require a sudden, massive jump from specifications to flowcharts, causing many errors.
    - Flowchart errors could only be discovered after coding and running the program, diagnosing where the flowchart was wrong, fixing it, and recoding—repeated "surgery" on the flowchart ruins it and can cause the project to fail.
    - Modern design notations operate at a **higher level** than flowcharts, allowing multiple levels of design that replace one or two massive jumps with a series of small, manageable jumps.

Comparison of the 3 Design Strategies (Sections 5.3.1, 5.3.2, 5.3.3)

Strategy	How It Works	Best Used When...	Strengths & Weaknesses

1. Bottom-Up Design *(Section 5.3.1, Fig. 5.13)*	Starts by identifying common utility modules used by many programs (e.g., math, I/O, graphics, or result-preparation modules like "maintain student detail", "maintain subject detail", "marks entry") and collecting them in a **"library"**. Small modules are combined into larger ones, progressing upward into one top module forming a **cross-linked tree structure (Fig. 5.13)** where a lower module can be subordinate to multiple higher modules.	Building a new system from an **existing system** or existing reusable modules.	• **Strength:** Coded low-level modules can be tested and design-validated immediately without waiting for sub-modules to be designed. • **Terrible Weakness:** Requires heavy intuition to guess what functionality upper layers will need. If you guess wrong, you discover at the top level that it doesn't meet requirements, forcing lower-level redesign and recoding.
2. Top-Down Design *(Section 5.3.2)*	Views the system specification as a single top-level **black box**, decomposes it into smaller inner black boxes, and repeats (**stepwise refinement**) from an abstract design down to a concrete level that can be coded directly.	Specifications are clear and development is **from scratch** (most design methodologies are based on this).	• **Strength:** Stays aligned with high-level system requirements from the start. • **Weakness:** If you code a high-level module right after designing it, **nothing can be tested** until all of its subordinate lower-level modules are coded.
3. Hybrid Design *(Section 5.3.3)*	Combines top-down and bottom-up approaches because pure top-down or pure bottom-up designs are rarely practical on their own. Bottom-up needs a clear idea of the top [JALO98], while top-down needs bottom-up elements at the lowest levels.	Almost all real-world projects, especially those using **reusable modules** (standard libraries, Microsoft Foundation Classes [MFCs], OO concepts).	3 Reasons Top-Down Needs Bottom-Up at Low Levels: 1. To permit **common sub-modules**. 2. Near the bottom of the hierarchy, intuition is simpler and the need for **bottom-up testing** is greater (since there are more modules at low levels than high levels). 3. To use and reuse **pre-written library modules**.

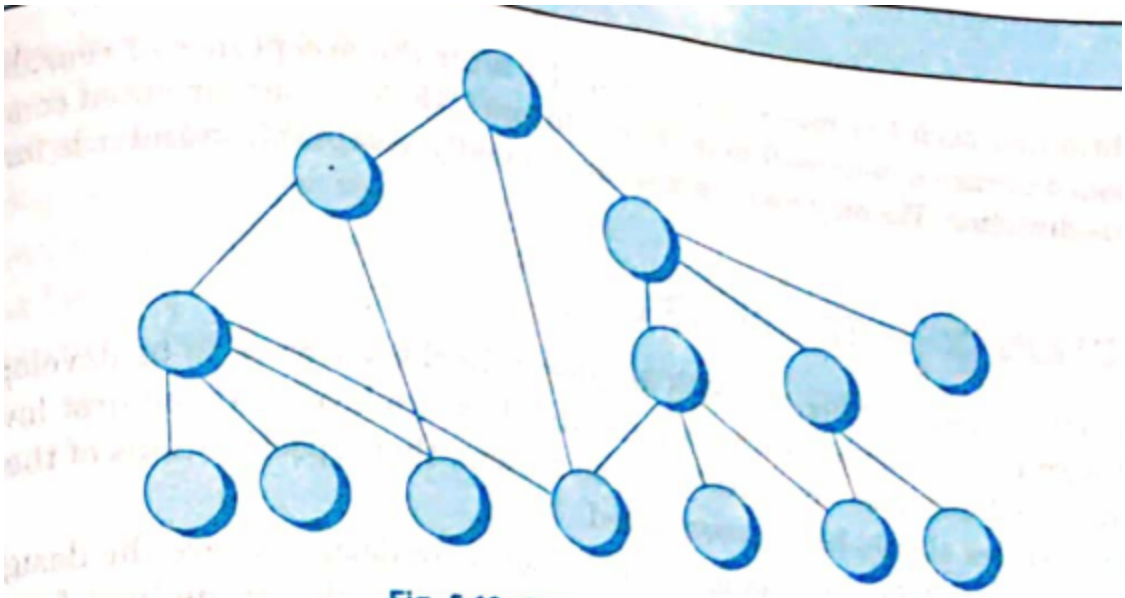


Fig. 5.13: Bottom-up tree structure

5.4 Function-Oriented Design

- **Two-Level Design Process:** Design begins once the SRS document is available. At the first level, the focus is on deciding which modules are needed for the system, the specifications of those modules, and how they should be interconnected.
- **Core Definition: Function-oriented design** decomposes a software design into a set of interacting units where each unit has a **clearly defined function** (designing the system from a functional viewpoint).
- **Niklaus Wirth's Stepwise Refinement:** Advocated by Niklaus Wirth (creator of PASCAL), **stepwise refinement** is a top-down method where you start with a high-level description of what the program does, and in each step refine one part in greater detail until you reach programming language statements.

3 Major Limitations of Top-Down Function-Oriented Design for Large Programs

1. **Hard to Define a Single Top-Level Function:** While stepwise refinement works well for small programs, it is questionable for large programs (like UNIX, an airline reservation system, or a Scheme interpreter) because what the system does depends on what the user types at the terminal. Any single top-level function risks being a highly artificial description of reality.
2. **Adding New Data Types Forces Changes Everywhere (The Scheme Interpreter Example):**
    - Suppose a Scheme interpreter's top-level loop is divided into Read, Evaluate, and Print modules:

```
While (not finished) {
        Read an expression from the terminal;
        Evaluate the expression;
        Print the value;
}
```

- Inside Print(expression exp), the module uses a Switch (exp type) statement with cases for integer, real, and list.
- *The Problem:* Every module (Read, Evaluate, Print) must know about every data type (integer, real, list). If you need to add a new data type later, **every single module must be altered**.

3. **Poor Reusability in Pure Top-Down Trees (Figure 5.14 vs. Figure 5.15):**
   - Creating a program purely top-down produces a strict tree (**Figure 5.14**) where modules become overly specialized and each module is used by *at most one parent module*.

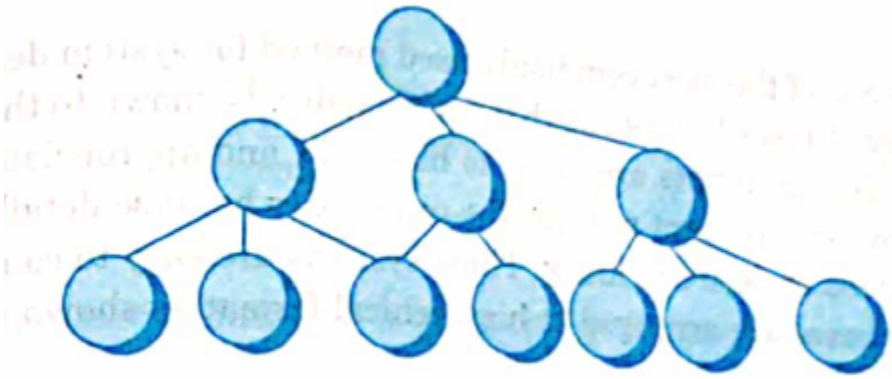


Fig. 5.14: Top-down structure

   - For a module to be **reusable**, several different modules must be able to call and share it, as shown in the **Design reusable structure (Figure 5.15)**.
   - *Data-Centered Alternative:* If we want to delay deciding what the system does for as long as possible, it is better to structure the program around the **data** rather than the actions taken by the program.

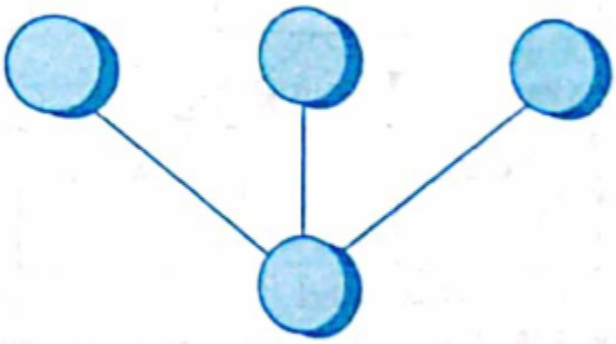


Fig. 5.15: Design reusable structure

5.4.1 Design Notations (Structure Charts & Pseudocode)

During the design phase, **design notations** are used graphically or mathematically to represent the design and design decisions. For function-oriented design, the **4 main notations** are:

1. **Data Flow Diagrams (DFDs)** *(covered in Chapter 3)*
2. **Data Dictionaries** *(covered in Chapter 3)*
3. **Structure Charts**

4. **Pseudocode**

1. Structure Charts (Figures 5.16 – 5.19)

- **What It Is:** One of the most commonly used methods for system design, a **structure chart** partitions a system into hierarchical **black boxes** (Figure 5.16).

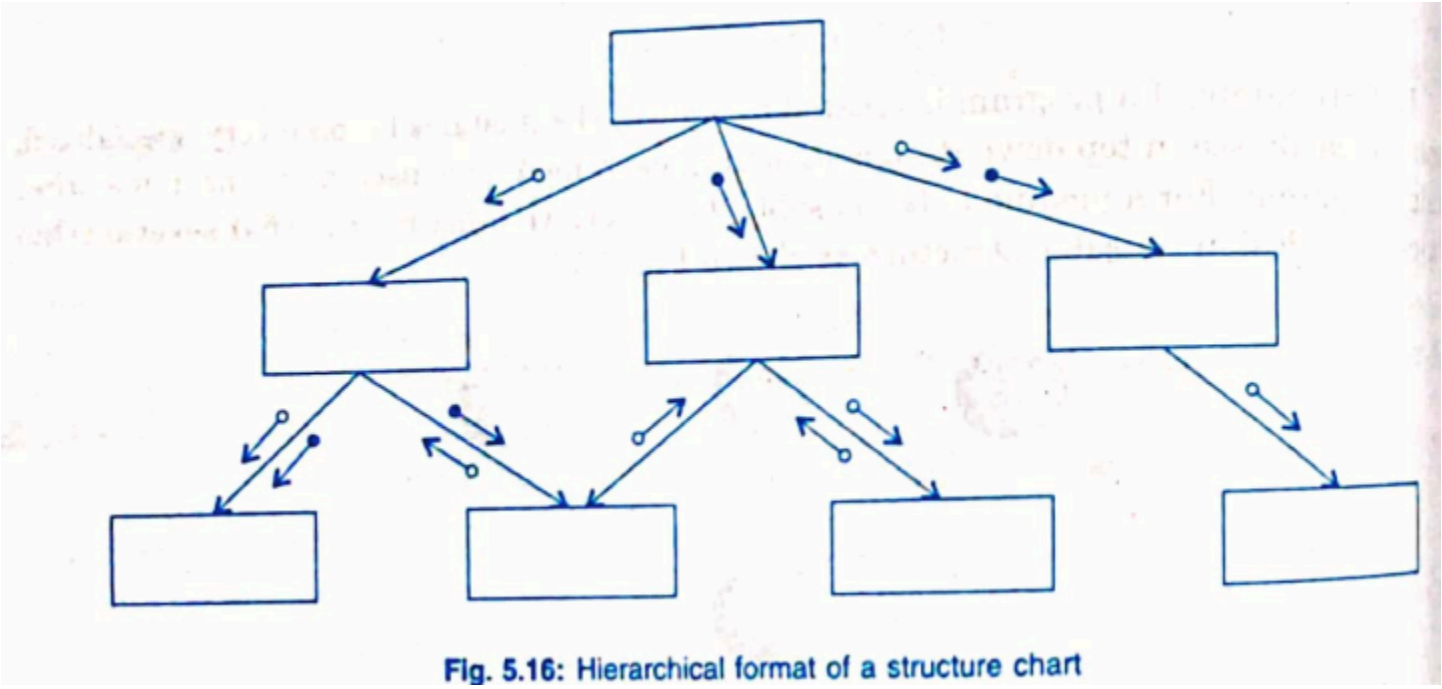


Fig. 5.16: Hierarchical format of a structure chart

- **Black-Box Principle:** A black box means the user/caller knows *what* functionality the module performs (inputs given and outputs generated) without knowing its internal design details. Hiding internal details reduces complexity and makes systems easy to construct and maintain.
- **Reading & Numbering Rules:** Top-level modules call lower-level modules, connected by lines. Components are generally read **from top to bottom, left to right**, and modules are numbered using a hierarchical numbering scheme.

Standard Structure Chart Notations (Figure 5.17)

Symbol in Structure Chart (Fig. 5.17)	**Meaning & Usage**
Rectangular Box	Represents a standard **Program Module**.
Rectangle with Double Vertical Side Lines	Represents a **Library Module** (a module reused/called by many other modules).
Arrow with Open Circle (◦→)	**Data Couple:** Represents **data** parameters passed between calling and called modules.
Arrow with Filled/Solid Circle (•→)	**Control Couple:** Represents **control data** (such as flags or status conditions) passed between modules.

Diamond Symbol (⬨)	**Conditional Call:** Attached to the bottom of a calling module to show that *one* module out of several connected modules is called depending on the outcome of a condition.
Curved Loop Arrow Around Control Lines	**Repetitive Call (Repetition Symbol):** Denotes that the enclosed modules are called repeatedly in a loop.
Cylinder / Curved-Side Box	Represents **Physical Storage**.

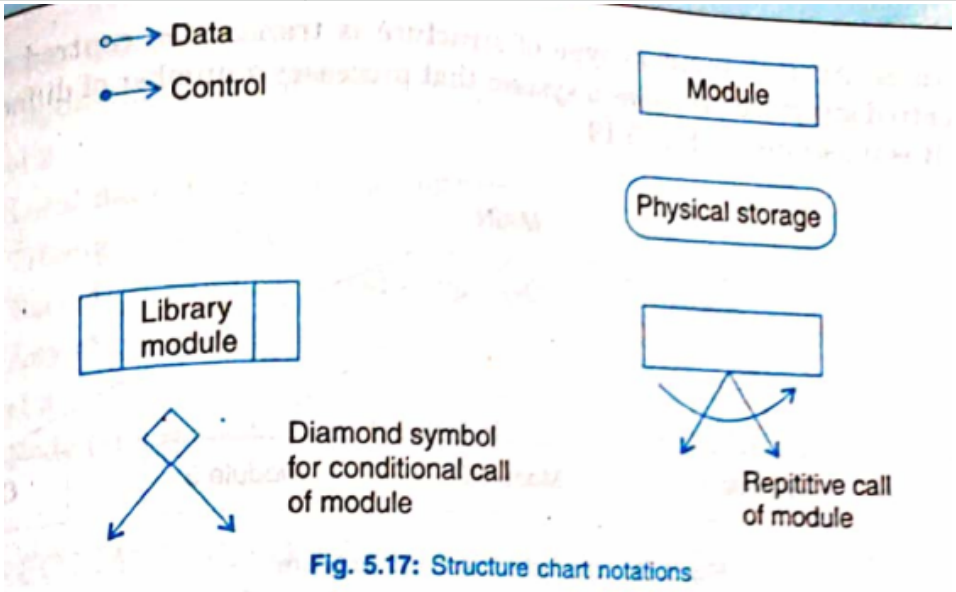


Fig. 5.17: Structure chart notations

Two Common Types of Structure Charts

Type A: Transform-Centered Structure (Example: Update File in Figure 5.18)

- **Definition:** A transform-centered structure receives an input that is transformed through a sequence of operations, with each operation carried out by one module.
- **Step-by-Step Walkthrough of Update File (Figure 5.18):**

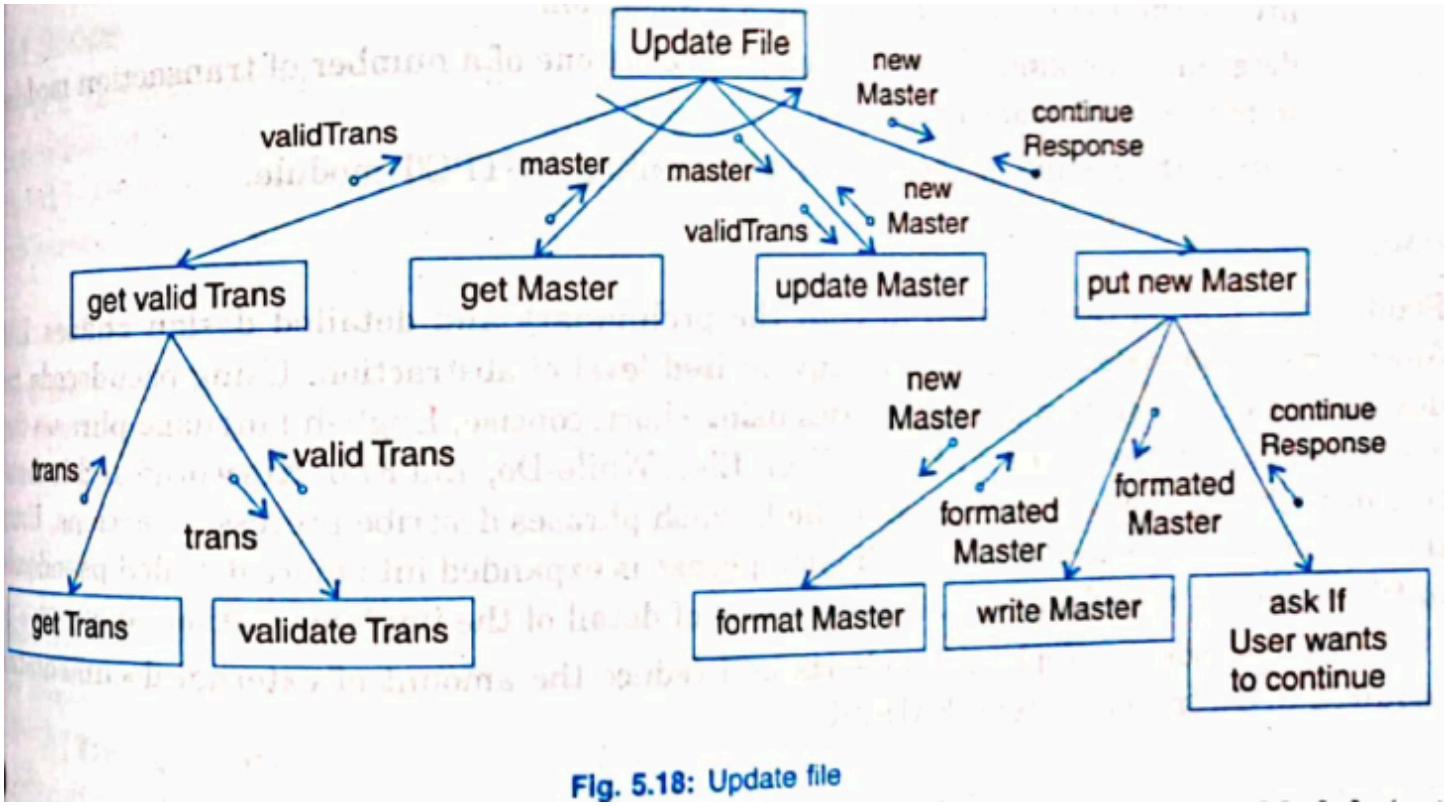


Fig. 5.18: Update file

1. The top module Update File has a **repetition loop arrow** around its four main branches:
2. **Branch 1 (get valid Trans):** Update File calls get valid Trans to obtain the input parameter valid Trans. Inside this branch, get valid Trans calls get Trans (which reads raw trans from the input device and passes it back) and then calls validate Trans (passing trans and receiving valid Trans to pass up to Update File).
3. **Branch 2 (get Master):** Update File calls get Master, which returns the master record.
4. **Branch 3 (update Master):** Update File passes both master and valid Trans down to update Master, which processes them and returns new Master up to Update File.
5. **Branch 4 (put new Master):** Update File passes new Master to put new Master, which calls three sub-modules: format Master (converts new Master into formated Master), write Master (writes formated Master), and ask if User wants to continue (returns the control flag continue Response all the way back to Update File to decide whether to repeat the loop).

Type B: Transaction-Centered Structure (Figure 5.19)

- **Definition:** Describes a system that processes a number of different types of transactions.
- **Step-by-Step Walkthrough of Figure 5.19:**
  1. The top MAIN module controls the overall system operation by performing three functions:
  2. Invokes the Input module to read an incoming transaction.
  3. Uses a **diamond conditional symbol (⬨)** to determine the kind of transaction and select one of several transaction modules (Module 1, Module 2, or Module 3) to process that specific transaction.
  4. Outputs the results of the processing by calling the Output module.

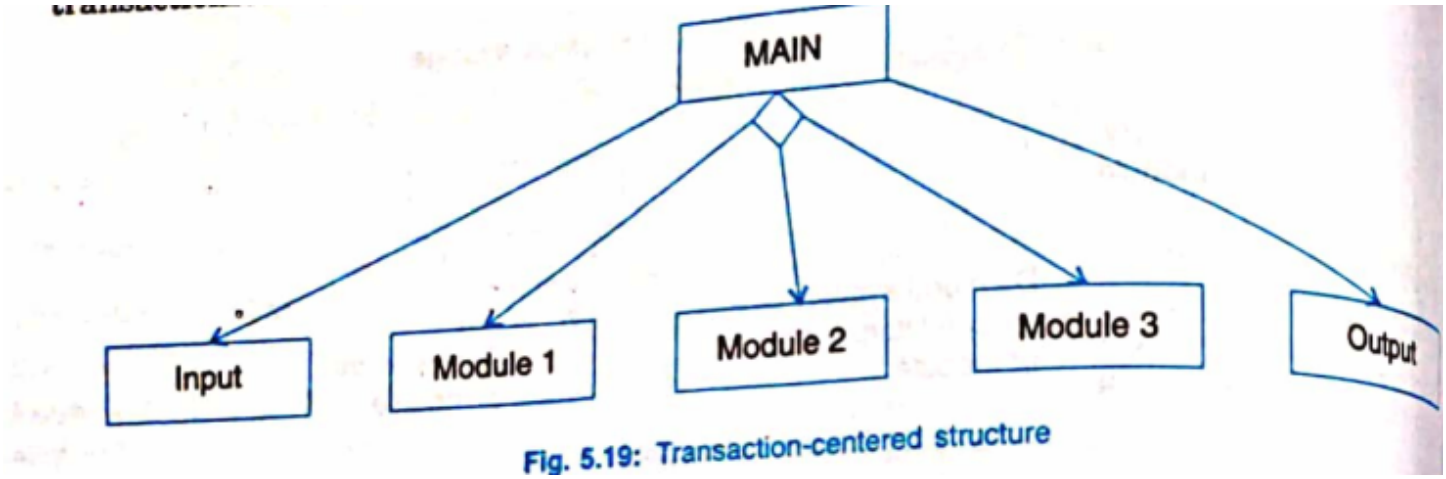


Fig. 5.19: Transaction-centered structure

2. Pseudocode

- **What It Is:** A text-based design notation that uses short, concise **English language phrases** structured by programming keywords (such as If-Then-Else, While-Do, and End) and indentation.
- **How It Works:** Keywords and indentation describe the **flow of control**, while the English phrases describe **processing actions**. Using a top-down strategy, each English phrase is progressively expanded into more detailed pseudocode until it reaches the level of detail of the actual programming language.
- **Advantages Over Flowcharts [FAIR2K]:** Unlike flowcharts, pseudocode can be used in **both preliminary and detailed design phases** at any desired level of abstraction, replacing flowcharts and reducing the amount of external documentation needed to describe a system.

5.4.2 Functional Procedure Layers

When specifying functions in detail during design, functions are built across **4 progressive layers of detail (Levels 0 to 3)**:

Layer	Information Specified at This Level
Level 0 *(Basic Identity & Context)*	• Function or procedure name • Relationship to other system components (e.g., part of which system, called by which routines, etc.) • Brief description of the function's purpose • Author and date
Level 1 *(Interfaces & External Interactions)*	• Function parameters (problem variables, types, purpose, etc.) • Global variables (problem variable, type, purpose, sharing information) • Routines called by the function • Side effects

	• Input/Output assertions
Level 2 *(Internal Constraints & Exceptions)*	• Local data structures (variables, etc.) • Timing constraints • Exception handling (conditions, responses, events) • Any other limitations
Level 3 *(Internal Logic / Body)*	• Body of the function (represented using a structure chart, English pseudocode, decision tables, flowcharts, etc.)

23.1.1 Measures, Metrics, and Indicators

Although people often use the words *measure*, *measurement*, and *metric* as if they mean the same thing, each term represents a distinct step in analyzing software:

Term	Definition in Simple Language	Concrete Example
1. Measure	A **single raw data point** collected from a software component or activity.	Counting the number of errors found inside *one* single software component.
2. Measurement	The act or result of collecting **multiple data points** (one or more measures) across several components or events.	Examining several component reviews and unit tests to record the number of errors found in *each* one.
3. Software Metric	A calculation or comparison that **relates individual measures to one another** in a meaningful way (such as a ratio or average).	Calculating the *average number of errors found per review* or the *average number of errors found per unit test*.
4. Indicator	A single metric—or a combination of metrics—that gives engineers and managers **actionable insight** into the software process, project, or product so they can make adjustments.	Combining error rates and review effort to see whether quality assurance is working effectively.

23.8 Metrics for Software Quality

Why We Measure Software Quality

- **The Foundation of Quality:** A software product is only as good as the **requirements** that describe the problem, the **design** that models the solution, the **code** that builds the executable program, and the **tests** that exercise the software to uncover errors.
- **Three Levels of Quality Metrics:** Because software is complex, errors are unavoidable during development:

    - **Process Metrics:** Used to improve the overall *software development process* so that errors are discovered as effectively as possible.
    - **Product Metrics:** Used in real time to objectively (rather than subjectively) evaluate the quality of *software work products*—including requirements models, design models, source code, and test cases.
    - **Project-Level Metrics:** Created by combining private metrics collected by individual software engineers into project-wide results. At the project level, the main goal is to measure **errors and defects** to see how well individual and group Software Quality Assurance (SQA) and quality control activities are performing.
- **Examples of Actionable Quality Metrics:**
    - Work product errors per function point (FP)
    - Errors uncovered per review hour
    - Errors uncovered per testing hour
    - Defect Removal Efficiency (DRE)

Gilb's 4 Software Quality Indicators [Gil88]

Tom Gilb proposed four practical attributes—**Correctness, Maintainability, Integrity, and Usability**—that serve as useful quality indicators for a project team:

1. Correctness

- **Definition:** The degree to which the software performs its required functions.
- **What is a Defect?** A *defect* (lack of correctness) is a verified lack of conformance to requirements reported by an end user **after** the program has been released for general use.
- **How It Is Measured:** Counted over a standard time period (typically **1 year**) and most commonly measured as **Defects per KLOC** (defects per thousand lines of code).

2. Maintainability

- **Definition:** How easily a program can be:
    1. **Corrected** if an error is found,
    2. **Adapted** if its operating environment changes, or
    3. **Enhanced** if the customer requests new or changed requirements.
- **How It Is Measured (Indirectly):** Maintainability cannot be measured directly, so teams use indirect, time-oriented metrics like **Mean Time to Change (MTTC)**.
- **MTTC Formula/Components:** The total average time required to:
- $MTTC = Analyzechangerequest + Designmodification + Implementchange + Testchange + Distributecha$

3. Integrity

- **Definition:** A system's ability to withstand both **accidental** and **intentional** attacks on its security.
- **The 2 Underlying Attributes:**

- $Threat$**:** The probability (estimated or based on past data) that a specific type of attack will happen within a given time.
- $Security$**:** The probability (estimated or based on past data) that a specific type of attack will be successfully repelled.

- **Integrity Formula:**

$$Integrity = \sum[1 - (threat \times (1 - security))]$$

- **Worked Examples:**
  - *Case 1 (High Integrity):* Suppose the probability of an attack ($threat$) is 0.25 and the likelihood of repelling it ($security$) is 0.95:

$$Integrity = 1 - (0.25 \times (1 - 0.95)) = 1 - (0.25 \times 0.05) = 1 - 0.0125 = 0.9875 \approx 0.99(VeryHigh)$$

  - *Case 2 (Low Integrity):* Suppose the probability of an attack ($threat$) is 0.50 and the likelihood of repelling it ($security$) is only 0.25:

$$Integrity = 1 - (0.50 \times (1 - 0.25)) = 1 - (0.50 \times 0.75) = 1 - 0.375 = 0.625 \approx 0.63(UnacceptablyLow)$$

4. Usability

- **Definition:** Quantifies how easy the software is for people to use.

Defect Removal Efficiency (DRE)

Defect Removal Efficiency (DRE) is a powerful quality metric used at both the **project level** and the **process level** to measure the "filtering ability" of quality assurance and control activities—how well the team catches errors before they slip through to the next stage or to the customer.

Level	Formula	Meaning of Variables	Goal & Interpretation
1. Project-Level DRE *(Whole Project)*	$DRE = \frac{E}{E+D}$	• E: Number of **errors** found *before* software delivery to the end user. • D: Number of **defects** found *after* delivery.	• **Ideal Value:** $DRE = 1$ (meaning $D = 0$, zero post-release defects). • **In Practice:** Realistically $D > 0$, but as the team finds more errors early (E increases), D drops and DRE approaches 1. Encourages teams to find as many errors as possible before release.
2. Process/Activity-Level DRE_i *(Between Phases)*	$DRE_i = \frac{E_i}{E_i+E_{i+1}}$	• E_i: Number of errors found during software engineering action i (e.g., requirements review). • E_{i+1}: Number of errors found in the next action $i + 1$ (e.g., design) that	• **Goal:** Achieve a DRE_i approaching 1 at every step so errors are filtered out before passing to the next phase. • **Actionable Fix:** If DRE_i is low between analysis and design, the team needs to improve how it conducts **Formal Technical Reviews (FTRs)**.

		originated from missed errors in action i.	

Case Study Summary: SafeHome – A Metrics-Based Quality Approach

- **Setting Goals First:** Manager Doug Miller reminds team members Vinod and Jamie that you should always establish clear goals before collecting metrics.
- **The Team's Quality Goals:** Keep errors passed from one activity to the next at an absolute minimum ($DRE_i \rightarrow 1$), and keep post-release product defects as close to zero as possible ($DRE \rightarrow 1$).
- **Candidate Metrics Proposed:** Jamie and Vinod suggest tracking project and activity-level DRE, hours spent on reviews, overall effort per software task (which Doug notes can be used to compute a *review-to-development ratio*), and effort required to develop and code each use case.
- **Key Takeaway ("Walk Before You Run"):** When the team gets carried away listing metrics, Doug advises keeping things simple by limiting collection to **5 or 6 core items** directly tied to their quality goal.

25.6.1 Software Sizing

Because a project estimate is only as accurate as your estimate of the size of the work to be done, **software sizing** is a project planner's first major challenge.

4 Factors That Determine Estimate Accuracy

1. How accurately you have estimated the **size** of the product to be built.
2. Your ability to **translate that size estimate into human effort, calendar time, and dollars** (which depends on having reliable software metrics from past projects).
3. How well the project plan reflects the actual **abilities of the software team**.
4. The **stability of product requirements** and the environment supporting the software engineering effort.

Direct vs. Indirect Approaches to Measuring Size

In project planning, **size** refers to a quantifiable outcome of the software project:

- **Direct Approach:** Size is measured directly in **Lines of Code (LOC)**.
- **Indirect Approach:** Size is represented functionally as **Function Points (FP)**.
- **4 Considerations When Estimating Size:**
    1. The type of project and its application domain.
    2. The functionality delivered (number of Function Points).
    3. The number of components (or use cases) to be delivered.
    4. The degree to which existing components must be modified for the new system.

25.6.4 An Example of FP-Based Estimation

Instead of decomposing a project by software functions, **Function Point (FP) estimation** decomposes the system into **5 information domain values**. Below is the complete step-by-step calculation for a Computer-Aided Design (CAD) software project.

Step 1: Estimate the 5 Information Domain Values and Compute Count Total (Table 25.1) For each of the 5 information domain categories, planners estimate an **Optimistic (Opt.)**, **Most Likely (Likely)**, and **Pessimistic (Pess.)** count to determine the **Estimated Count (Est. count)**, then multiply by an average **Complexity Weight** to get the **FP Count**:

Information Domain Value	Opt.	Likely	Pess.	Est. Count	Weight (Average)	FP Count (Est. Count×Weight)
1. Number of external inputs	20	24	30	24	4	$96 (24 \times 4 = 96)$
2. Number of external outputs	12	14	22	14	5	$70 (14 \times 5 = 70)$
3. Number of external inquiries	16	20	28	20	5	$100 (20 \times 5 = 100)$
4. Number of internal logical files	4	4	5	4	10	$40 (4 \times 10 = 40)$
5. Number of external interface files	2	2	3	2	7	$14 (2 \times 7 = 14)$
Count Total						320

Step 2: Score the 14 Complexity Weighting Factors $\sum(F_i)$ (Table 25.2) Next, each of the **14 complexity factors (F_i)** is rated on a scale from **0 (not important)** to **5 (very important)**:

#	Complexity Factor (F_i)	Assigned Value (0 to 5)
1	Backup and recovery	4
2	Data communications	2
3	Distributed processing	0

4	Performance critical	4
5	Existing operating environment	3
6	Online data entry	4
7	Input transaction over multiple screens	5
8	Master files updated online	3
9	Information domain values complex	5
10	Internal processing complex	5
11	Code designed for reuse	4
12	Conversion/installation in design	3
13	Multiple installations	5
14	Application designed for change	5
	Sum of Complexity Factors $\sum(F_i)$	52

Step 3: Calculate the Value Adjustment Factor and $FP_{estimated}$

Using the Function Point formula:

$$FP_{estimated} = counttotal \times \left[0.65 + 0.01 \times \sum(F_i)\right]$$

1. **Calculate the Adjustment Factor:**

$$0.65 + 0.01 \times \sum(F_i) = 0.65 + (0.01 \times 52) = 0.65 + 0.52 = 1.17$$

2. **Calculate $FP_{estimated}$:**

$$FP_{estimated} = 320 \times 1.17 = 374.4 \approx 375FP$$

Step 4: Translate $FP_{estimated}$ into Effort and Project Cost

Using historical data from the organization for similar systems:

- **Historical Productivity Rate:** $6.5FP/pm$ (Function Points per person-month)
- **Burdened Labor Rate:** $\$8,000\text{ per person-month}$
- **Cost per FP:**
  Cost per FP = $8,000 / 6.5 FP = $1,230 per FP
- **Total Estimated Effort:**

$$Effort = \frac{375FP}{6.5FP/pm} \approx 58person - months$$

- **Total Estimated Project Cost:**
  Total Cost = 375 FP * $1,230/FP = $461,000

25.10 Scheduling

Scheduling a software project is very similar to scheduling any other multitask engineering project, meaning standard project scheduling tools and techniques can be used with minimal changes [Fer14].

- **Work Breakdown Structure (WBS) & Task Networks:**
  1. Project tasks—often called the **Work Breakdown Structure (WBS)**—are defined either for the product as a whole or for individual functions.
  2. **Task Networks** are used to map out the interdependencies among these tasks.
- **3 Key Capabilities of Project Scheduling Tools [Ker17]:**
  1. **Critical Path Identification:** Finds the specific chain of dependent tasks that determines the minimum total duration of the project.
  2. **"Most Likely" Time Estimates:** Uses statistical models to calculate realistic duration estimates for individual tasks.
  3. **Boundary Times:** Calculates the earliest and latest start/finish times that define the allowable time "window" for a task.

25.10.1 Time-Line Charts (Gantt Charts) and Project Tables

How a Schedule is Generated [Toc18]

1. **Inputs:** The planner enters the list of tasks (the WBS) into an automated scheduling tool as a task network or task outline, along with the **effort**, **duration**, **start date**, and **assigned team member(s)** for each task.
2. **Outputs:** The tool generates two complementary views that can be scoped to the whole project, a specific function, or an individual team member:

Scheduling Output	How It Is Structured & Used	Key Visual Elements (Figures 25.6 & 25.7)
1. Time-Line Chart (Gantt Chart) *(Figure 25.6)*	A visual calendar chart showing when tasks occur across weeks (e.g., Concept Scoping for a Word-Processing [WP] product across Weeks 1 to 5).	• **Left Column:** Lists all WBS tasks (I.1.1 Identify needs and benefits down to I.1.8 Create a scope definition). • **Horizontal Gray Bars:** Show the start, end, and duration of each task. • **Overlapping Horizontal Bars:** Indicate **task concurrency** (tasks performed in parallel at the same time on the calendar). • **Black Diamonds:** Mark formal **milestones** (e.g., Milestone: Product statement defined, Milestone: OCI defined, Milestone: Reusable components identified, Milestone: Scope document complete).
2. Project Table *(Figure 25.7)*	A detailed tabular spreadsheet used alongside the Gantt chart to track planned vs. actual progress.	Lists 8 columns for every task and milestone: 1. Work tasks 2. Planned start (e.g., wk1, d1 = Week 1, Day 1) 3. Actual start 4. Planned complete 5. Actual complete 6. Assigned person (e.g., BLS, JPP, MLL, all) 7. Effort allocated (in person-days, e.g., 2 p-d, 1.5 p-d) 8. Notes (e.g., Scoping will require more effort/time)

25.10.2 Tracking the Schedule, Project Control, and Time-Boxing

6 Ways Experienced Managers Track a Project Schedule

A well-built schedule acts as a road map of tasks and milestones that managers track using six techniques:

1. **Periodic Project Status Meetings:** Regular meetings where each team member reports their progress and any problems.
2. **Evaluating Review Results:** Checking the outcomes of all technical reviews held throughout the process.
3. **Checking Formal Milestones:** Verifying whether project milestones (the diamonds in the Gantt chart) have been achieved by their scheduled dates.
4. **Comparing Planned vs. Actual Start Dates:** Checking the project/resource table to see if actual start dates are slipping behind planned start dates.
5. **Informal Practitioner Meetings:** Casual conversations with developers to get their honest, subjective sense of current progress and upcoming problems on the horizon.
6. **Tracking Project Velocity:** In agile projects, measuring how fast the team is clearing items from the user story backlog.

Case Study Summary: SafeHome – Tracking the Schedule

- **The Problem with Vague Milestones:** When reviewing the schedule for the first *SafeHome* increment, Doug notices the team listed milestones like "analysis classes defined" without a clear way to verify completion. When Jamie argues, *"They're done when they're done,"* Doug rejects that as unmeasurable.
- **The Solution (Linking Milestones to Technical Reviews):** Doug requires the team to schedule formal **Technical Reviews (TRs)**. A milestone like an analysis model is only officially "complete" when it successfully passes its technical review.

Exercising Project Control

Project managers use **control** to manage resources, solve problems, and guide staff:

- **When Things Go Well:** If the project is on schedule, within budget, passing reviews, and hitting milestones, management control stays **light**.
- **When Problems Occur:** The manager must step in quickly to diagnose the issue and take corrective action:
    1. Focus **additional resources** on the problem area,
    2. **Redeploy staff**, or
    3. **Redefine the project schedule**.

Time-Boxing (Handling Severe Deadline Pressure) [Jal04]

- **When It Is Used:** When a project faces strict, immovable deadlines and the manager realizes the complete product cannot possibly be finished in full by the deadline. Commonly used alongside **agile incremental process models** (and tracked in tools like Microsoft Project).
- **How Time-Boxing Works Step-by-Step:**
    - Instead of scheduling the whole product at once, a separate schedule is created for each **increment**.
    - Tasks for the increment are scheduled by **working backward from the increment's fixed delivery date**.
    - A strict time "box" is drawn around each task.
    - **The Boundary Rule (+/- 10%):** When a task hits the end of its allocated time box (plus or minus 10%), **work on that task stops immediately and the next task begins**.
- **Why Stopping Work Works (The 90/10 Rule):**
    - Developers often react negatively at first, asking: *"If the work isn't finished, how can we move on?"*
    - In practice, by the time a task hits its time-box boundary, roughly **90% of the task is already finished**.
    - Instead of getting "stuck" perfecting the last **10%** and missing the delivery deadline, that remaining 10% can either be **(1) delayed until the next increment** or **(2) finished later if needed**, ensuring the project keeps moving forward on time.

15.4 Achieving Software Quality

Software quality does not happen by accident—it is the direct result of **good project management** and **solid software engineering practice** working together across **4 broad activities**:

1. **Software engineering methods**
2. **Project management techniques**
3. **Quality control (QC) actions**
4. **Software quality assurance (SQA)**

15.4.1 to 15.4.5 The Building Blocks of Software Quality

Activity / Technique	Role in Achieving Software Quality	Key Actions & Details
1. Software Engineering Methods *(Section 15.4.1)*	Builds quality into the product from the ground up through proper analysis and design.	• **Understand the problem:** Use analysis concepts and methods to gain a complete understanding of the problem to be solved. • **Create a solid design:** Build a comprehensive design that matches the problem, exhibits key quality dimensions/factors, and lays a strong foundation for coding (construction).
2. Project Management Techniques *(Section 15.4.2)*	Prevents poor management decisions from ruining software quality.	Software quality improves when a manager: 1. Uses **estimation** to verify that delivery dates are realistically achievable. 2. Understands **schedule dependencies** and ensures the team resists taking shortcuts. 3. Conducts **risk planning** so unexpected problems do not turn into chaos. 4. Includes explicit techniques for **quality management** and **change management** in the project plan.
3. Machine Learning & Defect Prediction *(Section 15.4.3)*	Identifies error-prone software components early to cut costs and development time [Mun17, Mal16].	• **Defect Prediction Models:** Use statistical techniques to analyze relationships between combinations of software metrics and components with known defects, helping developers quickly spot **defect-prone classes**. • **Machine Learning (ML):** A branch of Artificial Intelligence (AI) where computer programs access data and learn/improve from experience automatically without explicit programming. • **How ML Automates Defect Prediction [Ort17, Li16, Mal16]:** ML systems process large datasets of metrics from both *defective* and *non-defective* components to train and tune **classification algorithms**. Once trained, the model predicts

		defects and assesses quality on future software products (a core task of modern data scientists).
4. Quality Control (QC) *(Section 15.4.4)*	Technical actions that check whether each work product meets its specific quality goals.	• **Model Reviews:** Reviewing analysis and design models to make sure they are complete and consistent. • **Code Inspections:** Inspecting source code to find and fix errors *before* testing begins. • **Testing Steps:** Running a series of tests to uncover errors in **processing logic**, **data manipulation**, and **interface communication**. • **Measurement & Feedback:** Using metrics and feedback to fine-tune the development process whenever a work product misses its quality goals.
5. Quality Assurance (QA) *(Section 15.4.5)*	Establishes the organizational infrastructure and audits the effectiveness of QC actions.	• **Infrastructure:** Supports solid engineering methods, rational project management, and quality control. • **Auditing & Reporting:** Assesses how complete and effective the team's QC actions are. • **Core Goal:** Gives management and technical staff accurate data on product quality so they have insight and confidence that quality actions are working. *(If QA data reveals problems, **management** is responsible for stepping in and allocating resources to fix them.)*

17.2 Elements of Software Quality Assurance (SQA)

Software Quality Assurance (SQA) covers **10 core elements** [Hor03], plus ongoing support quality:

1. **Standards:**
    - Organizations like **IEEE** and **ISO** publish software engineering standards that a team may adopt voluntarily or have imposed on them by customers/stakeholders.
    - *SQA's Job:* Ensure that all adopted standards are actually followed and that every work product conforms to them.
2. **Reviews and Audits:**
    - **Technical Reviews:** A *Quality Control (QC)* activity performed by software engineers for other software engineers to find errors.
    - **Audits:** A special type of review performed by **SQA personnel** to verify that quality guidelines are being followed (for example, auditing the technical review process itself to make sure reviews are being conducted effectively).
3. **Testing:**

- Software testing is a QC function with one primary goal: **to find errors**.
- *SQA's Job:* Ensure that testing is properly planned and efficiently executed so it has the highest chance of finding errors.

4. **Error/Defect Collection and Analysis:**
   - Because you can only improve what you measure, SQA collects and analyzes error and defect data to understand *how* errors are introduced and *which* engineering activities are best at eliminating them.
5. **Change Management:**
   - Unmanaged change is one of the most disruptive forces in software—it causes confusion, and confusion almost always leads to poor quality.
   - *SQA's Job:* Ensure proper change management practices are in place and followed.
6. **Education:**
   - Educating software engineers, managers, and stakeholders is vital for improving practices.
   - *SQA's Job:* Take the lead in **Software Process Improvement (SPI)** and sponsor educational and training programs.
7. **Vendor Management:**
   - Organizations acquire three types of software from external vendors:
     1. **Shrink-wrapped packages:** Off-the-shelf software (e.g., Microsoft Office).
     2. **Tailored shells [Hor03]:** A basic skeletal software structure that is custom-tailored to the buyer's needs.
     3. **Contracted software:** Custom-designed and built from scratch based on the customer's specifications.
   - *SQA's Job:* Suggest specific quality practices for the vendor to follow (when possible) and write strict quality mandates directly into external vendor contracts.
8. **Security Management:**
   - Rising cybercrime and strict privacy laws require policies that protect data at all levels, put firewall protection around mobile apps, and prevent internal tampering.
   - *SQA's Job:* Ensure the right processes and technologies are used to achieve software security.
9. **Safety:**
   - In human-rated systems (like cars or aircraft), hidden software defects can be catastrophic.
   - *SQA's Job:* Assess the impact of potential software failures and initiate steps to reduce safety risks.
10. **Risk Management:**

- While software engineers handle day-to-day risk analysis and mitigation, **SQA** ensures that risk management activities are properly conducted and that risk-related **contingency plans** are established.

- **Software Support Activities:** Beyond these 10 elements, SQA also ensures that post-release support activities—such as **maintenance, help lines, documentation, and user manuals**—are produced with quality as a top priority.

28.3 The CMMI (Capability Maturity Model Integration)

Origins & Core Concept

- Developed by the **Software Engineering Institute (SEI)** in the 1990s as the original Capability Maturity Model (CMM) for Software Process Improvement (SPI), it evolved into the **CMMI** [CMM18].

- The CMMI is a comprehensive **process meta-model** based on the system and software engineering capabilities an organization should possess as it reaches higher levels of process capability and maturity.

- **Goals and Practices Structure:**

  - **Specific Goals:** Define the required characteristics that must exist for activities in a process area to be effective.

  - **Specific Practices:** Break a specific goal down into concrete process-related activities.

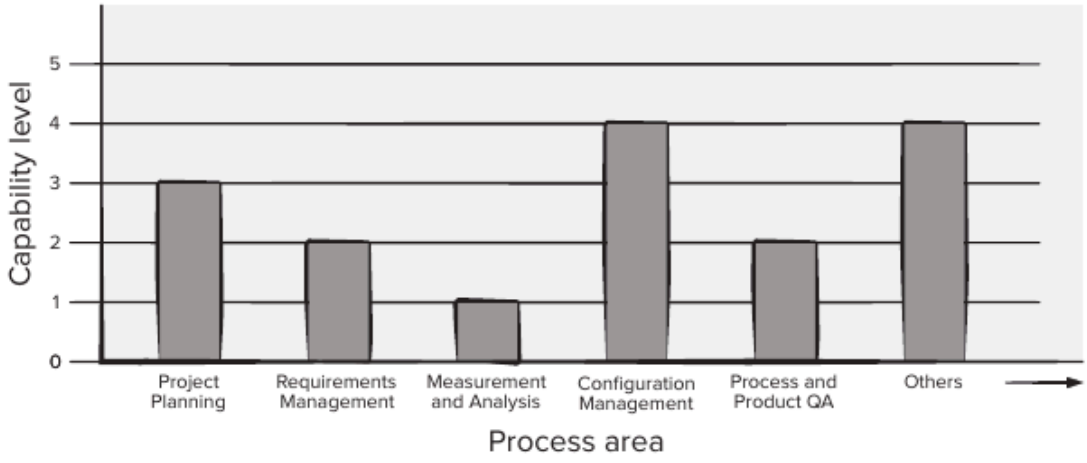
  - **Generic Goals & Practices:** The CMMI also defines **5 generic goals** (and related practices) for every process area, corresponding directly to Capability Levels 1 through 5.

The Two Representations of CMMI: Continuous vs. Staged

1. The Continuous CMMI Model (Capability Levels 0 to 5 — Figure 28.2) The continuous model evaluates individual process areas (such as *Project Planning*, *Requirements Management*, *Measurement and Analysis*, *Configuration Management*, and *Process and Product QA*) across two dimensions: the **Process Area** on the horizontal axis and its **Capability Level (0–5)** on the vertical axis.

FIGURE 28.2

CMMI process area capability profile
Source: Phillips, Mike, "CMMI V1.1 Tutorial," April 9, 2002.



Capability Level	Name	Requirements to Achieve This Level [CMM18]
Level 0	**Incomplete**	The process area is either not performed at all, or fails to achieve all goals and objectives required for Level 1 capability.
Level 1	**Performed**	All **specific goals** of the process area are satisfied. Work tasks needed to produce defined work products are being carried out.
Level 2	**Managed**	Meets all Level 1 criteria **plus**: • All work conforms to an organizationally defined policy. • Staff have adequate resources to do the job. • Stakeholders are actively involved as required. • All work tasks and work products are monitored, controlled, reviewed, and evaluated for adherence to the process description.
Level 3	**Defined**	Meets all Level 2 criteria **plus**: • The process is tailored from the organization's set of standard processes using formal tailoring guidelines. • The team contributes work products, measures, and process-improvement data back to the organization's process assets.
Level 4	**Quantitatively Managed**	Meets all Level 3 criteria **plus**: • The process area is controlled and improved using measurement and **quantitative assessment**. • Quantitative objectives for quality and process performance are established and used to manage the process.
Level 5	**Optimized**	Meets all Level 4 criteria **plus**: • The process area is adapted and optimized using **quantitative (statistical) means** to meet changing customer needs and continually improve process effectiveness.

2. The Staged CMMI Model (Maturity Levels & Required Process Areas — Figure 28.3) In the staged model, an organization achieves an overall **Maturity Level** by satisfying the specific goals and practices for an entire group of process areas:

Maturity Level	**Primary Focus**	**Required Process Areas (Figure 28.3) PNG**
Level 5: Optimizing	Continuous Process Improvement	• Causal analysis and resolution • Organizational innovation and deployment
Level 4: Quantitatively Managed	Quantitative Management	• Quantitative project management • Organizational process performance
Level 3: Defined	Process Standardization	• Technical solution • Verification & Validation • Organizational training • Integrated project management & Integrated teaming • Requirements development • Decision analysis and resolution • Organizational environment for integration & Product integration • Organizational process definition & Organizational process focus • Integrated supplier management • Risk management
Level 2: Managed	Basic Product Management	• Supplier agreement management • Process and product quality assurance • Project planning • Requirements management • Configuration management • Measurement and analysis • Project monitoring and control
Level 1: Performed	*(Base execution)*	*(Informal/ad-hoc execution of tasks)*

The People CMM [CMM18a]

- Because no software process can succeed without talented, motivated people, the **People CMM** defines **5 organizational maturity levels** to improve workforce competence and culture.
- It focuses on continuous improvement in three areas:
    1. **Core competencies:** Generic workforce knowledge.
    2. **Workforce competencies:** Specific software engineering and project management skills.
    3. **Process-related abilities.**

Info Box Summary: The CMMI—Should We, or Shouldn't We?

- **The Debate ("Is CMMI Overkill?"):** Spanning 700+ pages, the detailed CMMI framework is "just right" for organizations building **large, complex systems** involving dozens or hundreds of people over many months or years—provided management is committed and the culture fits standard process models. For smaller or more agile organizations, full CMMI adoption may be too heavy to assimilate.
- **Adopt the Spirit Always:** Even if a team does not adopt every formal CMMI detail, **every software team should embrace the spirit of the CMMI**—planning thoroughly, controlling uniformly, tracking accurately, acting professionally, and focusing on stakeholder needs, engineer skills, and end-product quality.

INFO



Seven Principles of Risk Management

The Software Engineering Institute (SEI) (**www.sei.cmu.edu**) identifies seven principles that "provide a framework to accomplish effective risk management." They are:

Maintain a global perspective. View software risks within the context of a system in which it is a component and the business problem that it is intended to solve.

Take a forward-looking view. Think about the risks that may arise in the future (e.g., due to changes in the software); establish contingency plans so that future events are manageable.

Encourage open communication. If someone states a potential risk, don't discount it. If a risk is proposed in an informal manner, consider it. Encourage all stakeholders and users to suggest risks at any time.

Integrate. A consideration of risk must be integrated into the software process.

Emphasize a continuous process. The team must be vigilant throughout the software process, modifying identified risks as more information is known and adding new ones as better insight is achieved.

Develop a shared product vision. If all stakeholders share the same vision of the software, it is likely that better risk identification and assessment will occur.

Encourage teamwork. The talents, skills, and knowledge of all stakeholders should be pooled when risk management activities are conducted.

Chapter 26: Software Risk Management

26.1 Reactive vs. Proactive Risk Strategies

Feature	Reactive Risk Strategy	Proactive Risk Strategy
Nickname / Philosophy	**"Indiana Jones school of risk management"** [Tho92] (*"Don't worry, I'll think of something!"*—ignoring problems until they strike).	Forward-thinking risk avoidance and contingency planning that reduces **technical debt**.
When It Starts	Only *after* something goes wrong and becomes a real problem.	Long **before** any technical work begins.
How It Works	• At best, monitors likely risks and sets aside reserve resources. • Usually does nothing until a risk strikes, forcing the team into **"fire-fighting mode"** to fix the problem rapidly. • If fire-fighting fails, **"crisis management"** [Cha92] takes over, putting the entire project in severe jeopardy.	1. Identifies potential risks upfront. 2. Assesses their **probability** and **impact**. 3. Ranks them by importance. 4. Creates a plan to **avoid (mitigate)** risks first, plus a **contingency plan** to respond smoothly if unavoidable risks occur.

26.2 Software Risks

2 Universal Characteristics of Every Software Risk [Hig95]

1. **Uncertainty:** The risk may or may not happen—its probability is always **less than 100%** *(Note: A risk that is 100% certain to happen is no longer a risk; it is a **project constraint**)*.
2. **Loss:** If the risk becomes a reality, unwanted consequences or losses will occur.

Classification 1: Project, Technical, and Business Risks

- **1. Project Risks (Threaten the Project Plan)**
    - **Consequence:** If they become real, the project schedule will slip and costs will rise.
    - **Sources:** Budget, schedule, personnel (staffing and organization), resources, stakeholders, and requirements problems—plus **project complexity**, **project size**, and **structural uncertainty**.
- **2. Technical Risks (Threaten Software Quality and Timeliness)**
    - **Consequence:** Implementation becomes difficult or completely impossible because the problem turns out to be harder to solve than expected.

    - **Sources:** Design, implementation, interface, verification, and maintenance problems—plus **specification ambiguity**, **technical uncertainty**, **technical obsolescence**, and unproven **"leading-edge" technology**.

- **3. Business Risks (Threaten Project or Product Viability)**
    - **The Top 5 Business Risks:**
        1. **Market Risk:** Building an excellent product or system that no one actually wants to buy or use.
        2. **Strategic Risk:** Building a product that no longer fits into the company's overall business strategy.
        3. **Sales Risk:** Building a product that the sales team does not understand how to sell.
        4. **Management Risk:** Losing senior management support due to a shift in business focus or a change in leadership personnel.
        5. **Budget Risk:** Losing funding (budgetary) or staffing commitment.

Classification 2: Charette's Risk Categories [Cha89]

1. **Known Risks:** Uncovered by carefully evaluating the project plan, the business/technical environment, and reliable information sources *(e.g., an unrealistic delivery date, lack of documented requirements/scope, or a poor development environment)*.
2. **Predictable Risks:** Extrapolated from past project experience *(e.g., staff turnover, poor communication with the customer, or staff effort getting diluted by ongoing maintenance requests)*.
3. **Unpredictable Risks:** The "joker in the deck"—risks that can and do happen, but are extremely difficult to foresee in advance.

Info Box Summary: SEI's Seven Principles of Risk Management

1. **Maintain a global perspective:** View software risks within the larger system and the business problem it solves.
2. **Take a forward-looking view:** Anticipate future risks (e.g., from software changes) and set up contingency plans early.
3. **Encourage open communication:** Never discount a potential risk raised by anyone (even informally); encourage all stakeholders and users to suggest risks at any time.
4. **Integrate:** Make risk consideration a built-in part of the software process.
5. **Emphasize a continuous process:** Stay vigilant throughout the project, updating known risks and adding new ones as you learn more.
6. **Develop a shared product vision:** When all stakeholders share the same vision, risk identification and assessment improve.
7. **Encourage teamwork:** Pool the talents, skills, and knowledge of all stakeholders when conducting risk management.

26.3.1 Assessing Overall Project Risk

Based on global surveys of experienced project managers [Kei98], the following **11 questions**—listed in order of importance to project success—measure overall project risk:

1. Have top software and customer managers formally committed to support the project?
2. Are end users enthusiastically committed to the project and the system/product to be built?
3. Are requirements fully understood by the software engineering team and its customers?
4. Have customers been involved fully in the definition of requirements?
5. Do end users have realistic expectations?
6. Is the project scope stable?
7. Does the software engineering team have the right mix of skills?
8. Are project requirements stable?
9. Does the project team have experience with the technology to be implemented?
10. Is the number of people on the project team adequate to do the job?
11. Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

- **Rule for Action:** If **any one** of these 11 questions is answered **"No"**, the team must immediately institute **mitigation, monitoring, and management** steps. The overall risk level of the project is directly proportional to how many questions receive a negative answer.

26.4.1 Developing a Risk Table (Risk Projection)

A **Risk Table** (usually built in a spreadsheet for easy sorting) is a simple, step-by-step tool for **risk projection** and prioritization:

Step-by-Step Construction of a Risk Table (Figure 26.1)

FIGURE 26.1 Sample risk table prior to sorting

Risk	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet exceptions	TR	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	

Impact values:
1 – catastrophic
2 – critical
3 – marginal
4 – negligible

2 The risk table can be implemented as a spreadsheet model. This enables easy manipulation and sorting of the entries.

1. **Column 1 (Risk):** List all potential risks (no matter how remote) using risk checklists or risk tables from past projects. *(Agile teams often save money by sharing reusable risk lists across the company.)*
2. **Column 2 (Category):** Assign each risk an abbreviation showing its category:
    - PS = Project Size risk
    - BU = Business risk
    - CU = Customer risk
    - TR = Technology risk
    - DE = Development Environment risk
    - ST = Staff risk
3. **Column 3 (Probability):** Estimate the percentage likelihood (0% to 100%) of each risk occurring.
    - *How to Estimate:* Poll team members in a round-robin fashion until their estimates converge into a consensus, or use advanced AI methods like **fuzzy logic** [Rod16] (which handles incomplete or uncertain information by assigning degrees of truth/falsehood rather than simple true/false values).
4. **Column 4 (Impact):** Evaluate the impact of the risk across **4 risk components—Performance, Support, Cost, and Schedule**—and average them (or take a weighted average) using a 1-to-4 scale:
    - 1 = **Catastrophic**

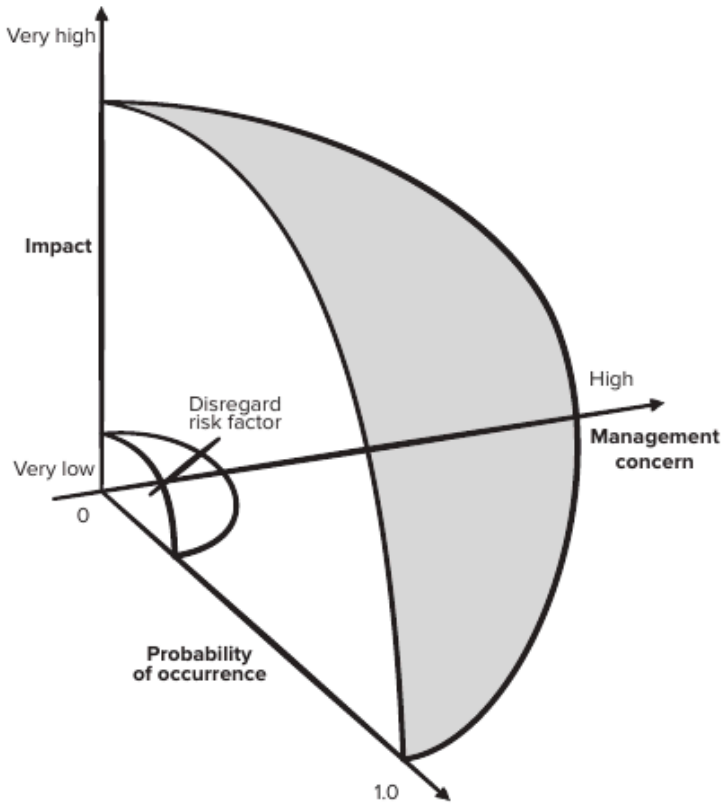
- 2 = **Critical**
- 3 = **Marginal**
- 4 = **Negligible**

5. **Sort & Draw the Cutoff Line (First-Order Risk Prioritization):**
   - Sort the table so **high-probability, high-impact risks** rise to the top and low-probability risks drop to the bottom.
   - The project manager draws a horizontal **cutoff line** across the table:
     - **Above the cutoff line:** Risks that require active management. Column 5 (RMMM) gets a pointer to a **Risk Mitigation, Monitoring, and Management (RMMM) plan** or a **Risk Information Sheet**.
     - **Below the cutoff line:** Lower-priority risks that are set aside.

- **How Probability and Impact Drive Management Concern (Figure 26.2):**

FIGURE 26.2

Risk and management concern



   - A risk with **high impact** but **very low probability** should *not* consume much management time.
   - Management should focus its time on **high-impact risks with moderate-to-high probability** and **low-impact risks with high probability**.

26.6 Risk Mitigation, Monitoring, and Management (RMMM)

An effective risk strategy addresses **3 core activities**—illustrated below using the textbook's example of **High Staff Turnover (r_1)** with a **70% probability ($l_1 = 0.70$)** and **Critical impact (x_1)**:

RMMM Pillar	Purpose & When It Happens	Concrete Actions for "High Staff Turnover (r1)"
1. Risk Mitigation (Avoidance)	Proactive steps taken **before and from the start of the project** to reduce the likelihood or impact of the risk. *(Always the best strategy!)*	1. Meet with current staff to find the root causes of turnover (e.g., poor working conditions, low pay, competitive job market). 2. Fix/mitigate the causes that are under your control *before* the project starts. 3. Once the project starts, assume turnover *will* happen and build continuity habits: • Organize teams so knowledge is widely shared. • Set work product standards so models/documents are written on time. • Conduct **peer reviews** of all work so multiple people stay "up to speed." • Assign a **backup staff member** for every critical technologist.
2. Risk Monitoring	Continuous tracking **as the project proceeds** to see if the risk is becoming more or less likely, *and* checking whether mitigation steps are actually working.	• **Monitor Risk Indicators:** Track team morale/attitude under project pressure, how well the team has "jelled," interpersonal relationships, pay/benefits dissatisfaction, and internal/external job openings. • **Monitor Mitigation Effectiveness:** Check work products carefully to verify that documentation is clear enough for a brand-new developer to step in mid-project and understand it on their own.
3. Risk Management & Contingency Planning	Executed **when mitigation fails and the risk becomes a reality** (e.g., mid-project, several key developers announce they are quitting).	1. Rely on the backups, documentation, and shared team knowledge built during mitigation. 2. Temporarily shift resources and adjust the schedule toward fully staffed features while newly hired replacements get up to speed. 3. Put departing employees into **"knowledge transfer mode"** for their final weeks: stop their regular coding work and have them record video walkthroughs, write

		commentary docs/Wikis, and hold handoff meetings with remaining staff.

4 Essential Management Rules for RMMM

1. **Cost-Benefit Analysis & Risk Exposure (RE):**
    - RMMM steps cost money and time (for example, assigning a backup person for every critical specialist is expensive).
    - Managers must compare the **Risk Exposure (RE)** of a risk against the **cost of mitigating it**:
        - **Rule:** If $RE < Costof RiskMitigation$, **do not mitigate the risk**—simply continue to monitor it.
        - *Example:* If turnover mitigation steps (mostly backup staffing) would increase project cost and duration by **15%**, management may skip the backup step. If they only increase cost by **5%** and duration by **3%**, management will likely implement all of them.
2. **The Pareto 80–20 Rule for Software Risk:**
    - A large project might identify 30 to 40 risks, each with 3 to 7 management steps—too many to manage at once.
    - Experience shows that **80% of overall project risk** (80% of the potential for project failure) comes from just **20% of the identified risks**.
    - Therefore, only the **critical 20%** (the risks with the highest Risk Exposure / priority) are included in the formal RMMM plan.
3. **Using Gamification for Risk Monitoring [Det11, Ped14, Baj11, Bri13]:**

SAFEHOME



Gamification and Risk Management

The scene: Doug Miller's office, prior to the initiation of the *SafeHome* software project.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: I'd like to spend some time brainstorming ideas for getting all developers on board with monitoring and mitigating the risks we identified for the *SafeHome* project.

Jamie: I thought it was your job to keep track of the project?

Doug: It is, but the development team members are able to see potential problems much quicker than I can, since they're working in the trenches.

Vinod: Working is right. We already have a lot to do to keep this project moving forward . . .

Doug: Exactly, we don't have time to fix problems that could have been prevented by addressing risks early.

Jamie: Okay . . . so what are you thinking?

Doug: I'm thinking since you guys like games, that may be a way to make risk monitoring more like a game than a project task.

Jamie: You know, my buddy—he works at another company—told me about something called "gamification." Sounds like what you're talking about, Doug.

    - **What Gamification Is:** Applying game design elements (points, badges, public leaderboards) in non-game settings to boost developer motivation and attention on tasks like quality and risk compliance.

- How It Works (and the *SafeHome* Case Study): Because developers working "in the trenches" spot problems faster than managers, teams can use automated data collection or project dashboards to award points/badges on a Google Doc leaderboard to the first person who spots a risk trigger value. Badges can be cashed in for perks like getting first pick of a user story in the next sprint or cash rewards for saving rework costs.

Doug: Yeah, gamification is growing in popularity as a compliance tool in software process areas like quality assurance and risk management.

Vinod: So how can we do this?

Doug: I was thinking that most of the triggers for our risk mitigation tasks are based on the metrics we already have in the project management dashboard. It might just be a question of encouraging developers to check the dashboard on a regular basis.

Jamie: Are we going to use "snoopware" to track the number of times people check?

Doug: Maybe not. Perhaps we can reward the first person who reports a trigger value that indicates something's wrong.

Jamie: My buddy mentioned that they use leader boards to encourage competition. We can do a simple leader board as a Google doc spreadsheet.

Vinod: Some games have badges. We might create badges for the leader board.

Doug: Would people work for badges?

Jamie: They might if they could cash badges in for rewards.

Doug: What kind of rewards?

Jamie: Maybe something like getting first pick on a user story to develop in the current sprint. Or a cash reward if a person's actions save the company a lot of rework costs.

Doug: Let me think some more on this and let you know what I come up with for rewards and a point system for the leader board. We'll meet again tomorrow. For now, think about the most important risks we should be monitoring when we start our first sprint.

- **Important Caution [Bri13]:** Make sure developers cannot "game the system" by intentionally injecting problems into the project just to earn badges for fixing them!

4. **Post-Delivery Risk: Software Safety and Hazard Analysis:**
   - Risks do not end at delivery—software failure in the field can cause severe post-release risks.
   - **Software safety and hazard analysis** [Fir15, Har12a, Lev12] are SQA activities that identify and assess potential hazards early in the software process so engineers can build specific software design features to eliminate or control those hazards before the system fails in the field.

Unit 6: Software Testing (Chapter 8 — K.K. Aggarwal & Yogesh Singh, 3rd Ed.)

8.1 Testing Fundamentals (Strategic Introduction to Testing)

1. What is Testing?

Many people hold common—but **incorrect**—beliefs about how to define software testing:

1. *"Testing is the process of demonstrating that errors are not present."* (Incorrect)
2. *"The purpose of testing is to show that a program performs its intended functions correctly."* (Incorrect)

3. *"Testing is the process of establishing confidence that a program does what it is supposed to do."* (Incorrect)

Why These Definitions Are Wrong (Glenford Myers' Principle):

When you test a program with the goal of showing that it works (that it has no errors), you subconsciously steer toward test data that will succeed rather than fail. Furthermore, it is impossible to prove that software is 100% free of errors.

Correct Definition of Testing:

"Testing is the process of executing a program with the intent of finding errors."

A *successful test* is one that uncovers an as-yet-undiscovered error.

2. Why Should We Test?

- **High Cost of Untested Software:** Although software testing is an expensive activity itself (often consuming 30% to 40% or more of total project effort), launching software without testing leads to costs that are potentially much higher than the cost of testing—especially in systems where **human safety** is involved (e.g., medical devices, aircraft, nuclear control).
- **Early Error Detection Saves Money:** In the Software Development Life Cycle (SDLC), **the earlier errors are discovered and removed, the lower the cost of their removal**. Fixing a fault after release costs exponentially more than catching it during development.

3. Who Should Do the Testing?

- **The Psychological Problem for Developers:** Testing requires finding errors in software. However, it is psychologically and practically very difficult for a software developer to point out errors in their **own creation**:
    - Developers view their program as their "own child" and naturally adopt a **constructive** mindset rather than the **destructive** mindset required to break the software.
    - Any misunderstanding of the SRS that caused the developer to write defective code will also cause them to write flawed test cases.
- **Organizational Separation:** Therefore, most organizations make a strict distinction between the **development phase** and the **testing phase** by making **different people (Independent Test Groups / ITG)** responsible for system and validation testing, while developers primarily perform initial **unit testing**.

4. What Should We Test? (Why Complete / Exhaustive Testing is Impossible)

Ideally, we would like to test a program's response to **every possible input** (both valid and invalid) and **every possible execution path**. However, Aggarwal & Singh demonstrate through two classic examples that **complete (exhaustive) testing is mathematically impossible**:

A. Exhaustive Input Testing Example (Two 8-Bit Integers)

- Suppose a tiny program requires just **two 8-bit integers** as inputs.
- Total possible combinations of valid inputs $= 2^8 \times 2^8 = 256 \times 256 = 65,536$ combinations.

- If it takes just **1 second** to execute one set of inputs, testing all valid combinations will take:

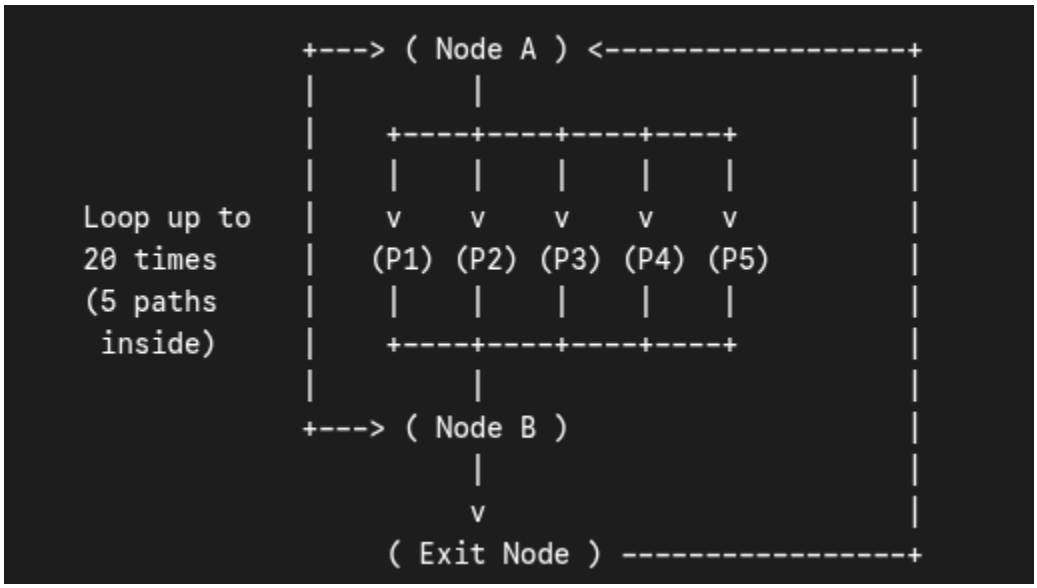
  - $\frac{65{,}536 seconds}{3600 seconds/hour} \approx 18 hours$

- In real life, programs have far more than two inputs, data types are 32-bit or 64-bit, and **invalid inputs** have virtually infinite combinations. Thus, exhaustive input testing is impossible.

B. Exhaustive Path Testing Example (Control Flow Graph — Figure 8.1)

Consider a small control flow graph with a loop that repeats up to **20 times** and contains **5 independent branching paths** inside the loop body:

- **Figure 8.1: Control Flow Graph with a 20-Iteration Loop**



- **Total Number of Possible Execution Paths:**
  Total Paths = 5^20 + 5^19 + 5^18 + ... + 5^1 = 10^14 (or 100 trillion paths)

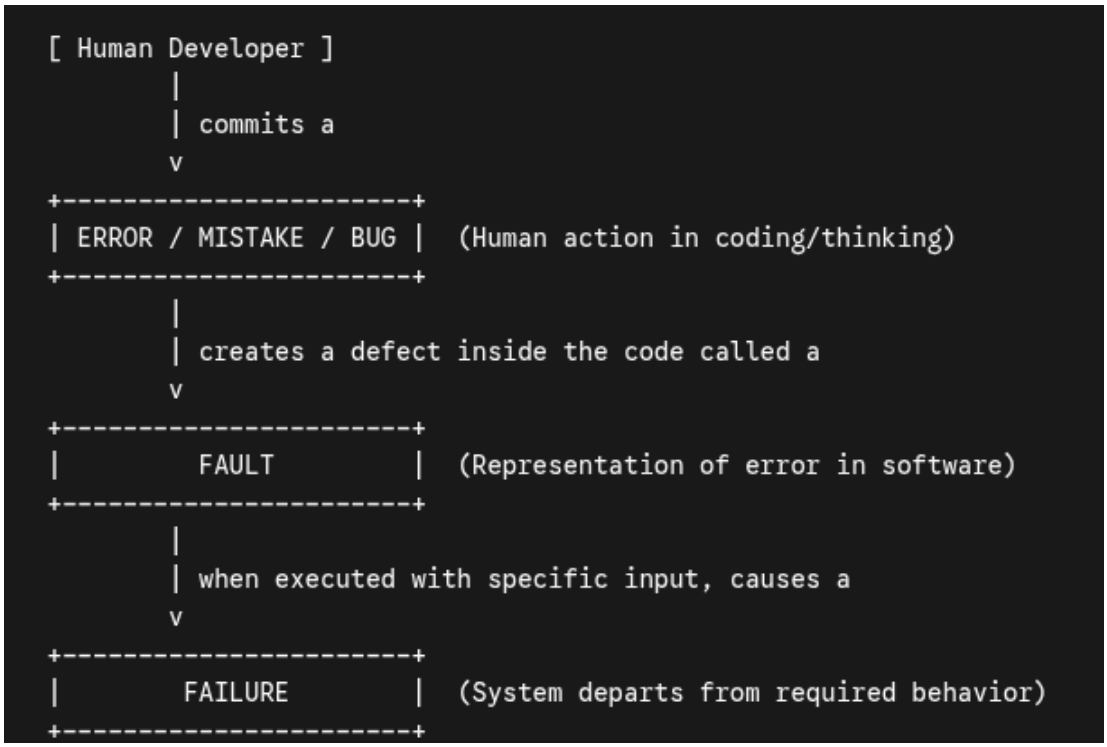
- **Time Required to Test All Paths:** If it takes only **5 minutes** to design, execute, and verify one test path, executing every path would take approximately **1 billion years**!

- **Conclusion:** Because neither exhaustive input testing nor exhaustive path testing is possible, we need a **strategic approach** (Black-Box and White-Box testing techniques) to select a small, optimal subset of test cases with the highest probability of finding errors.

8.2 Some Terminologies

1. Error, Mistake, Bug, Fault, and Failure

People make errors, which lead to faults in code, which cause failures during execution:



Term	Exact Definition in Aggarwal & Singh
Error / Mistake	People make **errors**. A good synonym is **mistake**. It is a human action that produces an incorrect result (e.g., misunderstanding a requirement or typing + instead of -).
Bug	When developers make mistakes while **coding**, we call these mistakes **"bugs"**.
Fault	A **fault** is the representation of an error in the software, where *representation* is the mode of expression, such as narrative text, Data Flow Diagrams, hierarchy charts, or source code. In short, a fault is the physical defect residing inside the code/document.
Failure	A **failure** occurs when a fault executes. It is the departure of the operational system's output/behavior from its specified requirements. *(Note: Every fault does not necessarily cause a failure—if the faulty line of code is never executed, no failure occurs.)*

2. Test, Test Case, and Test Suite

- **Test and Test Case:** Used interchangeably as synonyms in practice. A **test case** is a triplet $[I, S, O]$, where:
    - I = **Input data** given to the system.
    - S = **State** of the system at which the input data is given (preconditions).
    - O = **Expected Output** of the system (postconditions).
- **Test Suite:** The **set of all test cases** with which a given software product is to be tested.

3. Verification and Validation (V&V)

$$Testing = Verification + Validation$$

Feature	Verification	Validation
IEEE / Book Definition	The process of evaluating a system or component to determine whether the products of a **given development phase** satisfy the conditions imposed at the **start of that phase**.	The process of evaluating a system or component **during or at the end of the development process** to determine whether it satisfies the **specified customer requirements**.
Boehm's Classic Question	*"Are we building the product right?"*	*"Are we building the right product?"*
What It Checks	Checks conformance of a phase's output against its immediate predecessor document (e.g., Design against SRS; Code against Design).	Checks whether the actual working software meets the true needs and expectations of the customer/end user.
Nature of Activities	Primarily **static testing** activities: inspections, reviews, walkthroughs, and desk-checking (does not require code execution).	Primarily **dynamic testing** activities: executing the actual code with test cases (Unit, Integration, System, Acceptance testing).

4. Alpha, Beta, and Acceptance Testing (Overview)

(Covered in full detail in Section 8.6 below)

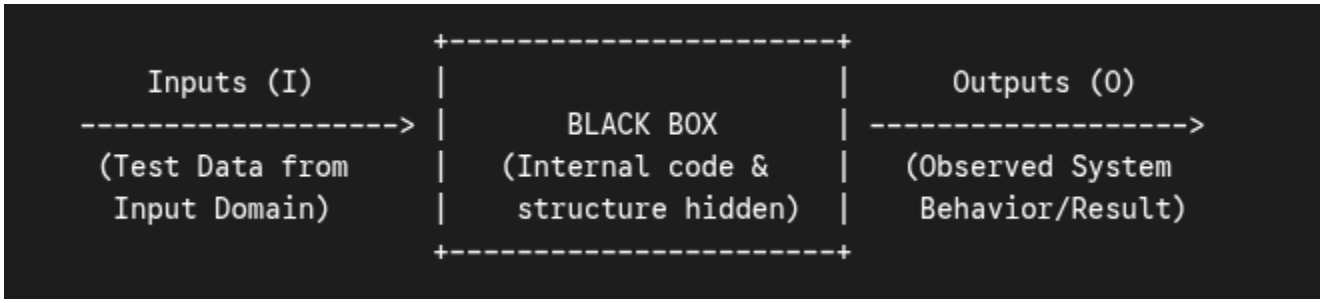
- **Acceptance Testing:** Conducted when software is custom-developed for a **specific customer** (bespoke software). The customer runs a series of tests to verify the system meets all contract requirements before accepting delivery.
- **Alpha (α) Testing:** Conducted at the **developer's site** by a representative group of end users (or customer) in a **controlled environment** with the developer present looking over their shoulder to record errors and usage problems.
- **Beta (β) Testing:** Conducted at **one or more customer sites** by the end users of the software in a **real, live environment** that cannot be controlled by the developer (the developer is *not* present). Users report all problems back to the developer at regular intervals.

8.3 Functional Testing (Black-Box Testing)

Concept of Functional / Black-Box Testing (Figure 8.4)

- Also called **Behavioral Testing**, functional testing treats the software program as a **"Black Box"**.

- The tester has **zero knowledge of the internal structure or source code** of the program. Instead, test cases are designed strictly from the **functional specifications (SRS)** by examining the **input domain** and **output domain**.



- **Figure 8.4: Black-Box (Functional) Testing Model**

8.3.1 Boundary Value Analysis (BVA)

Why Boundary Value Analysis Works:

Experience shows that programmers make the highest number of errors at the **boundaries (edges) of the input domain** rather than in the "center" (for example, using < instead of <=, or off-by-one loop indexing errors).

Core Idea of BVA (Figure 8.5):

Suppose a program has two input variables x and y, where:

$$a \leq x \leq bandc \leq y \leq d(e.g., 100 \leq x \leq 300and100 \leq y \leq 300)$$

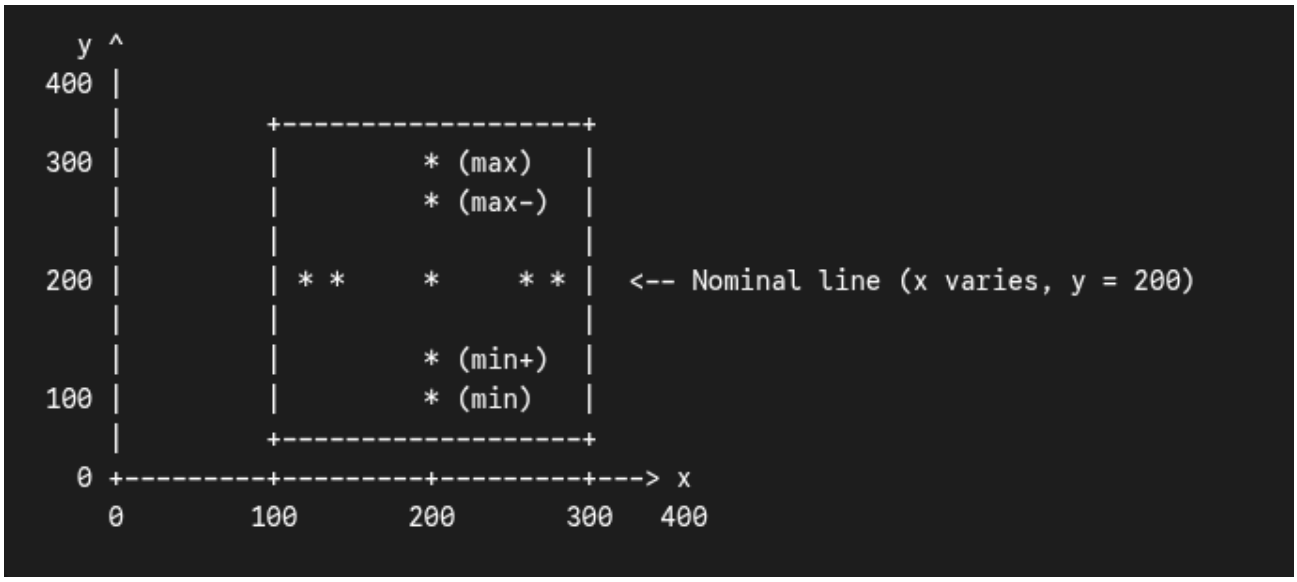
Instead of picking random points, BVA selects **5 test values for each variable**:

1. **Minimum value (**min**)** $\rightarrow e.g., 100$
2. **Just above minimum (**min^{+}**)** $\rightarrow e.g., 101$
3. **Nominal (average/middle) value (**nom**)** $\rightarrow e.g., 200$
4. **Just below maximum (**max^{-}**)** $\rightarrow e.g., 299$
5. **Maximum value (**max**)** $\rightarrow e.g., 300$

Single Fault Assumption:

Standard BVA is based on the **"Single Fault Assumption"** from reliability theory, which states that failures are rarely the result of two or more simultaneous faults. Therefore, to generate BVA test cases:

- **Hold all variables except one at their nominal (average) value**, and let the remaining variable step through its 5 boundary values ($min, min^{+}, nom, max^{-}, max$).
- Repeat this for each of the n input variables.



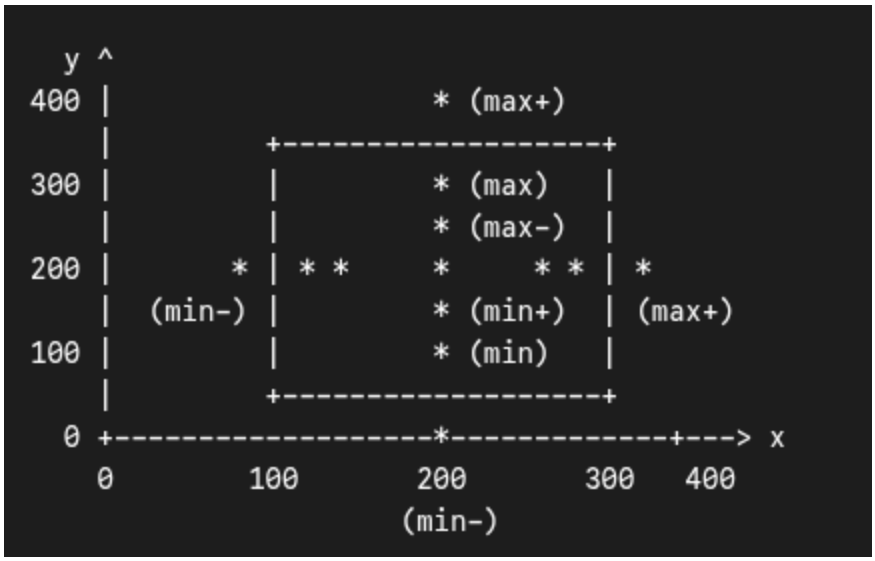
- **Figure 8.5: Boundary Value Analysis (BVA) test cases for two variables x and y with boundaries $[100, 300]$ each (Total = $4(2) + 1 = 9$ test cases)**
- **Formula for Number of BVA Test Cases:** For a program of n **variables**, standard BVA generates:

$$TotalBVATestCases = 4n + 1$$

The 3 Extensions of Boundary Value Analysis

1. Robustness Testing (Figure 8.6)

- **Concept:** An extension of BVA where we also want to see what happens when the extreme boundaries are **exceeded**—testing slightly *outside* the legitimate input domain using two additional invalid values per variable:
    - **Just below minimum (min^-)** → $e.g., 99$
    - **Just above maximum (max^+)** → $e.g., 301$
- Each variable now takes **7 values**: $(min^-, min, min^+, nom, max^-, max, max^+)$, while holding all other variables at their nominal value.

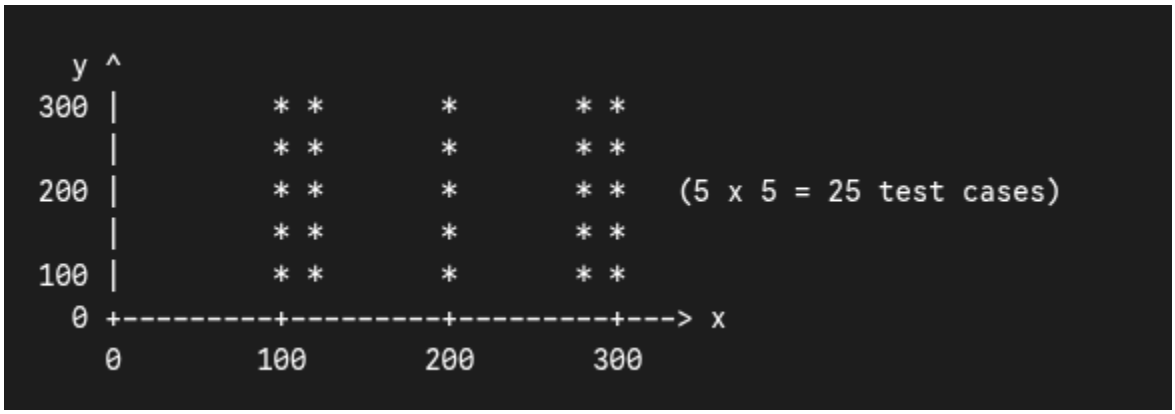


- **Figure 8.6: Robustness test cases for two variables x and y with range $[100, 300]$ each (Total = $6(2) + 1 = 13$ test cases)**
- **Formula for Number of Robustness Test Cases:** For n **variables**:

$$TotalRobustnessTestCases = 6n + 1$$

2. Worst-Case Testing (Figure 8.7)

- **Concept:** Rejects the "Single Fault Assumption" and asks what happens when **more than one variable has an extreme value at the same time**.
- We take the Cartesian product of the **5 BVA values** ($min, min^+, nom, max^-, max$) for all n variables.



- **Figure 8.7: Worst-Case test cases for two variables x and y (Total = $5^2 = 25$ test cases)**
- **Formula for Number of Worst-Case Test Cases:** For n **variables**:

$$TotalWorst - CaseTestCases = 5^n$$

3. Robust Worst-Case Testing (Figure 8.8)

- **Concept:** Combines Robustness Testing and Worst-Case Testing by taking the Cartesian product of all **7 values** ($min^-, min, min^+, nom, max^-, max, max^+$) across all n variables.
- **Formula for Number of Robust Worst-Case Test Cases:** For n **variables**:

$$TotalRobustWorst - CaseTestCases = 7^n$$

Summary Table of BVA Formulas (High-Yield Exam Reference)

Technique	Values Tested per Variable	Assumption	Formula for n Variables	Test Cases for n=2	Test Cases for n=3
1. Boundary Value Analysis (BVA)	5 ($min, min^+, nom, max^-, max$)	Single fault	$4n + 1$	9	13
2. Robustness Testing	7 ($min^-, min, min^+, nom, max^-, max, max^+$)	Single fault + Invalid bounds	$6n + 1$	13	19
3. Worst-Case Testing	5 (all combinations)	Multiple faults	5^n	25	125
4. Robust Worst-Case Testing	7 (all combinations)	Multiple faults + Invalid bounds	7^n	49	343

Textbook Solved Examples for Boundary Value Analysis

Example 8.1: Quadratic Equation Problem

- **Problem Statement:** Consider a program to determine the nature of roots of a quadratic equation $ax^2 + bx + c = 0$. Its inputs are three positive integers a, b, c in the range $[0, 100]$. Outputs may be: *Not a quadratic equation*, *Real roots*, *Imaginary roots*, *Equal roots*, or *Invalid input*.
    - Rules: Not quadratic if $a = 0$; Real roots if $(b^2 - 4ac) > 0$; Imaginary roots if $(b^2 - 4ac) < 0$; Equal roots if $(b^2 - 4ac) = 0$.
- **Test Case Counts ($n = 3$ variables):**
    - **BVA Test Cases** $= 4(3) + 1 = 13 testcases$ (using values $0, 1, 50, 99, 100$ for each variable while holding the other two at 50).
    - **Robustness Test Cases** $= 6(3) + 1 = 19 testcases$ (adding -1 and 101 for each variable, as shown below):
        - Varying a (with $b = 50, c = 50$): $a =$ -1 (Invalid input), 0 (Not quadratic), 1 (Real roots), 50 (Imaginary), 99 (Imaginary), 100 (Imaginary), 101 (Invalid input).
        - Varying b (with $a = 50, c = 50$): $b =$ -1 (Invalid), 0 (Imaginary), 1 (Imaginary), 99 (Imaginary), 100 (Equal roots, since $100^2 - 4(50)(50) = 0$), 101 (Invalid).
        - Varying c (with $a = 50, b = 50$): $c =$ -1 (Invalid), 0 (Real), 1 (Real), 99 (Imaginary), 100 (Imaginary), 101 (Invalid).
    - **Worst-Case Test Cases** $= 5^3 = 125 testcases$.

Example 8.2: Previous Date Problem

- **Problem Statement:** A program takes a date (month, day, year) as input with ranges $1 \leq month \leq 12$, $1 \leq day \leq 31$, and $1900 \leq year \leq 2025$, and outputs the previous day's date.
- **BVA Test Cases ($n = 3 \Rightarrow 4(3) + 1 = 13$ test cases):**
    - Nominal values: $month = 6$ (June), $day = 15$, $year = 1962$.
    - Cases 1–5 (Varying year at $1900, 1901, 1962, 2024, 2025$ with $month = 6, day = 15$) → 14 June, <year>.
    - Cases 6–9 (Varying day at $1, 2, 30, 31$ with $month = 6, year = 1962$) → 31 May 1962, 1 June 1962, 29 June 1962, and Invalid date (since June 31 is invalid).
    - Cases 10–13 (Varying month at $1, 2, 11, 12$ with $day = 15, year = 1962$) → 14 January 1962, 14 February 1962, 14 November 1962, 14 December 1962.

Example 8.3: Triangle Classification Problem

- **Problem Statement:** A program takes three positive integers x, y, z in the range $(0, 100]$ (i.e., $1 \leq x, y, z \leq 100$) representing the side lengths of a triangle and outputs: *Equilateral*, *Isosceles*, *Scalene*, or *Not a triangle*.
- **BVA Test Cases ($n = 3 \Rightarrow 4(3) + 1 = 13$ test cases):**
    - Using boundary values $\{1, 2, 50, 99, 100\}$ with nominal value 50:
    - When two sides are held at 50 and the third side is $1, 2$, or 99 → *Isosceles*; when all three are 50 → *Equilateral*; when two sides are 50 and the third is 100 → *Not a triangle* (since 50 + 50 is not greater than 100).

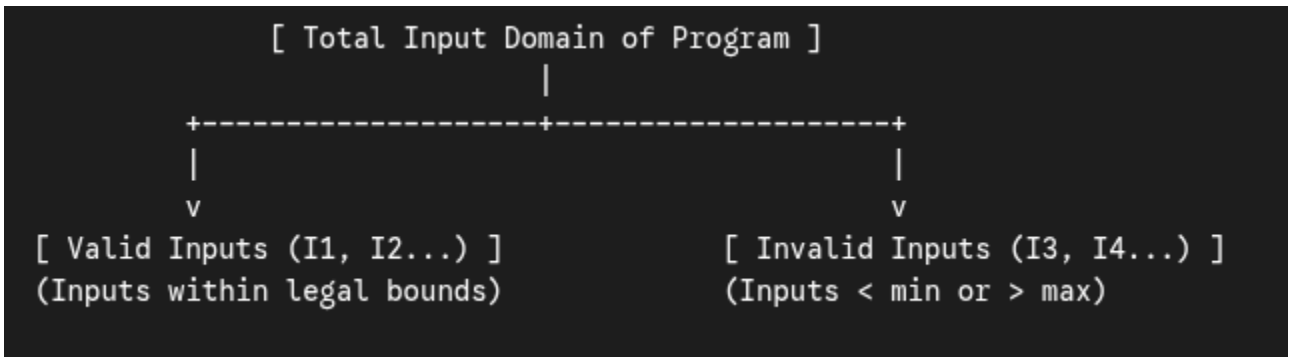
8.3.2 Equivalence Class Testing (Equivalence Partitioning)

Core Concept:

- Boundary Value Analysis tests edges well, but generates redundant tests or misses specific functional categories.
- **Equivalence Class Testing** divides (partitions) the input domain (and output domain) of a program into a finite number of **Equivalence Classes (EC)** such that **every test case inside a given class behaves the exact same way**—meaning if one test case in an equivalence class catches a bug, any other point in that same class would catch it too.
- Thus, we only need to pick **one representative value (usually the nominal/middle value)** from each equivalence class!

Two Steps to Design Equivalence Class Test Cases:

1. **Identify the Equivalence Classes:** Partition both the **Input Domain** (into **Valid Input Equivalence Classes** and **Invalid Input Equivalence Classes**) and the **Output Domain** (into distinct output classes).
2. **Select One Representative Test Case per Class:** Pick one test input from each identified equivalence class.



Example: Equivalence Classes for the Triangle Problem ($1 \leq x, y, z \leq 100$)

Method 1: Input Domain Partitioning

For each variable (x, y, z), we partition its range into 1 valid class and 2 invalid classes:

- **Valid Input Classes:**

- $I_1 = \{x: 1 \leq x \leq 100\}$
- $I_2 = \{y: 1 \leq y \leq 100\}$
- $I_3 = \{z: 1 \leq z \leq 100\}$

- **Invalid Input Classes:**
  - $I_4 = \{x: x < 1\}, I_5 = \{x: x > 100\}$
  - $I_6 = \{y: y < 1\}, I_7 = \{y: y > 100\}$
  - $I_8 = \{z: z < 1\}, I_9 = \{z: z > 100\}$

- This yields **1 valid test case** $(50, 50, 50)$ and **6 invalid test cases** (one for each invalid class while keeping the other two variables at 50, e.g., $(0, 50, 50)$, $(101, 50, 50)$, etc.).

Method 2: Output Domain Partitioning

Because the program produces 4 distinct valid/functional outputs, we also partition based on the output domain:

- O1 = {x, y, z : Equilateral triangle with sides x, y, z} -> Test case: (50, 50, 50)
- O2 = {x, y, z : Isosceles triangle with sides x, y, z} -> Test cases: (50, 50, 60), (50, 60, 50), (60, 50, 50)
- O3 = {x, y, z : Scalene triangle with sides x, y, z} -> Test case: (50, 60, 70)
- O4 = {x, y, z : x, y, z do not form a triangle} -> Test cases: (100, 50, 50), (50, 100, 50), (50, 50, 100)
- **Limitation of BVA & Equivalence Partitioning:** Neither method tests **combinations of interdependent input conditions** well (which is why Decision Tables and Cause-Effect Graphing exist for complex logical dependencies).

8.4 Structural Testing (White-Box Testing — Up to Page 408 & 8.4.2)

Concept of Structural / White-Box / Glass-Box Testing:

- Structural testing is the **complementary approach** to functional (black-box) testing.
- Here, the tester looks **inside the "box"**—examining the **internal logical structure, control paths, conditions, and source code** of the program to design test cases.
- Why is Black-Box testing alone not enough?
  1. Black-box testing cannot discover **unintended extra code** (e.g., a hidden logic bomb or dead code) that isn't mentioned in the SRS.
  2. Certain logical branches or error-handling paths in the code may never be triggered by boundary or equivalence inputs unless we explicitly trace the code structure.

8.4.1 Path Testing (Basis Path Testing)

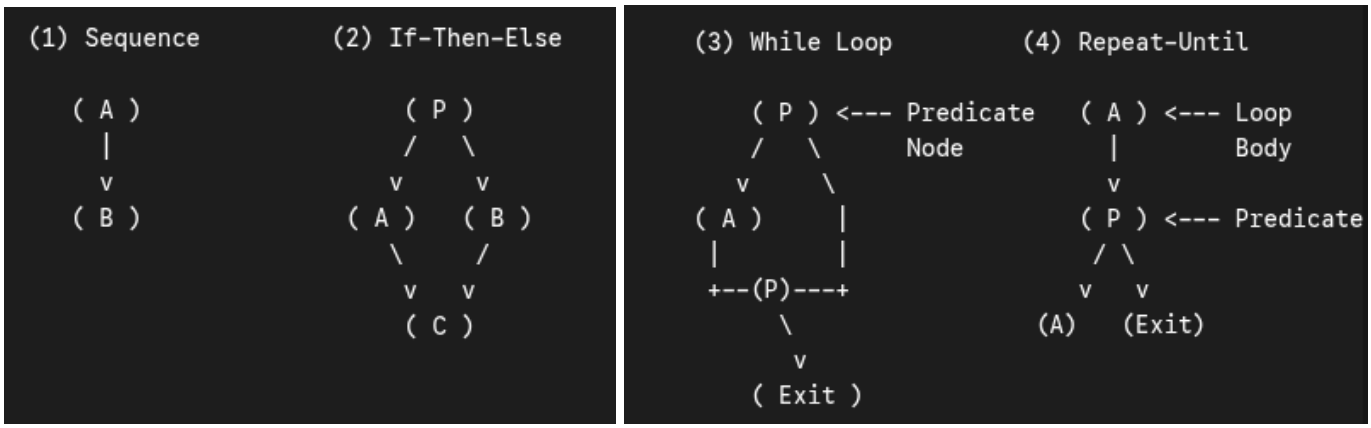
Path testing is a structural testing technique introduced by Tom McCabe that exercises the **control flow structure** of a program to ensure that every independent execution path (and therefore every statement and branch) is executed at least once.

The 4 Steps of Path Testing:

1. Draw the **Flow Graph (Control Flow Graph)** from the program's source code.
2. Reduce the Flow Graph into a **DD Path Graph (Decision-to-Decision Path Graph)**.
3. Calculate the **Cyclomatic Complexity** $V(G)$ to find the exact number of independent paths.
4. Identify the **Basis Set of Independent Paths** and design one test case for each path.

1. Flow Graph (Control Flow Graph) Notations

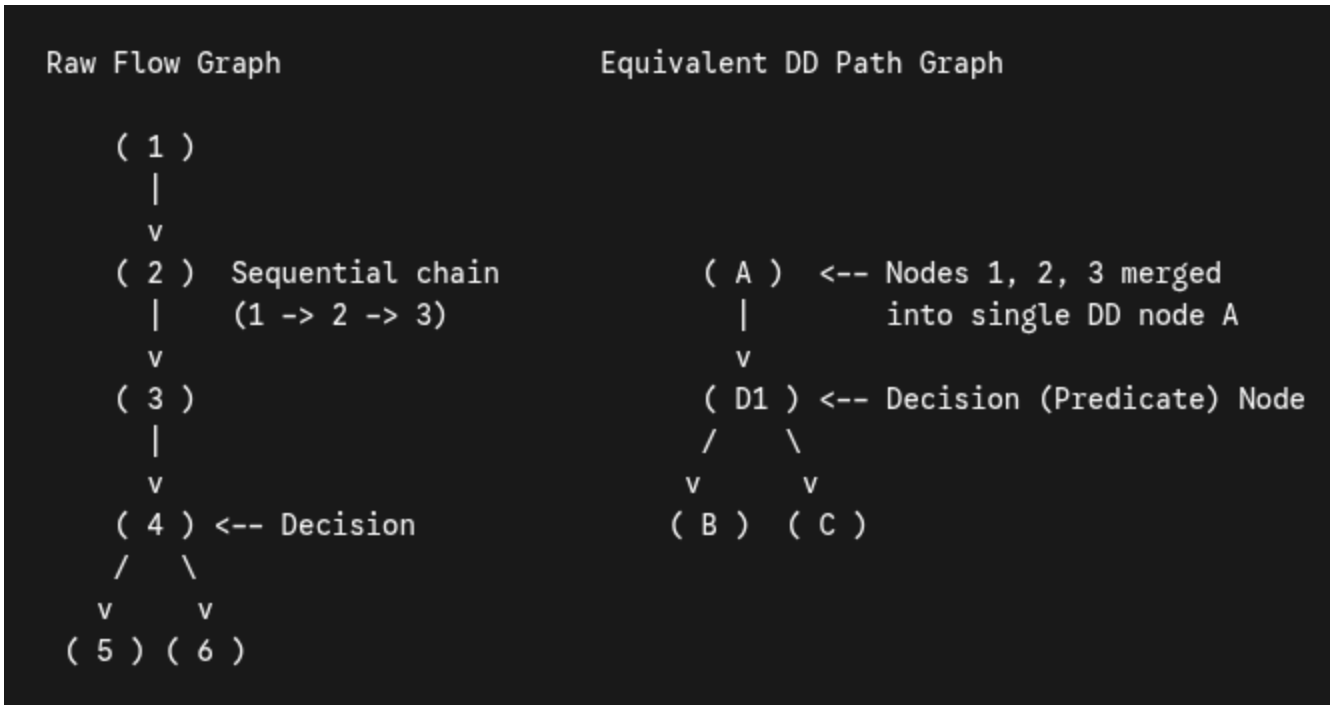
A **Flow Graph** is a directed graph $G = (V, E)$ consisting of **nodes** (circles representing one or more procedural program statements) and **edges/links** (directed arrows representing flow of control):



- **Predicate Node:** Any node that contains a **condition** (like an if, while, or case check) and therefore has **two or more edges leaving it**.

2. DD Path Graph (Decision-to-Decision Path Graph)

- In a raw Flow Graph, there are often long chains of sequential nodes (e.g., nodes $1 \to 2 \to 3 \to 4$) where control simply flows straight through with no branching decisions.
- A **DD Path Graph** simplifies the Flow Graph by **merging (collapsing) all sequential chains of nodes into a single node**, leaving only the decision (predicate) nodes, junction nodes, and combined sequential blocks.



3. Independent Paths (Basis Set)

- **Path:** Any sequence of nodes and edges starting from the **Entry (Start) node** and ending at the **Exit (Terminal) node**.
- **Independent Path:** Any path through the DD Path Graph that introduces **at least one new edge (or new set of processing statements/conditions)** that has not been traversed in any previously defined independent path.
- **Basis Set:** The complete set of independent paths for a graph. Testing every path in the basis set guarantees **100% statement coverage** and **100% branch coverage**.

8.4.2 Cyclomatic Complexity

What is Cyclomatic Complexity?

Developed by **Thomas J. McCabe (1976)**, **Cyclomatic Complexity** $V(G)$ is a graph-theoretic software metric that provides a quantitative measure of the **logical complexity** of a program.

- In the context of **Basis Path Testing**, the value of $V(G)$ gives the **upper bound on the number of independent paths** in the basis set—meaning it tells you the **exact minimum number of test cases** required to guarantee that every statement in the program is executed at least once!

The 3 Formulas to Calculate Cyclomatic Complexity $V(G)$

Given a strongly connected flow graph or DD Path Graph G with e **edges**, n **nodes**, P **connected components** (for a single standalone program/module, $P = 1$), and π **predicate (decision) nodes**:

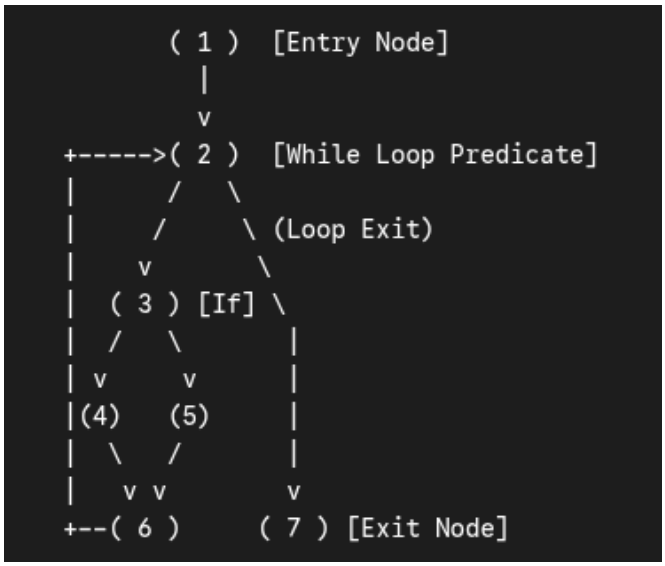
Method	Formula	Variable Definitions & Rules

Method 1: Edges & Nodes Formula	$V(G) = e - n + 2P$ *(For a single module where $P = 1$: $V(G) = e - n + 2$)*	• e = Total number of **edges** (arrows) in the flow graph. • n = Total number of **nodes** (circles) in the flow graph. • P = Number of connected components ($P = 1$ for a single program). *(Note: If you add an imaginary edge from the Exit node back to the Entry node to make it strongly connected, the formula becomes $e - n + P$.)*
Method 2: Predicate Nodes Formula	$V(G) = \pi + 1$	• π = Number of **predicate (decision) nodes** enclosed in the graph. • **Important Rule for Multi-Way Decisions (Switch/Case or Compound AND/OR):** A predicate node must only have *two* outgoing edges. If a decision node has k outgoing branches ($k > 2$), it contributes $k - 1$ to the count of π. Similarly, a compound condition if (A and B) counts as **2** predicate nodes.
Method 3: Regions Formula	$V(G) = NumberofRegions$	• A planar flow graph divides the 2D plane into **enclosed areas (regions)** bounded by edges and nodes, **plus 1 outer (unbounded) region** surrounding the graph. • $TotalRegions = EnclosedRegions + 1$.

Worked Example of Flow Graph, Cyclomatic Complexity, and Independent Paths

Consider a program segment represented by the following DD Path Graph:

Step 1: Count Graph Elements



- Number of nodes (n) $= 7$ (Nodes 1, 2, 3, 4, 5, 6, 7)
- Number of edges (e) $= 8$:
  - $(1 \rightarrow 2)$, $(2 \rightarrow 3)$, $(2 \rightarrow 7)$, $(3 \rightarrow 4)$, $(3 \rightarrow 5)$, $(4 \rightarrow 6)$, $(5 \rightarrow 6)$, $(6 \rightarrow 2)$
- Number of predicate nodes (π) $= 2$ (Node 2 and Node 3)
- Number of regions $= 3$ (1 region inside the $3 - 4 - 6 - 5$ diamond, 1 region inside the $2 - 3 - 6$ loop, plus 1 outer unbounded region)

Step 2: Calculate Cyclomatic Complexity $V(G)$ Using All 3 Methods

1. **By Edges & Nodes:**

$$V(G) = e - n + 2P = 8 - 7 + 2(1) = 3$$

2. **By Predicate Nodes:**

$$V(G) = \pi + 1 = 2 + 1 = 3$$

3. **By Regions:**

$$V(G) = 2enclosedregions + 1outerregion = 3$$

Step 3: Identify the $V(G) = 3$ Independent Paths (Basis Set)

- **Path 1:** $1 \rightarrow 2 \rightarrow 7$ *(Exits loop immediately)*
- **Path 2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 2 \rightarrow 7$ *(Takes left branch of If)*
- **Path 3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 7$ *(Takes right branch of If)*

Properties of Cyclomatic Complexity $V(G)$ (Aggarwal & Singh)

1. $V(G) \geq 1$ (Every program has at least 1 path).
2. $V(G)$ is the **maximum number of independent paths** in graph G.
3. Inserting or deleting functional (sequential) statements to G **does not** change $V(G)$.
4. G has only one path if and only if $V(G) = 1$ (meaning zero predicate nodes).
5. **Threshold Guideline:** McCabe suggested that the cyclomatic complexity of any single module should **not exceed 10** ($V(G) \leq 10$). If $V(G) > 10$, the module is too complex, error-prone, and difficult to test, and should be decomposed into smaller sub-modules.

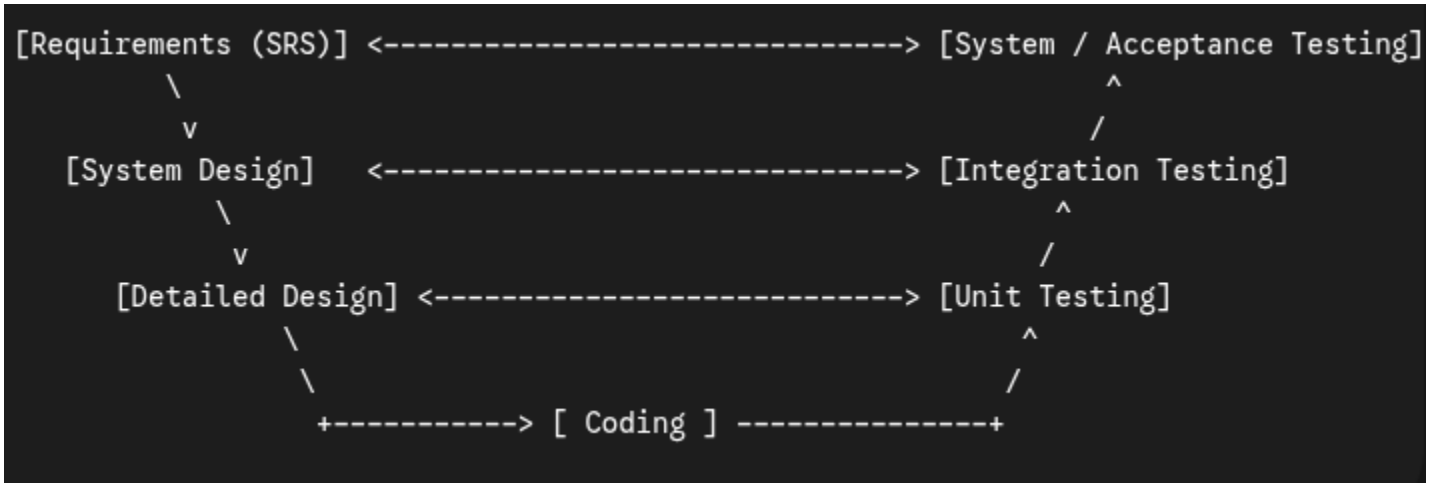
Quick Comparison: Functional (Black-Box) vs. Structural (White-Box) Testing

Feature	Functional (Black-Box) Testing	Structural (White-Box) Testing
Knowledge of Internal Code	Not required (internal code is hidden).	Required (tests are derived directly from source code structure).
Basis for Test Cases	Software Requirements Specification (SRS).	Control flow graph, logic branches, and internal paths.
Primary Techniques	Boundary Value Analysis (BVA), Robustness, Worst-Case, Equivalence Class Partitioning.	Basis Path Testing, Cyclomatic Complexity, Statement/Branch Coverage.
Detects Missing Functions?	**Yes**—easily catches requirements mentioned in the	**No**—only tests code that is actually written; cannot see missing SRS features.

	SRS that were forgotten in code.	
Detects Extra / Unintended Code?	**No**—if undocumented extra code exists, black-box tests won't look for it.	**Yes**—every branch and path in the code is mapped and exercised.

8.5 Levels of Testing (Strategic Approach to Software Testing)

Software is tested progressively in **3 main levels**—moving from small individual modules up to the complete integrated system (often visualized alongside the **V-Shaped Model** or **Spiral of Testing**):



- **Visual Mapping: Development Phases vs. Corresponding Levels of Testing**

1. **Unit Testing:** Tests each individual module/component in isolation.
2. **Integration Testing:** Tests the interfaces and interactions as modules are combined.
3. **System Testing:** Tests the complete, integrated system as a whole against the SRS.

1. Unit Testing

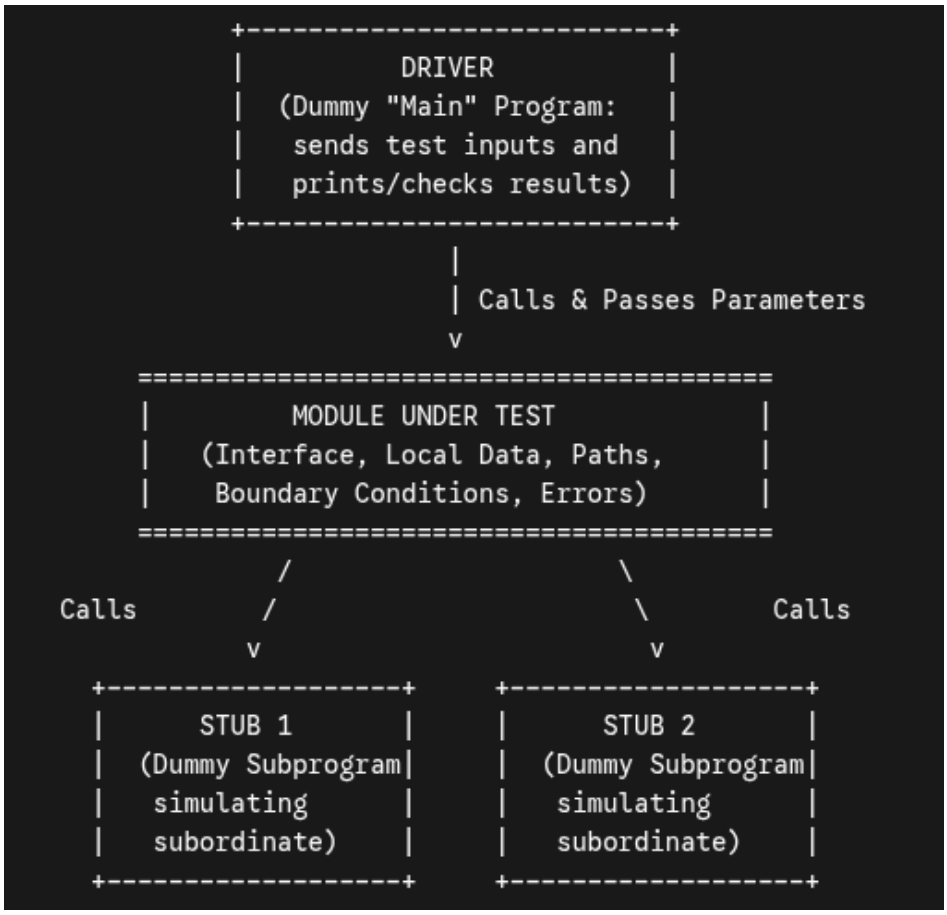
- **Purpose:** Unit testing focuses on the smallest unit of software design—the **individual module, procedure, function, or class**—to uncover errors in:
    1. Module interface (input parameters and return values)
    2. Local data structures
    3. Boundary conditions
    4. Independent execution paths (Basis Paths)
    5. Error-handling paths
- **Why Do Unit Testing First?**
    1. The size of a single module is small enough that we can locate and fix errors very easily.

2. Confusing interactions of multiple errors across widely different parts of the software are eliminated.

3. Primarily uses **White-Box (Structural) Testing** techniques and is usually performed by the programmer who wrote the module.

Drivers and Stubs (The Unit Test Environment)

Because a single module is not a standalone program, it cannot run by itself—it is called by a higher-level module and calls lower-level sub-modules. Therefore, to test a Module Under Test in isolation, we must write temporary **"throwaway" dummy code** called **Drivers** and **Stubs**:



- **Visual Diagram: Unit Testing Environment Using Driver and Stubs**
- **Driver:** A dummy "main program" that sits **above** the module under test. It accepts test case data, passes it as parameters to the module under test, and prints or evaluates the returned results.
- **Stub:** A dummy subprogram that sits **below** the module under test, replacing a subordinate module that the module under test calls. A stub does minimal data manipulation, prints verification of entry, and returns a canned/hardcoded output back to the module being tested.

2. Integration Testing

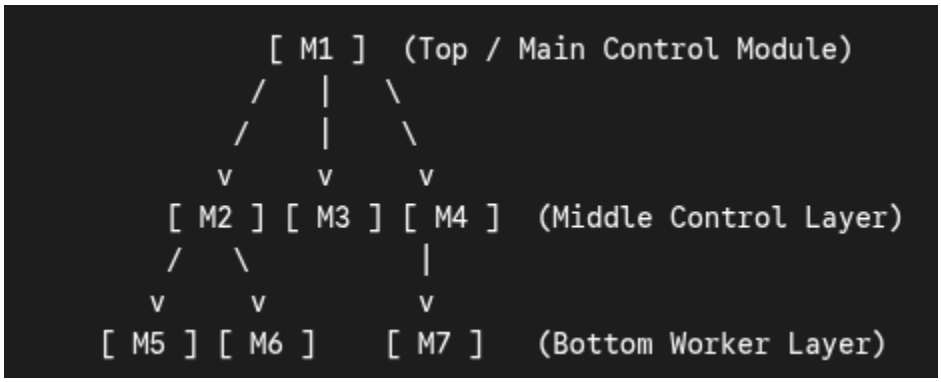
Even if every individual module passes unit testing 100%, modules frequently fail when put together!

- **Why Modules Fail During Integration:**
    - Data can be lost across an interface (parameter mismatch).

    - One module may have an unintended **adverse side effect** on another module (e.g., modifying shared global data).
    - Sub-functions, when combined, may not produce the desired major function.
    - Individually acceptable **imprecision (rounding errors)** may get magnified to unacceptable levels when modules are chained together.
- **Purpose:** Integration testing is the systematic technique for constructing the program structure while simultaneously conducting tests to uncover **errors associated with interfacing**.

The 4 Integration Testing Strategies

Consider the following module hierarchy chart:



Strategy	How It Works	Pros & Cons
1. Big-Bang Integration *(Non-Incremental)*	All modules are tested individually first, and then **all modules are combined at once** and tested as a single whole program.	• **Chaos in Practice:** A terrible approach for large programs! When a failure occurs, it is nearly impossible to isolate *which* module caused it, and fixing one bug introduces new ones.
2. Top-Down Integration *(Incremental)*	Starts with the **Main control module (M1)**, replacing all its subordinates (M2, M3, M4) with **Stubs**. One by one (using either Depth-First or Breadth-First order), stubs are replaced with real modules and re-tested.	• **Pros:** Verifies major top-level control and decision points early; requires **no Drivers**. • **Cons:** Requires writing many **Stubs**, and because low-level processing (M5, M6, M7) is stubbed out at first, real data doesn't flow upward until late in testing.
3. Bottom-Up Integration *(Incremental)*	Starts at the **lowest level (atomic worker modules M5, M6, M7)**. Low-level modules are grouped into builds (clusters), and a **Driver** is written to coordinate test input/output for each cluster. Then drivers are removed and replaced by actual	• **Pros:** Low-level data processing is thoroughly tested right away; requires **zero Stubs**. • **Cons:** Requires writing **Drivers** for every cluster, and the working top-level program (M1) does not

	higher-level modules (M2, M4, then M1).	exist as a whole until the very last module is added.
4. Sandwich (Hybrid) Integration *(Incremental)*	Combines Top-Down and Bottom-Up simultaneously: • **Upper layers** are integrated **Top-Down** (using stubs). • **Bottom worker layers** are integrated **Bottom-Up** (using drivers). • Both meet at a chosen **middle target layer** (M2, M3, M4).	• **Pros:** Combines the advantages of both approaches; ideal for large software projects with multiple sub-trees.

Regression Testing (Essential Part of Incremental Integration):

Every time a new module is added during integration, the software changes—new data paths are established, new I/O may occur, and new control logic is invoked. These changes may cause side effects that **break functions that previously worked**.

Regression Testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

3. System Testing

Once all modules are integrated, **System Testing** verifies that all system elements (software, hardware, databases, people, and external interfaces) mesh properly and achieve overall system function and performance as specified in the **SRS**.

Major Types of System Testing:

1. **Recovery Testing:** Forces the software to fail in a variety of ways (e.g., power loss, hardware crash, network disconnection) and verifies that **recovery is properly performed**—checking automatic re-initialization, checkpointing, data recovery, and restart within a specified Mean Time To Repair (MTTR).
2. **Security Testing:** Attempts to verify that protection mechanisms built into a system will actually protect it from **improper penetration** (hackers, unauthorized users, disgruntled employees). The tester plays the role of an attacker trying to bypass passwords, overwhelm the system, or access restricted data.
3. **Stress Testing:** Executes a system in a manner that demands resources in **abnormal quantity, frequency, or volume** (e.g., generating 10 times the normal interrupt rate, flooding maximum memory/disk, or exceeding maximum simultaneous users) to see how the system behaves under extreme load and whether it fails gracefully (*"sensitivity testing"* is a variation that searches for specific data combinations within valid bounds that cause sudden performance degradation).
4. **Performance Testing:** Tests the **run-time performance** of software within the context of an integrated system (measuring response times, throughput, CPU utilization, and memory usage under normal and peak workloads). Often paired with stress testing.

8.6 Validation Testing

Overview & Purpose

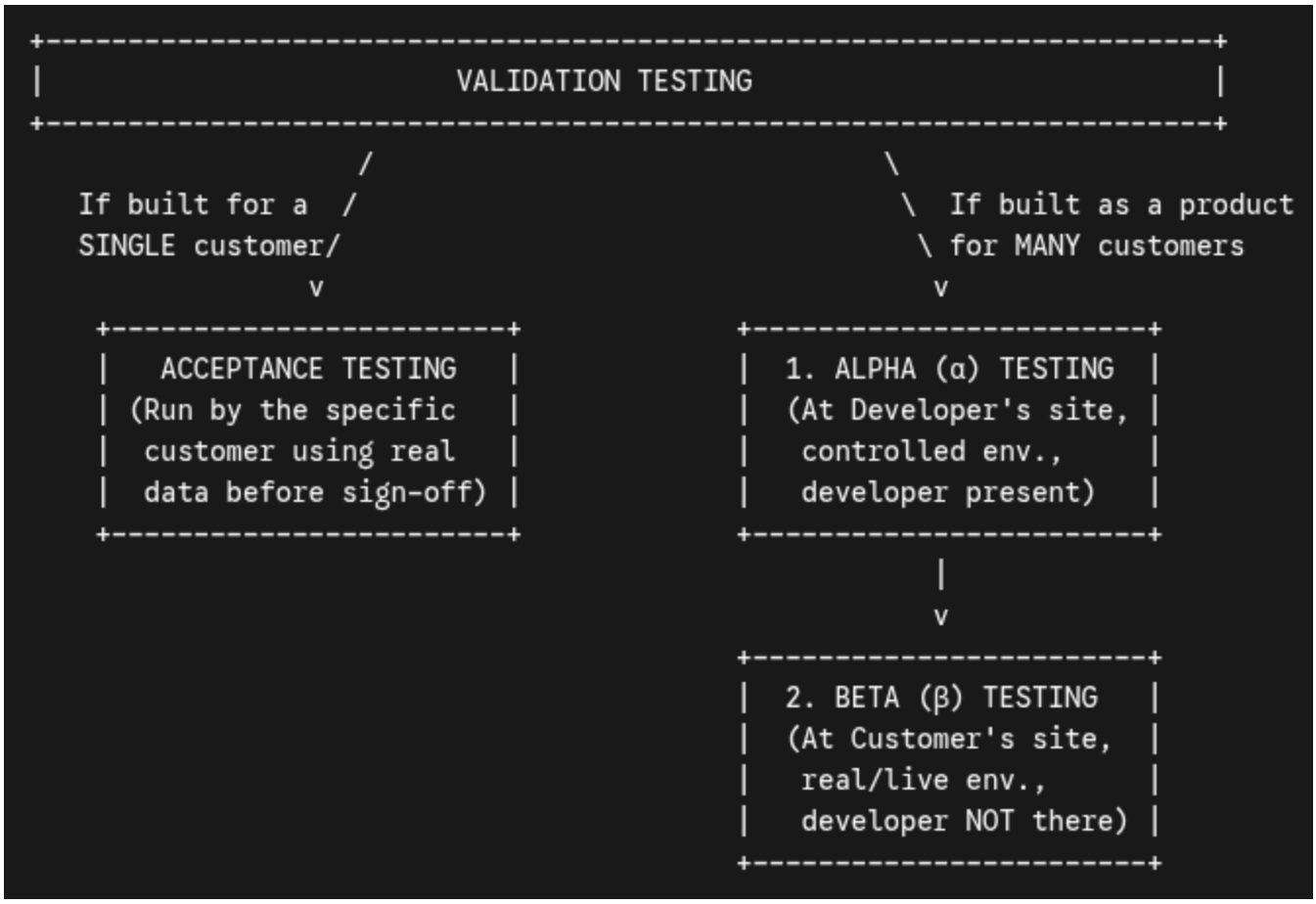
- At the culmination of integration testing, the software is completely assembled as a package and all interface errors have been uncovered and corrected. At this point, **Validation Testing** begins.
- **Definition:** Validation testing succeeds when the software functions in a manner that can be **reasonably expected by the customer**.

1. Validation Test Criteria & Configuration Review

- **Validation Criteria (From the SRS):** Reasonable expectations are defined in the **Software Requirements Specification (SRS)**, which contains a *Validation Criteria* section that forms the basis for a black-box testing approach to demonstrate conformity with all functional, behavioral, and performance requirements.
- **Two Possible Outcomes for Every Validation Test Case:**
    1. The function or performance characteristic conforms to specification and is **accepted**.
    2. A deviation from specification is uncovered and a **deficiency list** is created (which must be resolved with the customer before delivery).
- **Configuration Review (Audit):** An important element of the validation process Whose purpose is to ensure that **all elements of the software configuration** (source code, user manuals, maintenance documentation, support guides, installation scripts) have been properly developed, cataloged, and have the necessary detail to support the maintenance phase of the software life cycle.

2. Alpha, Beta, and Acceptance Testing (Detailed Comparison)

When software is built, it is impossible for a developer to foresee how the customer will actually use the program in practice. Therefore, end-user validation is carried out through **Acceptance Testing** (for custom/contract software) or **Alpha and Beta Testing** (for product software built for many customers):



Feature	Alpha (α) Testing	Beta (β) Testing	Acceptance Testing
When Used	When software is built as a commercial product to be used by **many customers**.	Conducted **after Alpha testing** for software built for **many customers**.	When software is custom-developed for **one specific customer** under contract.
Where Conducted	At the **Developer's site**.	At **one or more Customer sites**.	At the **Customer's site** (or agreed testing environment).
Who Performs It	A representative group of **end users / customers**.	The actual **end users** of the software.	The **Customer** (and their end users).
Is the Developer Present?	**Yes**—the developer "looks over the shoulder" of the user and records errors and usage problems.	**No**—the developer is **not present**.	Usually conducted by the customer (often with developer support).

Environment	**Controlled environment** (controlled by the developer).	**Real, live environment** (cannot be controlled by the developer).	**Customer's operational environment** using real customer data.
Error Reporting & Goal	Immediate recording and fixing of bugs found during guided sessions.	Customer records all problems (real or imagined) and reports them to the developer at regular intervals so the developer can make final modifications before commercial release.	Enables the customer to validate all requirements; often spans a few days to several weeks to uncover cumulative errors that might degrade the system over time before final contract sign-off.